Facultad de Ingeniería

Carrera de Ingeniería de Sistemas e Informática

Tesis

Optimización del Desarrollo de Software con GitHub Copilot: Análisis Comparativo de Productividad y Calidad en una Empresa de Lima 2024

Autores

Bach. Pizarro Romero, Angelo Patrick

Bach. Paredes Orihuela, Jose Luis

para obtener el Título Profesional de

Ingeniero de Sistemas e Informática

Docentes

Asesor Metodólogo: Marcos Teodoro Yerren Huima

Asesor Temático: Karem Menacho Navarrete

Lima – Perú

Enero; 2025

TURNITIN

19% Similitud general

El total combinado de todas las coincidencias, incluidas las fuentes superpuestas, para ca...

Fuentes principales

17% Fuentes de Internet
6% Publicaciones
8% Trabajos entregados (trabajos del estudiante)

Marcas de integridad

N.° de alertas de integridad para revisión

No se han detectado manipulaciones de texto sospechosas.

Los algoritmos de nuestro sistema analizan un documento en profundidad para buscar inconsistencias que permitirían distinguirlo de una entrega normal. Si advertimos algo extraño, lo marcamos como una alerta para que pueda revisarlo.

Una marca de alerta no es necesariamente un indicador de problemas. Sin embargo, recomendamos que preste atención y la revise.

DEDICATORIA

Dedico esta investigación a mis padres y abuelos, quienes con su apoyo incondicional han sido pilares fundamentales en mi desarrollo profesional. Sus valores y constante respaldo han hecho posibles mis logros. También agradezco profundamente a mis amigos y a mi enamorada por estar siempre a mi lado, brindándome su apoyo y contribuyendo de distintas maneras al cumplimiento de mis objetivos, los agradezco de todo corazón.

Angelo Patrick Pizarro Romero

Dedico esta investigación a mi madre y a mi hermana, quienes han sido mi mayor fuente de amor, apoyo y motivación. Gracias a su respaldo incondicional, su paciencia y su fe en mí, he logrado superar los desafíos que se presentaron en este camino. Este logro es tan suyo como mío, y les agradezco profundamente por estar siempre a mi lado, brindándome su fuerza y confianza en cada paso.

Jose Luis Paredes Orihuela

AGRADECIMIENTO

Ante todo, agradezco a Dios por darme buena salud y guiarme por el buen camino, también agradezco profundamente a mis docentes por su apoyo constante y dedicación, así como a mis padres y abuelos, quienes me brindaron su respaldo incondicional y siempre me motivaron a seguir adelante. Finalmente, me reconozco a mí mismo por mi perseverancia y por no rendirme ante los desafíos, lo que me permitió alcanzar este logro.

Angelo Patrick Pizarro Romero

Ante todo, agradezco a Dios por darme salud, fortaleza y guía en este camino. Expreso mi más profundo agradecimiento a mi madre y a mi hermana, quienes con su amor, apoyo incondicional y constantes palabras de aliento han sido mi mayor inspiración. Gracias por estar siempre a mi lado, impulsándome a seguir adelante incluso en los momentos más difíciles. Este logro también es suyo, porque sin su respaldo nada de esto habría sido posible.

Jose Luis Paredes Orihuela

ÍNDICE DE CONTENIDO

DEDICATORIA I

AGRADECIMIENTO II

RESUMEN XI

ABSTRACT XII

CAPÍTULO I: PROBLEMA DE INVESTIGACIÓN 1

1.1 Planteamiento y Formulación del Problema 1

1.1.1. Descripción de la Realidad Problemática 1

1.1.2. Problema General 3

1.1.3 Problemas Específicos 3

1.2 Objetivos 3

1.2.1 Objetivo General: 3

1.2.2 Objetivos Específicos: 3

1.3 Justificación 4

1.3.1 Teórica 4

1.3.2 Práctica 4

1.3.3 Factibilidad 5

1.3.4 Relevancia 5

1.4 Viabilidad de la Investigación 5

1.5 Alcance y Limitaciones 6

1.5.1 Alcances 6

1.5.2 Limitaciones 7

1.6 Hipótesis y Variables 7

1.6.1 Hipótesis General: 7

1.6.2 Hipótesis Especifica: 7

1.6.3 Variable dependiente 8

1.6.4 Variable Independiente 8

CAPÍTULO II. MARCO TEÓRICO 9

2.1 Estado del Arte 9

2.2 Antecedentes Internacionales y Nacionales 14
2.3 Bases Teóricas 16
2.3.1 Introducción al Desarrollo de Software 16
2.3.1.1 Definición del Desarrollo de Software 16
2.3.1.2 Proceso de Desarrollo de Software 17
2.3.1.3 Modelo de Desarrollo de Software 17
2.3.2 Herramientas de Asistencia en el Desarrollo de Software 19
2.3.2.1 Automatización y Herramientas de Codificación Asistida 19
2.3.2.2 Historia y Evolución de las Herramientas de Asistencia de Código 20
2.3.3 GitHub Copilot 21
2.3.3.1 Descripción 21
2.3.3.2 Funcionamiento 22
2.3.3.3 Ventajas y Limitaciones 23
2.3.4 Comparación de Método de Desarrollo de Software 24
2.3.5 La Productividad y la Calidad en el Desarrollo de Software 25
2.3.5.1 Definición de Productividad en el Desarrollo de Software 25
2.3.5.2 Indicadores de Calidad del Código 25
2.3.5.3 Pruebas Unitarias y Tasas de Resolución de Errores 26
2.3.6 Eficiencia en Detección y Resolución de Errores 26
2.3.6.1 Concepto de Detección de Bugs y Errores 26
2.3.6.2 Impacto de GitHub Copilot en la Detección de Errores 27
2.3.7 Revisión de Estudios Previos 27
2.3.8 Método Cuasiexperimental 29
2.3.9 Variable Independiente: Uso de IA GitHub Copilot 30
2.3.9.1 Asistencia en la Generación de Código 30
2.3.9.2 Reducción de Tareas Repetitivas con GitHub Copilot 30
2.3.9.3 Frecuencia de Uso de IA GitHub Copilot 30
2.3.10 Variable Dependiente: Productividad y Calidad de Desarrollo de Software 31
2.3.10.1 Teoría de la Productividad en el Desarrollo de Software 31

2.3.10.2 Calidad en el Desarrollo de Software 31
2.3.11 Calidad del Código 33
2.3.12 Eficiencia en la Detección de Bugs 34
2.3.13 Efectividad en Pruebas Unitarias 34
2.4 Marco Conceptual 35
2.4.1 Inteligencia Artificial (IA) 35
2.4.2 GitHub Copilot 35
2.4.3 Productividad en el Desarrollo de Software 36
2.4.4 Calidad del Código 36
2.4.5 Automatización de Pruebas 37
2.4.6 Beneficios de Utilizar Herramientas de IA 37
2.5 Marco Normativo y Legal 37
2.6 Teoría General de Sistemas (TGS) 38
CAPÍTULO III. METODOLOGÍA DE LA INVESTIGACIÓN 41
3.1 Diseño de Investigación 41
3.1.1 Diseño de la Investigación 41
3.1.2 Tipo de Investigación 41
3.1.3 Nivel de la Investigación 42
3.1.4 Enfoque de la Investigación 43
3.1.5 Población 43
3.1.6 Muestra 44
3.1.7 Operacionalización de Variables 45
3.1.7.1 Variable 1 45
3.1.7.2 Variable 2 46
3.2 Instrumentos de Investigación / Herramientas 47
3.2.1. Técnicas 47
3.2.2. Instrumentos 48
3.2.3. Validez 50
3.2.4. Confiabilidad 51

3.2.5. Análisis de Recolección de Datos 52
3.3. Metodología de la Implementación de la Solución 53
3.3.1. Selección de la Metodología 53
3.3.2. Desarrollo de la Metodología 54
3.4. Metodología Para la Medición de Resultados de la Implementación 56
3.4.1. Metodología de Medición 56
3.4.2. Métrica de la Variable Dependiente 56
3.5. Cronograma de Trabajo 57
3.6. Costos Directos 58
3.6.1. Costos Materiales 58
3.6.2. Costos de Personal 59
3.7. Costos Indirectos 59
3.8. Costos Fijos 60
3.9. Costos Variables 60
3.10. Presupuesto 61
3.10.1. Resumen Total de Costos 61
CAPÍTULO IV: DESARROLLO DE LA SOLUCIÓN 62
4.1. Propuesta de la Solución 62
4.1.1. Fase de Planificación 62
4.1.1.1. Identificación de Actores del Sistema. 62
4.1.1.2. Identificación de Roles del Proyecto. 63
4.1.1.3. Requerimientos 64
4.1.1.4. Identificación de Tablas, Actores y Flujos 67
4.1.2. Fase de Estimación 70
4.1.2.1. Backlog. 70
4.1.2.2. Planificación y Estimación de Tareas 72
4.1.3. Fase de Implementación 72
4.1.4. Fase de revisión y retrospectiva 92
4.1.5. Fase de Lanzamiento 94

4.2 Medición de la Solución 95
4.2.1. Análisis Descriptivo 95
4.2.1.1. Análisis de Pre-Test. 96
4.2.1.2. Análisis de Pos-Test. 107
4.2.2. Resultados 124
4.2.2.1. Resultados estadísticos descriptivos 124
4.2.2.1. Resultados Estadísticos Inferenciales 127
CAPÍTULO V: CONCLUSIONES Y RECOMENDACIONES 141
5.1. Discusión y Conclusiones 141
5.1.1. Discusión 141
5.1.2. Conclusiones 142
5.2. Recomendaciones 144
REFERENCIA BIBLIOGRAFICA 146
ANEXOS 155
ANEXO 1: PERMISO DE AUTORIZACION DE EMPRESA 155
ANEXO 2: MATRIZ DE CONSISTENCIA 156
ANEXO 3: CAPTURAS USO DE COPILOT - PROYECTO API CROSSELING FRONT 157
ANEXO 4: CAPTURAS USO DE COPILOT - PROYECTO CROSSELING FRONT ... 161
ANEXO 5: CAPTURAS USO DE COPILOT - GESTOR CONSULTAS Y RECLAMOS 165
ANEXO 6: FORMATO DE PROMP PARA COPILOT 168

ÍNDICE DE TABLAS

TABLA 1 *COMPARACIÓN DE HERRAMIENTAS DE IA*....20
TABLA 2 *COMPARACIÓN DE MÉTODO DE DESARROLLO DE SOFTWARE*....24
TABLA 3 *VARIABLE INDEPENDIENTE: USO DE GITHUB COPILOT*....45
TABLA 4 *VARIABLE DEPENDIENTE: PRODUCTIVIDAD Y CALIDAD EN EL DESARROLLO DE SOFTWARE*....46
TABLA 5 *ANÁLISIS DE DATOS*....52
TABLA 6 *CRONOGRAMA DE ACTIVIDADES*....57
TABLA 7 *COSTOS MATERIALES*....58
TABLA 8 *COSTOS PERSONAL*....59
TABLA 9 *COSTOS INDIRECTOS*....59
TABLA 10 *COSTOS FIJOS*....60
TABLA 11 *COSTOS VARIABLES*....61
TABLA 12 *PRESUPUESTO TOTAL DE COSTOS*....61
TABLA 13 *ROLES Y DEFINICIONES*....63
TABLA14 *REQUERIMIENTOS FUNCIONALES PROYECTO 1*....64
TABLA15 *REQUERIMIENTOS FUNCIONALES PROYECTO 2*....65
TABLA16 *REQUERIMIENTOS FUNCIONALES PROYECTO 3*....65
TABLA17 *REQUERIMIENTOS NO FUNCIONALES*....66
TABLA 18 *PROYECTO 1: API CROSSELING FRONT*....70
TABLA 19 *PROYECTO 2: CROSSELING FRONT*....70
TABLA 20 *PROYECTO 3: GESTOR CONSULTAS Y RECLAMOS*....71
TABLA 21 *ESTIMACIÓN DE TAREAS*....72
TABLA 22 *RETROSPECTIVA DEL DESARROLLO CON COPILOT*....93
TABLA 23 *RETROSPECTIVA DEL DESARROLLO CON COPILOT*....94
TABLA 24 *RESULTADOS DEL PRE-TEST METODOLOGÍA TRADICIONAL PROYECTO 1*....99
TABLA 25 *RESULTADOS DEL PRE-TEST METODOLOGÍA TRADICIONAL PROYECTO 2*....102
TABLA 26 *RESULTADOS DEL PRE-TEST METODOLOGÍA TRADICIONAL PROYECTO 3*....106
TABLA 27 *RESULTADOS DEL POS-TEST PROYECTO 1*....110
TABLA 28 *DISTRIBUCIÓN DE FRECUENCIAS DE LA VARIABLE DEPENDIENTE EN EL POS-TEST PROYECTO 1.*....111
TABLA 29 *RESULTADOS DEL POS-TEST PROYECTO 2*....117
TABLA 30 *RESULTADOS DEL POS-TEST PROYECTO 2*....118
TABLA 31 *RESULTADOS DEL POS-TEST PROYECTO 2*....121
TABLA 32 *DISTRIBUCIÓN DE FRECUENCIAS DE LA VARIABLE DEPENDIENTE EN EL POS-TEST PROYECTO 3.*....122
TABLA 33 *ESTADÍSTICA DESCRIPTIVA MEJORA CON GITHUB COPILOT PROYECTO 1*....124
TABLA 34 *ESTADÍSTICA DESCRIPTIVA MEJORA CON GITHUB COPILOT PROYECTO 2*....125
TABLA 35 *ESTADÍSTICA DESCRIPTIVA MEJORA CON GITHUB COPILOT PROYECTO 3*....126
TABLA 36 *ESTADÍSTICA INFERENCIAL MEJORA CON GITHUB COPILOT PROYECTO 1*....131
TABLA 37 *ESTADÍSTICA INFERENCIAL MEJORA CON GITHUB COPILOT PROYECTO 2*....135
TABLA 38 *ESTADÍSTICA INFERENCIAL MEJORA CON GITHUB COPILOT PROYECTO 3*....139

ÍNDICE DE FIGURAS

FIGURA 1 ATRIBUTOS ISO/IEC 25010 32
FIGURA 2. MODELO DEL PROCESO DE NEGOCIO: API CROSSELING FRONT 67
FIGURA 3. MODELO DEL PROCESO DE NEGOCIO: CROSSELING FRONT 67
FIGURA 4. MODELO DEL PROCESO DE NEGOCIO: GESTOR CONSULTAS Y RECLAMOS 68
FIGURA 5. DIAGRAMA MODELO RELACIONAL: API CROSSELING FRONT 68
FIGURA 6. DIAGRAMA MODELO RELACIONAL: CROSSELING FRONT 69
FIGURA 7. DIAGRAMA MODELO RELACIONAL: GESTOR CONSULTAS Y RECLAMOS 69
FIGURA 8. CONTROLADOR DE LA API 74
FIGURA 9. IMPLEMENTACIÓN DEL SERVICIO DE LA API 75
FIGURA 10. PRUEBAS UNITARIAS API 75
FIGURA 11. EJECUCIÓN DE PRUEBAS UNITARIAS API 76
FIGURA 12. INTERFAZ DE PRUEBA UNITARIA API 76
FIGURA 13. CONTROLADOR DE GESTIÓN CROSSELING FRONT 79
FIGURA 14. CONTROLADOR DEL REPORTE CODEIGNITER 80
FIGURA 15. VISTA REPORTE CROSSELING FRONT 80
FIGURA 16. REPORTE DE CROSELLING FRONT 80
FIGURA 17. VISTA REPORTE CROSSELING FRONT 81
FIGURA 18. MODELO DE PARÁMETRO DE BD 83
FIGURA 19. MODELO DE REPORTES CROSSELING FRONT 84
FIGURA 20. REGISTRO DE USUARIOS CROSSELING FRONT 84
FIGURA 21. CONTROLADOR USUARIOS CESADOS 87
FIGURA 22. CONTROLADOR REPORTES GESTIÓN RECLAMOS 87
FIGURA 23. USUARIOS CESADOS GESTIÓN RECLAMOS 88
FIGURA 24 REGISTRO DE RECLAMOS 90
FIGURA 25. HISTÓRICO DE RECLAMOS 91
FIGURA 26. REPORTES DE RECLAMOS 91
FIGURA 27. PRUEBAS UNITARIA RECLAMOS 92
FIGURA 28. CRONOGRAMA JIRA PROYECTO 1 CON METODOLOGÍA TRADICIONAL 97
FIGURA 29. RESULTADO DE ANÁLISIS DE SONARQUBE PROYECTO 1 CON METODOLOGÍA TRADICIONAL 97
FIGURA 30. FORMULA GENERAL PARA CALCULAR LA CALIDAD DE CÓDIGO 98
FIGURA 31. RESULTADO DE BUGS DE SONARQUBE PROYECTO 1 CON METODOLOGÍA TRADICIONAL 99
FIGURA 32. RESULTADO DE PRUEBAS UNITARIAS DEL PROYECTO 1 CON METODOLOGÍA TRADICIONAL 99
FIGURA 33. CRONOGRAMA JIRA PROYECTO 2 CON METODOLOGÍA TRADICIONAL 100
FIGURA 34. RESULTADO DE ANÁLISIS DE SONARQUBE PROYECTO 2 CON METODOLOGÍA TRADICIONAL 101
FIGURA 35. RESULTADO DE BUGS DE SONARQUBE PROYECTO 2 CON METODOLOGÍA TRADICIONAL 102
FIGURA 36. RESULTADO DE PRUEBAS UNITARIAS DEL PROYECTO 2 CON METODOLOGÍA TRADICIONAL 102
FIGURA 37. CRONOGRAMA JIRA PROYECTO 3 CON METODOLOGÍA TRADICIONAL 103
FIGURA 38. RESULTADO DE ANÁLISIS DE SONARQUBE PROYECTO 3 CON METODOLOGÍA TRADICIONAL 104

FIGURA 39. RESULTA*DO DE BUGS DE SONARQUBE PROYECTO 3 CON METODOLOGÍA TRADICIONAL* 105
FIGURA 40. RESULTADO DE PRUEBAS UNITARIAS DEL PROYECTO 3 CON METODOLOGÍA TRADICIONAL 105
FIGURA 41. FÓRMULA PARA CALCULAR EL NIVEL DE EFECTIVIDAD DE PRUEBAS UNITARIAS 106
FIGURA 42. CRONOGRAMA JIRA PROYECTO 1 CON USO DE GITHUB COPILOT 107
FIGURA 43. CRONOGRAMA JIRA PROYECTO 1 CON USO DE GITHUB COPILOT 108
FIGURA 44. RESULTADO DE BUGS DE SONARQUBE PROYECTO 1 CON GITHUB COPILOT 109
FIGURA 45. RESULTADO DE PRUEBAS UNITARIAS DEL PROYECTO 1 CON GITHUB COPILOT 109
FIGURA 46. FÓRMULAS PARA LA OPERACIÓN DE LA VARIABLE DEPENDIENTE 111
FIGURA 47. GRÁFICO COMPARATIVO HORAS TOTALES.....113
FIGURA 48. GRÁFICO COMPARATIVO DE CÓDIGO LIBRE DE ERRORES 114
FIGURA 49. CRONOGRAMA JIRA PROYECTO 2 CON USO DE GITHUB COPILOT 115
FIGURA 50. SONARQUBE PROYECTO 2 CON USO DE GITHUB COPILOT.....115
FIGURA 51. RESULTADO DE BUGS DE SONARQUBE PROYECTO 2 CON GITHUB COPILOT 116
FIGURA 52. RESULTADO DE PRUEBAS UNITARIAS DEL PROYECTO 2 CON METODOLOGÍA TRADICIONAL 117
FIGURA 53. CRONOGRAMA JIRA PROYECTO 3 CON USO DE GITHUB COPILOT 119
FIGURA 54. SONARQUBE PROYECTO 3 CON GITHUB COPILOT.....120
FIGURA 55. RESULTADO DE BUGS DE SONARQUBE PROYECTO 3 CON GITHUB COPILOT 121
FIGURA 56. RESULTADO DE PRUEBAS UNITARIAS DEL PROYECTO 3 CON COPILOT 121

RESUMEN

La presente tesis aborda la optimización del desarrollo de software mediante la incorporación de GitHub Copilot, una herramienta basada en inteligencia artificial (IA) que ha emergido como una solución prometedora en la industria tecnológica. En un entorno competitivo, mejorar la eficiencia, reducir los tiempos de entrega y elevar la calidad del código son necesidades fundamentales. Los desafíos actuales incluyen retrasos prolongados en la finalización de proyectos, detección tardía de errores y la realización de tareas repetitivas, factores que afectan negativamente la productividad, la rentabilidad y la capacidad de las empresas para ofrecer productos de alta calidad. Esta investigación busca evaluar cómo la integración de GitHub Copilot en el ciclo de desarrollo puede abordar estos problemas, proponiendo mejoras tanto en los procesos como en los resultados. El estudio tiene como objetivo principal analizar el impacto de esta herramienta en el tiempo de desarrollo, la calidad del código y la eficiencia en la resolución de errores en una empresa con sede en Lima. La pregunta que guía esta investigación es: ¿Cómo influye GitHub Copilot en la optimización de los procesos de desarrollo de software en comparación con métodos tradicionales? Para responderla, se utilizará un enfoque cuantitativo que permitirá medir, mediante la recopilación y análisis de datos numéricos, el efecto de esta herramienta en la productividad y calidad del código. El propósito final es demostrar que GitHub Copilot puede generar mejoras significativas en los procesos de trabajo y en los resultados del desarrollo de software.

Palabras Claves: Inteligencia Artificial, GitHub Copilot, Desarrollo de Software.

ABSTRACT

This thesis focuses on the optimization of software development by incorporating GitHub Copilot, an artificial intelligence (AI) tool that has emerged as an effective solution in the industry. In a highly competitive development environment, enhancing efficiency, reducing delivery times, and improving code quality are critical needs. Current challenges, such as delays in project completion, late error detection, and repetitive tasks, negatively impact productivity, profitability, and the ability to deliver high-quality software. This research explores how integrating GitHub Copilot into the software development cycle can address these challenges, improving both process efficiency and software quality.

The primary objective is to analyze GitHub Copilot's impact on development time, code quality, and error identification and resolution efficiency within a Lima-based company. The guiding question is: What is the influence of GitHub Copilot on optimizing software development processes compared to traditional methods? Using a quantitative approach, the study will collect and analyze numerical data on productivity and code quality to evaluate this tool's effectiveness. The findings aim to demonstrate that GitHub Copilot significantly enhances workflows and outcomes in software development.

Keywords: Artificial Intelligence, GitHub Copilot, Software Development.

CAPÍTULO I: PROBLEMA DE INVESTIGACIÓN

1.1 Planteamiento y Formulación del Problema

1.1.1. Descripción de la Realidad Problemática

A nivel mundial, el uso de herramientas de inteligencia artificial (IA) en el desarrollo de software ha cobrado relevancia. Un estudio de Microsoft Research analiza cómo GitHub Copilot, una herramienta de asistencia en programación impulsada por IA ha mejorado significativamente la productividad de los desarrolladores. En un experimento controlado, se encontró que los desarrolladores que utilizaron GitHub Copilot completaron sus tareas un 55.8% más rápido que aquellos que no lo utilizaron. Este hallazgo sugiere que las herramientas de IA pueden ser particularmente beneficiosas, especialmente para aquellos que están comenzando en el desarrollo de software. Sin embargo, el desarrollo tradicional enfrenta problemas como largos tiempos de entrega y alta tasa de errores, lo que afecta la competitividad y calidad del software producido. Además, es importante considerar los riesgos asociados con el uso de IA, como la generación de código inseguro, que podrían impactar la calidad y la seguridad del software. Esto subraya la necesidad de un uso cauteloso y consciente de estas herramientas en entornos de desarrollo [6].

A nivel internacional, la aplicación de técnicas de inteligencia artificial en la ingeniería de software ha mostrado un creciente interés y desarrollo. Cruz Rubio, Blanco Puertas y Contreras Cortes examinan las estrategias para la implementación de IA en el equipo de desarrollo de SYSPOTEC SAS, revelando que la integración de estas tecnologías mejora significativamente la calidad del software y optimiza procesos como la detección de errores, la predicción de fallos y la automatización de tareas de programación. Sin embargo, la adopción generalizada enfrenta desafíos, como la necesidad de contar con datos de alta calidad y la capacitación del personal técnico. Aunque estas herramientas han demostrado ser efectivas para aumentar la productividad,

todavía existe una dependencia del juicio humano, especialmente en situaciones que requieren creatividad y un diseño algorítmico más sofisticado [5].

En el contexto de Perú, el desarrollo de software también se ha beneficiado de la incorporación de herramientas de IA, especialmente en el campo de aplicaciones móviles. Un estudio de Solís Villanueva en la Universidad Nacional Federico Villarreal, sobre la aplicación de redes neuronales en el desarrollo de aplicaciones móviles, muestra que el uso de modelos basados en IA tiene un impacto positivo en la eficiencia y calidad del desarrollo, permitiendo una integración de datos más ágil y precisa [4].

Este contexto pone de manifiesto la importancia de estudiar cómo la inteligencia artificial puede mejorar el desarrollo de software en comparación con los enfoques tradicionales. Este análisis es esencial para el progreso tecnológico y la competitividad en el sector del software del país, dado que el desarrollo tradicional, al no contar con estas herramientas, a menudo enfrenta problemas como largos tiempos de entrega, baja calidad del código y dificultades para detectar y solucionar errores. Esto puede resultar en un software menos competitivo y más propenso a fallos, lo que subraya la necesidad de una investigación más exhaustiva sobre las ventajas, limitaciones y recomendaciones para un uso ético y eficaz de herramientas como GitHub Copilot. Por lo tanto, surge la siguiente pregunta de investigación: ¿De qué manera impacta el uso de GitHub Copilot en la optimización de los procesos de desarrollo de software en comparación con el desarrollo tradicional, en aspectos como el tiempo de desarrollo, la calidad del código y la eficacia en la detección y resolución de errores en una empresa de Lima?

1.1.2. Problema General

¿Cómo influye el uso de GitHub Copilot en la optimización de los procesos de desarrollo de software en comparación con el desarrollo tradicional, en términos de tiempo de desarrollo, calidad del código y eficiencia en la detección y resolución de errores en una empresa de Lima?

1.1.3 Problemas Específicos

P.E.1: ¿Cómo afecta GitHub Copilot al tiempo de desarrollo de software en comparación con el método tradicional?

P.E.2: ¿Qué impacto tiene GitHub Copilot en la calidad del código generado, en comparación con el código producido mediante el método tradicional?

P.E.3: ¿Cómo influye GitHub Copilot en la eficiencia de la detección y resolución de errores en el código en comparación con métodos tradicionales?

P.E.4: ¿Cómo influye GitHub Copilot en la eficiencia de la detección y resolución de errores en el código en comparación con métodos tradicionales?

1.2 Objetivos

1.2.1 Objetivo General:

Determinar el impacto de GitHub Copilot en la optimización de los procesos de desarrollo de software en comparación con el desarrollo tradicional, evaluando métricas de tiempo, calidad y eficiencia en la detección y resolución de errores en una empresa de lima.

1.2.2 Objetivos Específicos:

O.E.1: Comparar el tiempo de desarrollo de software utilizando GitHub Copilot con el tiempo de desarrollo en proyectos realizados con métodos tradicionales

O.E.2: Evaluar la calidad del código generado con GitHub Copilot y compararla con la calidad del código desarrollado mediante métodos tradicionales.

O.E.3: Analizar la eficiencia en la detección y resolución de errores al usar GitHub Copilot en comparación con métodos tradicionales.

O.E.4: Comparar la efectividad de las pruebas unitarias realizadas con github copilot y las realizadas con métodos tradicionales midiendo la taza de resolución de errores.

1.3 Justificación

1.3.1 Teórica

Teóricamente, esta investigación amplía el conocimiento existente sobre la aplicación de herramientas de inteligencia artificial en el desarrollo de software, enfocándose en GitHub Copilot. Diversos estudios han mostrado que la IA tiene la capacidad de mejorar los procesos de desarrollo, acortando los tiempos de producción y elevando la calidad del código. Al realizar una comparación entre GitHub Copilot y los métodos tradicionales, este estudio proporcionará datos empíricos que servirán como base para futuras investigaciones y contribuirán a la formulación de nuevas teorías sobre la adopción de la IA en el desarrollo de software.

1.3.2 Práctica

Desde un enfoque práctico, la investigación ofrece una oportunidad para que las empresas de desarrollo de software en Lima evalúen el impacto real de GitHub Copilot en sus procesos. Al centrarse en métricas clave como la reducción del tiempo de desarrollo, la mejora de la calidad del código y la eficiencia en la resolución de errores, este estudio proporcionará datos concretos que podrán ser utilizados para optimizar la productividad y competitividad de las empresas tecnológicas. Los resultados permitirán a los desarrolladores y a los responsables de tecnología

tomar decisiones informadas sobre la adopción de herramientas de IA en sus proyectos, contribuyendo a mejorar los estándares de calidad y eficiencia en el sector.

1.3.3 Factibilidad

Este estudio es viable porque GitHub Copilot está fácilmente disponible y permite recopilar métricas clave en el desarrollo de software, como tiempo de programación, calidad del código y eficiencia en la resolución de errores. Al implementar y observar el uso de GitHub Copilot en un entorno controlado, es posible obtener datos precisos sin incurrir en altos costos. Además, los estudios previos sobre inteligencia artificial en el desarrollo de software proporcionan una base teórica sólida, lo que facilita el diseño de la investigación y asegura una comparación estructurada y significativa entre métodos tradicionales y el uso de IA en el proceso de desarrollo.

1.3.4 Relevancia

Esta investigación es relevante para el sector tecnológico, ya que evalúa el impacto de la IA en el desarrollo de software. Analizar GitHub Copilot ayudará a comprender su potencial para mejorar la competitividad, reducir tiempos y elevar la calidad del software. Los resultados servirán como guía para una adopción responsable y efectiva de IA en procesos de desarrollo.

1.4 Viabilidad de la Investigación

La viabilidad de esta investigación se sustenta en varios factores clave que garantizan su realización exitosa. En primer lugar, la disponibilidad de GitHub Copilot, una herramienta accesible para nosotros los desarrolladores mediante suscripción, esto asegura que podamos usarla sin inconvenientes técnicos.

Además, la investigación se llevará a cabo en una empresa que desarrolla software ubicada en Lima, las cual se ha mostrado dispuesta a colaborar en el análisis de productividad y calidad del

desarrollo de software. Se contempla un periodo de dos meses para implementar GitHub Copilot y evaluar los resultados, asegurando que se dispone de tiempo suficiente para obtener datos representativos.

La recopilación de métricas de productividad y calidad se podrá realizar mediante herramientas de gestión de proyectos y análisis de código en este caso se utilizará Jira como un gestor de tiempos para el desarrollo y SonarQube para medir calidad del código, lo que permitirá contar con datos fiables para la comparación entre los desarrolladores que utilicen GitHub Copilot y aquellos que no lo hagan.

1.5 Alcance y Limitaciones

1.5.1 Alcances

Esta investigación busca analizar el impacto de la inteligencia artificial, específicamente GitHub Copilot, en la productividad y calidad del desarrollo de software en una empresa de Lima. El estudio se centrará en la comparación de tres proyectos de software preexistentes, que serán replicados utilizando GitHub Copilot para evaluar su influencia en el tiempo de desarrollo, la calidad del código, la eficiencia en la detección de errores y la efectividad de las pruebas unitarias. Se utilizarán herramientas como Jira para rastrear el tiempo y SonarQube para medir la calidad del código, proporcionando un análisis cuantitativo riguroso de los resultados. La investigación se limitará al uso de GitHub Copilot como herramienta de IA, sin abordar otras plataformas o tecnologías de asistencia al desarrollo.

El alcance temporal cubre el ciclo completo de desarrollo de los tres proyectos replicados, desde su planificación hasta la evaluación final. Los resultados se analizarán empleando pruebas estadísticas y se visualizarán mediante gráficos para identificar diferencias significativas en la productividad y calidad entre los métodos tradicionales y aquellos que emplean IA.

1.5.2 Limitaciones

Una de las principales limitaciones del estudio es que se restringe a proyectos de software desarrollados en una única empresa de Lima, lo cual podría no representar el comportamiento de GitHub Copilot en diferentes contextos empresariales o industrias. Además, el tamaño de la muestra que son 3 proyectos podría limitar la generalización de los resultados a escenarios de desarrollo de software más amplios.

Otra limitación es la dependencia de la configuración y características específicas de GitHub Copilot en el momento de la investigación, lo cual podría variar si la herramienta evoluciona o recibe actualizaciones. Para mitigar estas limitaciones, el estudio adopta un enfoque cuasiexperimental controlado que permitirá una comparación precisa entre las versiones desarrolladas con y sin Copilot, asegurando que los resultados reflejen de forma consistente la influencia de esta herramienta de IA en el proceso de desarrollo de software.

1.6 Hipótesis y Variables

1.6.1 Hipótesis General:

El uso de GitHub Copilot optimiza significativamente los procesos de desarrollo de software en comparación con el desarrollo tradicional, mejorando el tiempo de desarrollo, la calidad del código y la eficiencia en la detección y resolución de errores en una empresa de Lima.

1.6.2 Hipótesis Especifica:

H.E. 01: GitHub Copilot reduce el tiempo de desarrollo de software en comparación con los métodos tradicionales.

H.E. 02: El código generado con GitHub Copilot presenta mayor calidad, en comparación con el código desarrollado por métodos tradicionales.

H.E. 03: GitHub Copilot mejora la eficiencia en la detecciones y resolución de errores, al reducir el número de bugs no detectados con el método tradicional.

H.E. 04: Las pruebas unitarias realizadas con GitHub Copilot tienen mayor efectividad en la detección y resolución de errores superando las pruebas realizadas con métodos tradicionales.

1.6.3 Variable dependiente

Productividad y calidad en el desarrollo de software.

1.6.4 Variable Independiente

GitHub Copilot.

CAPÍTULO II. MARCO TEÓRICO

2.1 Estado del Arte

2.1.1 Introducción al Estado del Arte

En los últimos años, la industria del software ha experimentado una transformación significativa con la integración de herramientas impulsadas por inteligencia artificial (IA) en el proceso de desarrollo. Estas herramientas buscan mejorar la eficiencia, productividad y calidad del código generado por los desarrolladores. Una de las innovaciones más destacadas en este ámbito es GitHub Copilot, un asistente de codificación basado en IA desarrollado por GitHub en colaboración con OpenAI. Desde su lanzamiento en 2021, GitHub Copilot ha sido adoptado por más de un millón de desarrolladores y más de 20,000 organizaciones, generando más de tres mil millones de líneas de código aceptadas [65]. Este crecimiento refleja la creciente confianza de la comunidad de desarrolladores en las capacidades de las herramientas de IA para asistir en la programación.

2.1.2. Desarrollo de Software y Herramientas de Asistencia

El desarrollo de software ha evolucionado significativamente, desde metodologías tradicionales como el modelo en cascada y RUP, hacia enfoques más ágiles como Scrum y DevOps, que permiten una mayor flexibilidad y respuesta rápida a los cambios [68]. Estos enfoques modernos optimizan el proceso de desarrollo al facilitar la colaboración y la entrega continua de software.

En este contexto, las herramientas de asistencia, especialmente aquellas basadas en inteligencia artificial (IA), han ganado protagonismo. GitHub Copilot, por ejemplo, utiliza IA para generar sugerencias de código basadas en el contexto, reduciendo el tiempo que los desarrolladores dedican a tareas repetitivas. Se ha demostrado que los desarrolladores que utilizan Copilot pueden completar tareas hasta un 55% más rápido que aquellos que no lo utilizan [69]. Además de GitHub

Copilot, existen otras herramientas como IntelliSense, TabNine y Amazon CodeWhisperer, que ofrecen autocompletado inteligente, mejorando la eficiencia en el proceso de codificación [70]. No obstante, a pesar de las ventajas en términos de productividad, existen preocupaciones sobre la dependencia excesiva de estas herramientas. Se han identificado riesgos como la generación de código subóptimo o inseguro, lo que resalta la importancia de que los desarrolladores mantengan una supervisión adecuada [71].

2.1.3. GitHub Copilot: Funcionamiento y Aplicaciones

GitHub Copilot es una herramienta de autocompletado de código impulsada por IA que proporciona sugerencias en tiempo real a los desarrolladores mientras escriben código. Utiliza el modelo de lenguaje GPT-3 de OpenAI, entrenado en miles de millones de líneas de código fuente de repositorios públicos, para predecir y generar fragmentos de código basados en el contexto proporcionado por el programador [66]. Esta capacidad permite a Copilot ofrecer desde líneas individuales hasta funciones completas, facilitando la escritura de código y reduciendo el tiempo dedicado a tareas repetitivas.

Las aplicaciones de GitHub Copilot son diversas. Además de asistir en la generación de código, se ha utilizado para traducir código entre diferentes lenguajes de programación, generar documentación y sugerir pruebas unitarias. Un estudio realizado por Zhang et al. analizó las prácticas y desafíos asociados con el uso de Copilot, identificando beneficios como la generación de código útil y limitaciones como dificultades en la integración con ciertos entornos de desarrollo [67]. Estos hallazgos resaltan tanto el potencial como las áreas de mejora para la adopción de Copilot en diversos contextos de desarrollo.

2.1.4. Productividad y Calidad en el Desarrollo de Software

La productividad en el desarrollo de software se refiere a la eficiencia con la que se completan tareas en relación con los recursos empleados, como el tiempo y los costos. GitHub Copilot y herramientas similares han mostrado un impacto positivo en la productividad al reducir el tiempo dedicado a tareas repetitivas y permitir que los desarrolladores se concentren en resolver problemas más complejos [69]. Además, la reducción de errores mediante la generación automatizada de pruebas unitarias mejora el flujo de trabajo, incrementando la productividad de los equipos de desarrollo [72].

En cuanto a la calidad, esta se mide utilizando modelos como ISO/IEC 25010, que evalúa aspectos como la funcionalidad, la mantenibilidad, el rendimiento y la seguridad del código [73]. Herramientas como SonarQube permiten realizar análisis automáticos del código, detectando vulnerabilidades y garantizando que el software cumpla con altos estándares de calidad [74].

El uso de IA en el desarrollo de software ha mostrado mejoras significativas en la calidad del código. Un estudio reciente mostró que el uso de herramientas como GitHub Copilot no solo aumenta la productividad, sino que también ayuda a reducir la cantidad de errores y mejora la mantenibilidad del código [72]. Sin embargo, es esencial que estas herramientas se utilicen como apoyo y no como reemplazo del juicio humano, para evitar la generación de código inseguro o ineficiente.

2.1.5. Estudios Previos y Trabajos Relacionados

El uso de herramientas de inteligencia artificial, como GitHub Copilot, ha generado un creciente interés en la comunidad académica y en la industria, debido a su impacto potencial en la productividad y calidad del desarrollo de software. Diversos estudios han investigado la efectividad de Copilot en la generación de código y su influencia en los tiempos de desarrollo. Un

estudio comparativo realizado por Yetiştiren et al. evaluó la calidad del código generado por GitHub Copilot en comparación con otras herramientas asistidas por IA, como Amazon CodeWhisperer y ChatGPT. Los resultados indicaron que Copilot tiene una tasa de éxito del 91.5% en la generación de código válido, aunque también mostró limitaciones, como la generación de código incorrecto en ciertos casos, lo que subraya la necesidad de una supervisión humana adecuada [72].

En estudios donde se comparan las herramientas de asistencia basadas en IA con los métodos tradicionales de desarrollo, se ha encontrado que GitHub Copilot puede acelerar significativamente el proceso de programación. Por ejemplo, Smit et al. analizaron cómo Copilot afecta la productividad en entornos ágiles y encontraron que reduce el tiempo de codificación en un 55%, lo que representa una mejora considerable en entornos de desarrollo continuo. Sin embargo, se concluyó que, si bien es eficiente para tareas repetitivas o estándar, el código generado puede no ser óptimo en comparación con el desarrollo manual cuando se requiere un diseño o solución más compleja [69]. Ferdiana también destaca que los desarrolladores que utilizan Copilot pueden completar tareas hasta un 30% más rápido que aquellos que no lo hacen, lo que refleja un aumento directo en la productividad y una reducción de errores [71].

El impacto de GitHub Copilot en entornos empresariales ha sido objeto de varios estudios de casos. Smit et al. documentaron la implementación de Copilot en una empresa automotriz, donde los desarrolladores que lo utilizaron lograron una reducción promedio de 30 minutos por día en tiempo de desarrollo, junto con mejoras en la calidad del código y en la satisfacción de los empleados [69].

2.1.6. Gap del Conocimiento

A pesar de los avances y la adopción creciente de herramientas como GitHub Copilot, existen áreas que requieren una investigación más profunda. La mayoría de los estudios se han centrado en evaluar la eficiencia y productividad desde una perspectiva técnica, pero hay una falta de investigaciones que analicen el impacto de estas herramientas en entornos específicos, como empresas de desarrollo de software en regiones particulares. Por ejemplo, no se han encontrado estudios que aborden el uso de Copilot en empresas de Lima, Perú, durante el año 2024. Además, aunque se han identificado desafíos en la integración de Copilot con ciertos entornos de desarrollo, se requiere una comprensión más detallada de cómo estas herramientas afectan la dinámica de equipo, la calidad del código y las prácticas de desarrollo en diferentes contextos organizacionales. Este vacío en la literatura ofrece una oportunidad para explorar y comprender mejor el impacto de GitHub Copilot en la productividad y calidad del desarrollo de software en empresas específicas, proporcionando información valiosa para su implementación efectiva.

Conclusión del Estado del Arte

El análisis del estado del arte sobre GitHub Copilot y su impacto en el desarrollo de software revela una evolución significativa en el uso de herramientas de inteligencia artificial para la asistencia en la programación. A lo largo de la revisión, se han identificado múltiples estudios que destacan los beneficios de Copilot en términos de productividad y calidad del código, pero también se han señalado desafíos relacionados con la integración en flujos de trabajo, dependencia de la herramienta y limitaciones en la generación de código óptimo.

Dado este panorama, la presente investigación busca contribuir al conocimiento al realizar un análisis comparativo de la productividad y calidad del código generado con GitHub Copilot en una empresa de desarrollo de software en Lima. Esto permitirá no solo validar los hallazgos previos

en un contexto particular, sino también generar nuevas perspectivas sobre el impacto real de esta herramienta en el desarrollo profesional de software.

2.2 Antecedentes Internacionales y Nacionales

Antecedentes Internacionales

Cruz Rubio, Blanco Puertas y Contreras Cortes llevaron a cabo un estudio sobre las estrategias para implementar inteligencia artificial en el equipo de desarrollo de SYSPOTEC SAS. Su investigación examina diversas aplicaciones de la IA en el entorno empresarial, subrayando su capacidad para optimizar procesos, mejorar la toma de decisiones y aumentar la eficiencia operativa. A través de una revisión sistemática de la literatura, identificaron herramientas de IA clave y evaluaron su impacto en el desarrollo de software. Además, el estudio destaca que la integración de la IA no solo mejora la calidad del producto, sino que también transforma la cultura organizacional, promoviendo una mentalidad innovadora entre los equipos. Un ejemplo significativo es la aplicación de DeepMind, que logró una mejora del 70% en la velocidad de una función de ordenamiento de listas en C++ [5].

Asimismo, un análisis reciente llevado a cabo por Milton Rafael Coral Navarro y Edison Alexander Ñacato Ganchala investiga cómo la Inteligencia Artificial (IA) influye en la automatización de tecnologías de software. Se entiende que la IA es un área de la informática dedicada a crear sistemas que pueden realizar tareas típicamente asociadas con la inteligencia humana, tales como aprender, razonar y tomar decisiones. Este progreso amplía la aplicación de herramientas y datos en múltiples sectores, incrementando la eficiencia operativa. En 2021, Colombia se situó en el puesto 45 de 160 países en el Índice de Preparación del Gobierno para la IA, lo que subraya la creciente integración de esta tecnología en el ámbito público y su relevancia en el sector tecnológico [7].

Finalmente, Aguilar Sanhueza realizó un estudio sobre el desarrollo de una interfaz gráfica para DashAI, una plataforma de inteligencia artificial diseñada para usuarios sin conocimientos en programación. En su evaluación, se utilizó una muestra de 5 usuarios para realizar una prueba de usabilidad, la cual se midió con indicadores de eficacia y eficiencia, alcanzando un promedio del 78% en tareas completadas y un tiempo medio de 45 segundos por tarea, cumpliendo el 70% del tiempo límite asignado. Además, el cuestionario NASA-TLX fue aplicado para evaluar la carga de trabajo percibida, donde los usuarios reportaron una satisfacción promedio de 4.2 en usabilidad y 4.5 en satisfacción visual [3]. Este análisis resalta cómo el diseño de interfaces intuitivas puede mejorar la experiencia en el desarrollo de software asistido por inteligencia artificial.

Antecedentes Nacionales

Robles L. [1] llevó a cabo un estudio sobre el impacto del uso de ChatGPT en la calidad del desarrollo web en una empresa privada de Lima. El enfoque de su investigación fue cuasiexperimental, y utilizó herramientas como SonarQube para medir la calidad del código, comparando dos grupos de desarrollo: uno que usó ChatGPT y otro que no lo hizo. Los resultados mostraron un incremento significativo en la calidad del código, con un aumento del 99.12% en la calidad general y mejoras del 80% en seguridad y 1.37% en rendimiento [1]. Este estudio destaca la capacidad de las herramientas de inteligencia artificial para mejorar el proceso de desarrollo de software, proporcionando un marco de referencia para el uso de GitHub Copilot en la optimización de la calidad del código.

Solís V. [4] examina el efecto de un modelo inteligente basado en técnicas de inteligencia artificial en el desarrollo de software para aplicaciones móviles. Mediante un enfoque cuantitativo, se establece que el 57.3% de los participantes opina que la implementación de

modelos de IA, en particular las redes neuronales, mejora tanto la eficiencia en el procesamiento de datos como la calidad del software. Esta investigación aporta una comprensión valiosa sobre la integración efectiva de estas tecnologías en el ámbito del desarrollo de software [4].

Por otro lado, Paz Díaz [2] investigó el uso de la automatización de pruebas con ChatGPT para mejorar la calidad del software en empresas del sector financiero en Lima. Su estudio demostró que la implementación de ChatGPT en las pruebas de software resultó en una mejora del 20% en la creación de casos de prueba y una optimización significativa en la detección de errores. Este trabajo subraya la relevancia de integrar herramientas de inteligencia artificial, no solo en el desarrollo de software, sino también en los procesos de calidad y testing [2]. Al comparar estas tecnologías con GitHub Copilot, se pueden extraer importantes lecciones sobre la eficiencia y la reducción de errores durante el desarrollo.

2.3 Bases Teóricas

2.3.1 Introducción al Desarrollo de Software

La creación de software es un campo que involucra las diferentes fases necesarias para desarrollar programas y aplicaciones informáticas. Su propósito es diseñar, implementar y mantener soluciones que cumplan con las expectativas de usuarios y empresas, mejorando la eficiencia y funcionalidad en el uso de la tecnología [8].

2.3.1.1 Definición del Desarrollo de Software

Una publicación de IBM menciona que el desarrollo de software es el proceso integral de creación de aplicaciones y sistemas, que abarca desde el análisis de los requisitos hasta el despliegue y mantenimiento continuo. Este proceso incluye etapas clave como el diseño arquitectónico, la programación, las pruebas y la gestión de versiones. A través de metodologías como ágil o DevOps, los equipos de desarrollo colaboran para producir

software de alta calidad de manera eficiente, adaptándose a cambios y mejorando continuamente. Estas metodologías también permiten ciclos de producción rápidos, lo cual ayuda a mantener el software alineado con las necesidades del usuario y del negocio [8].

2.3.1.2 Proceso de Desarrollo de Software

Según una publicación de [9], las fases del proceso de desarrollo de software son las siguientes:

- Planificación: En esta fase, se identifican los requisitos y se elabora un plan detallado que incluye estimaciones de costos y cronograma, asignando los recursos necesarios.
- Diseño: El equipo crea la arquitectura técnica del sistema y selecciona las herramientas y tecnologías adecuadas para el proyecto.
- Codificación: El software se construye a partir de los requisitos desglosados en tareas pequeñas y manejables.
- Pruebas: Se ejecutan pruebas manuales y automáticas para asegurar que el software funciona correctamente y cumple con los requisitos del cliente.
- Implementación o despliegue: El software se implementa en el entorno de producción tras las pruebas, asegurando su disponibilidad para los usuarios finales.
- Mantenimiento: Después del lanzamiento, se realiza un seguimiento para corregir errores, mejorar el rendimiento y adaptar el software a nuevos requisitos.

2.3.1.3 Modelo de Desarrollo de Software

- Métodos tradicionales

Un estudio realizado por [10] menciona que la metodología tradicional de desarrollo de software se basa en una planificación detallada y documentación exhaustiva desde el inicio del proyecto. Establece roles, actividades y herramientas de manera estricta, buscando un proceso controlado. Sin embargo, es menos adaptable a cambios, lo que la hace poco adecuada para proyectos con requisitos inciertos o variables, resultando en altos costos y baja flexibilidad, especialmente en entornos cambiantes [10]. Teniendo como referencia esta metodología, tenemos la modelo cascada que es un enfoque tradicional que sigue una secuencia lineal de etapas en un proyecto, como análisis, diseño, desarrollo, pruebas y entrega. En este modelo, cada fase debe completarse antes de pasar a la siguiente, lo que proporciona claridad y control durante todo el proceso. Sin embargo, su rigidez puede convertirse en un inconveniente cuando surgen cambios en los requisitos, lo que lo hace más adecuado para proyectos con objetivos fijos y bien definidos desde el principio.

- Métodos agiles

Asimismo [10] menciona que la metodología ágil se enfoca en la flexibilidad y adaptabilidad, con un proceso incremental que facilita entregas rápidas y la colaboración constante entre clientes y desarrolladores. Prioriza la capacidad de respuesta a cambios inesperados, minimizando la documentación formal y evitando los procesos burocráticos, lo que hace el desarrollo más eficiente y satisfactorio [10]. Podemos tener de referencia la herramienta que se utiliza para esta metodología es Jira que muy popular en proyectos ágiles, como Scrum o Kanban, que ayuda a gestionar las tareas de manera efectiva. Permite crear tableros visuales para organizar y priorizar actividades, planificar ciclos de trabajo llamados sprints y asignar responsabilidades. Además, facilita la colaboración continua y el seguimiento del progreso, brindando flexibilidad para ajustar los requisitos a medida que el proyecto avanza.

Con sus métricas y reportes, Jira es especialmente útil en proyectos dinámicos que requieren adaptabilidad.

2.3.2 Herramientas de Asistencia en el Desarrollo de Software

2.3.2.1 Automatización y Herramientas de Codificación Asistida

La automatización en el desarrollo de software busca optimizar tareas repetitivas, permitiendo a los desarrolladores centrarse en actividades de mayor complejidad y creatividad. Herramientas como GitHub Copilot, impulsadas por inteligencia artificial (IA), han transformado este proceso al generar sugerencias de código en tiempo real basándose en el contexto del programa en desarrollo. Estas herramientas analizan el entorno del código escrito y proponen fragmentos funcionales que cumplen con estándares de programación, lo que facilita el trabajo de los desarrolladores.

El impacto de estas tecnologías es significativo. Según un estudio experimental, GitHub Copilot permite completar tareas específicas un 55.8 % más rápido en comparación con métodos tradicionales, como el uso exclusivo de buscadores o foros técnicos. Este resultado evidencia cómo estas herramientas pueden incrementar la productividad en entornos de programación profesional [28]. Además, su utilidad se extiende a desarrolladores con menor experiencia, quienes obtienen beneficios adicionales al superar tareas básicas más rápidamente y al acceder a soluciones preconfiguradas [28].

Entre las funcionalidades destacadas de herramientas como GitHub Copilot se encuentran:

- Sugerencias contextuales automáticas: Basadas en comentarios o líneas de código existentes, permiten completar funciones comunes con solo unas pocas instrucciones.

- Generación de pruebas unitarias: Facilita la creación automatizada de pruebas básicas para verificar el correcto funcionamiento del software.
- Estandarización del código: Asegura que las soluciones propuestas sigan patrones establecidos, mejorando la mantenibilidad y calidad general del proyecto

Tabla 1

Comparación de herramientas de IA

Herramienta	Características Principales	Ventajas	Limitaciones
GitHub Copilot	Generación de código y pruebas unitarias, soporte multilenguaje.	Altamente contextual y aprendizaje continuo.	Dependencia del contexto proporcionado por el usuario.
Tabnine	Autocompletado basado en IA, soporte offline para empresas.	Compatible con múltiples IDEs, personalizable.	No genera código complejo como funciones completas.
Kite	Autocompletado avanzado, sugerencias en Python.	Muy rápido y ligero.	Limitado en soporte para lenguajes no populares.
Amazon CodeWhisperer	Integración con servicios en la nube, generación de código para AWS.	Ideal para proyectos basados en AWS.	Dependencia de ecosistema Amazon.

Fuente. Elaboración Propia

2.3.2.2 Historia y Evolución de las Herramientas de Asistencia de Código

Los primeros avances se dieron con la introducción de los editores de texto enriquecidos y los entornos de desarrollo integrados (IDEs) en los años 80 y 90. Herramientas como Eclipse y Visual Studio ofrecían características como autocompletado de código, resaltado de sintaxis y depuración en tiempo real, lo que permitió a los programadores trabajar de manera más eficiente

y estructurada. Estas herramientas revolucionaron el desarrollo al reducir los errores y optimizar las tareas repetitivas [29], [30].

En los últimos años, el uso de IA ha transformado los IDEs en entornos más inteligentes. GitHub Copilot, lanzado en 2021, utiliza modelos de aprendizaje profundo para comprender el contexto del código y generar automáticamente soluciones funcionales. Estas herramientas permiten a los desarrolladores completar tareas complejas de manera más rápida, con estudios que demuestran que hasta un 30.5 % del código en proyectos asistidos puede ser generado automáticamente, mejorando la productividad y la calidad del software [30], [31].

Sin embargo, los desarrolladores enfrentan desafíos como la necesidad de validar las sugerencias de la IA y asegurarse de que cumplan con estándares de calidad, seguridad y rendimiento. Además, se requiere un control adecuado para que las herramientas generen resultados específicos según las necesidades del proyecto [31].

El futuro de los entornos de desarrollo asistidos por IA promete avances como la integración de herramientas que validen automáticamente el código, predigan errores y optimicen arquitecturas de software antes de la implementación. Se prevé que los desarrolladores pasen de ser programadores activos a gestores de herramientas de IA, supervisando los resultados generados y ajustándolos a las necesidades específicas de los proyectos [29].

2.3.3 GitHub Copilot

2.3.3.1 Descripción

Una publicación de [17] menciona que, GitHub Copilot es una herramienta de asistencia para la programación que utiliza inteligencia artificial. Diseñada para actuar como un compañero en el proceso de codificación, su objetivo principal es ofrecer sugerencias automáticas mientras se escribe código. Estas sugerencias pueden basarse tanto en la

escritura directa dentro del editor como en descripciones expresadas en lenguaje natural. Copilot opera a través de OpenAI Codex, un avanzado sistema de IA desarrollado por OpenAI, que analiza el contexto del código y los comentarios para generar desde líneas individuales hasta funciones completas. Además, evalúa tanto el archivo en el que se está trabajando como los relacionados, proporcionando recomendaciones relevantes y útiles para el programador.

2.3.3.2 Funcionamiento

GitHub Copilot funciona analizando el código que el desarrollador escribe en un entorno compatible, entendiendo el contexto del proyecto y los objetivos que se buscan. Basándose en el texto público alojado en GitHub, la herramienta utiliza este conocimiento para generar sugerencias de código personalizadas, adaptándose al progreso del usuario. Estas propuestas pueden ser aceptadas para completar automáticamente el código o rechazadas para recibir nuevas opciones, lo que facilita la automatización de tareas repetitivas y ahorra tiempo en el desarrollo [11].

Otra publicación realizada por Microsoft menciona que el funcionamiento de GitHub Copilot se basa en modelos avanzados de aprendizaje automático, desarrollados a partir de un extenso conjunto de datos de código abierto extraído de los repositorios públicos de GitHub. Mientras el usuario escribe, la herramienta analiza el contexto de manera dinámica para ofrecer sugerencias relacionadas. Además, admite la generación de propuestas utilizando descripciones en lenguaje natural que especifiquen las acciones o el propósito del código [18].

2.3.3.3 Ventajas y Limitaciones

Según [19], GitHub Copilot presenta diversas ventajas y limitaciones que afectan tanto a su eficiencia como a su aplicabilidad en el desarrollo de software.

Ventajas:

- Reducción de tiempo: Copilot ayuda a los desarrolladores proporcionando sugerencias de autocompletado y funciones completas, lo que acelera la escritura de código y la depuración, permitiendo optimizar el tiempo de trabajo.
- Facilidad de aprendizaje: Es particularmente útil para los programadores novatos, ya que ofrece recomendaciones basadas en buenas prácticas y ejemplos comunes, facilitando la comprensión de estructuras y patrones en la programación.
- Adaptabilidad: GitHub Copilot es compatible con una amplia gama de lenguajes de programación y marcos de trabajo, lo que lo convierte en una herramienta flexible para trabajar en distintos entornos de desarrollo.
- Interacción intuitiva: Permite al usuario generar código describiendo tareas en lenguaje natural, lo que hace que la experiencia de desarrollo sea más accesible y sencilla para quienes no están familiarizados con sintaxis complejas.

Limitaciones:

- Inconsistencia en la fiabilidad: Aunque Copilot es eficaz en muchos casos, no siempre genera soluciones correctas. En ocasiones, puede sugerir código que no cumple con los requisitos esperados o incluso que no se compila, lo que obliga al desarrollador a revisar y corregir las sugerencias.

- Posibilidad de sesgos: Al estar entrenado con un gran volumen de código disponible públicamente, puede replicar errores o enfoques no óptimos presentes en los datos de entrenamiento, lo que podría afectar la calidad del código generado.
- Riesgo de dependencia: Si se utiliza con frecuencia, puede fomentar una dependencia excesiva de la herramienta, reduciendo la capacidad del desarrollador para escribir código de manera independiente y sin asistencia.
- Modelo de suscripción: Aunque GitHub Copilot ofrece opciones gratuitas, para acceder a todas sus funcionalidades avanzadas es necesario contar con una suscripción de pago, lo que puede resultar costoso para algunos desarrolladores.

2.3.4 Comparación de Método de Desarrollo de Software

Tabla 2

Comparación de Método de Desarrollo de Software

Aspecto	Método Tradicional	Desarrollo Asistido por IA
Tiempo de desarrollo	Lento, ya que depende exclusivamente de la experiencia del desarrollador y la codificación manual.	Más rápido gracias a la automatización de tareas repetitivas y sugerencias de código en tiempo real.
Calidad del código	Variable, dependiendo de revisiones manuales y buenas prácticas del equipo de desarrollo.	Mejora notable al utilizar métricas automatizadas y sugerencias basadas en mejores prácticas.
Detección de errores	Basada en revisiones manuales y pruebas exhaustivas, lo que puede ser propenso a omitir errores menores.	Más eficiente con análisis automatizados y recomendaciones para correcciones en tiempo real.
Pruebas unitarias	Requiere tiempo para escribir manualmente casos de prueba.	Genera automáticamente casos básicos y asegura mayor cobertura en menos tiempo.
Consistencia en el código	Puede variar según el estilo y experiencia de los desarrolladores individuales.	Alta consistencia gracias a la generación de código estandarizado.
Costo inicial	Menor, ya que no se necesita suscripción a herramientas avanzadas.	Puede ser alto debido al costo de las licencias de herramientas basadas en IA como GitHub Copilot o Tabnine.

Curva de aprendizaje	Baja para desarrolladores con experiencia; puede ser más alta para principiantes.	Curva inicial moderada para entender cómo usar las herramientas de IA de manera eficiente.
Dependencia de herramientas	Independencia total de herramientas externas.	Alta dependencia de las capacidades y limitaciones de la herramienta de IA seleccionada.

Fuente. Elaboración Propia

2.3.5 La Productividad y la Calidad en el Desarrollo de Software

2.3.5.1 Definición de Productividad en el Desarrollo de Software

La productividad en el ámbito del desarrollo de software puede entenderse como la relación entre los resultados generados y los recursos empleados para alcanzarlos. De acuerdo con el concepto general proporcionado por la RAE, esta relación mide la eficacia de un proceso productivo en términos de eficiencia. En el contexto del desarrollo de software, la productividad se ve influenciada por diversos factores como inconvenientes técnicos en la codificación, falta de motivación en los proyectos, desequilibrios en la asignación de tareas o dinámicas de equipo que no fomenten la colaboración [20].

Identificar y abordar estos aspectos es esencial para optimizar tanto los procesos como los recursos, asegurando que los objetivos del proyecto se cumplan de manera eficiente y efectiva. Este enfoque permite a los equipos de desarrollo alcanzar un equilibrio adecuado entre el esfuerzo invertido y los resultados obtenidos, garantizando un mejor rendimiento general.

2.3.5.2 Indicadores de Calidad del Código

Según la publicación de AWS [21], la calidad del código puede evaluarse utilizando diversas métricas que permiten determinar su eficacia, claridad y capacidad de mantenimiento:

- Claridad: El código debe ser comprensible, con nombres adecuados y estructuras coherentes que faciliten su lectura.
- Errores: Es crucial identificar y reducir los defectos que puedan afectar el funcionamiento del código.
- Rendimiento: El código debe optimizar los recursos y mejorar la eficiencia operativa.
- Mantenibilidad: Es importante que el código sea fácil de modificar o extender en el futuro, asegurando su sostenibilidad mediante buenas prácticas de diseño y documentación.

2.3.5.3 Pruebas Unitarias y Tasas de Resolución de Errores

Según la publicación de Microsoft [22], las pruebas unitarias consisten en evaluar funciones o unidades de código de manera individual para asegurarse de que funcionen como se espera. Estas pruebas son fundamentales para detectar errores y confirmar que el código mantiene su integridad a medida que se desarrollan nuevas características o se realizan modificaciones. Las pruebas unitarias también ayudan a mejorar la calidad general del software al permitir la identificación temprana de fallos.

2.3.6 Eficiencia en Detección y Resolución de Errores

2.3.6.1 Concepto de Detección de Bugs y Errores

Según la publicación de IBM, la detección de errores en el software, también conocidos como bugs, consiste en identificar y gestionar fallos mediante un sistema eficiente de seguimiento. Este proceso es esencial para garantizar la calidad del software, ya que permite a los desarrolladores localizar, corregir rápidamente los errores y mejorar el rendimiento general del sistema. Con un adecuado control, se pueden evitar problemas antes de que afecten al usuario final, mejorando así la experiencia y la fiabilidad del software [23].

2.3.6.2 Impacto de GitHub Copilot en la Detección de Errores

Según la publicación de Microsoft sobre la depuración con GitHub Copilot, este asistente basado en inteligencia artificial tiene un impacto considerable en la identificación de errores durante el proceso de desarrollo de software. Copilot utiliza su capacidad de analizar el código y su contexto para ofrecer sugerencias inmediatas, lo que facilita la detección y corrección de fallos de manera rápida y precisa. Estas recomendaciones, que se basan en patrones y buenas prácticas, permiten a los desarrolladores abordar errores complejos que, de otro modo, podrían resultar difíciles de identificar [24].

Además, la integración de Copilot con herramientas de desarrollo como Visual Studio optimiza el proceso de depuración al permitir realizar pruebas de manera más eficiente. De este modo, se mejora la calidad del código y se acelera el ciclo de desarrollo, reduciendo el tiempo invertido en resolver problemas [24].

2.3.7 Revisión de Estudios Previos

La integración de la inteligencia artificial (IA) en el desarrollo de software ha generado transformaciones significativas en la productividad y la calidad del código. Herramientas como GitHub Copilot, impulsadas por modelos de lenguaje masivos, han permitido automatizar tareas repetitivas y optimizar procesos clave en el ciclo de desarrollo de software. Según Ebert y Louridas [32], estas herramientas son capaces de completar funciones, generar casos de prueba y realizar refactorizaciones, facilitando que los desarrolladores enfoquen su tiempo en tareas más creativas y complejas. Además, destacan que GitHub Copilot ha evolucionado hacia funcionalidades más avanzadas, como la comprensión del código y el apoyo en revisiones de solicitudes de cambios, consolidándose como un aliado valioso para equipos de desarrollo.

Por su parte, Udoidiok et al. [33] resaltan que la adopción de herramientas generativas como Copilot no solo incrementa la eficiencia operativa, sino que también reduce la dependencia de recursos humanos altamente calificados, enfrentando uno de los principales desafíos de la industria: la escasez de talento. En su estudio, se analiza cómo Copilot y herramientas similares, como Vercel y Meta AI, han demostrado ser efectivas para automatizar tareas rutinarias, mejorar la calidad del código y, al mismo tiempo, disminuir errores humanos.

Además de los beneficios en productividad, el impacto de estas herramientas en la calidad del código es destacado por Durrani et al. [34], quienes realizaron una revisión sistemática sobre el uso de IA en diversas fases de la ingeniería de software. Según su investigación, los algoritmos de aprendizaje automático y profundo han transformado actividades como la generación de pruebas, la detección de vulnerabilidades y la recomendación de código, mejorando métricas críticas como la cobertura de pruebas y la detección temprana de defectos. Los autores subrayan cómo herramientas basadas en IA han contribuido a optimizar tanto la eficiencia como la precisión en múltiples etapas del desarrollo de software.

Sin embargo, a pesar de sus ventajas, estas tecnologías presentan desafíos importantes. Udoidiok et al. [33] y Ebert y Louridas [32] destacan problemas relacionados con la interpretabilidad de los modelos, la privacidad del código y los riesgos asociados a la seguridad, los cuales deben ser abordados para maximizar los beneficios de estas herramientas en entornos de desarrollo profesional. Además, la dependencia de grandes volúmenes de datos para entrenar estos modelos plantea preocupaciones éticas y técnicas, que según Durrani et al. [34], requieren un enfoque continuo hacia la creación de modelos más explicables y éticos.

2.3.8 Método Cuasiexperimental

El diseño cuasiexperimental es una metodología empleada para evaluar el impacto de una intervención cuando no es posible asignar aleatoriamente a los participantes en grupos experimentales y de control. Este enfoque es especialmente útil en entornos prácticos como las empresas de desarrollo de software, donde los proyectos y los equipos operan bajo restricciones reales. A diferencia de los experimentos aleatorios, los estudios cuasiexperimentales trabajan con grupos preexistentes, lo que facilita su implementación en contextos empresariales [35].

En el ámbito del desarrollo de software, este método permite analizar el impacto de herramientas y tecnologías en indicadores clave como la productividad y la calidad del código. Por ejemplo, se puede medir cómo el uso de GitHub Copilot afecta el tiempo de desarrollo y la reducción de errores al comparar métricas antes y después de su adopción. Este tipo de diseño también resulta adecuado para evaluar metodologías ágiles o herramientas como SonarQube en entornos empresariales reales, asegurando que los resultados sean aplicables en situaciones prácticas [36].

La principal ventaja del diseño cuasiexperimental es su capacidad para trabajar con datos recogidos en condiciones reales, permitiendo una evaluación más cercana a los contextos operativos. Este enfoque es ideal para empresas que desean probar nuevas tecnologías o procesos sin interrumpir significativamente sus operaciones [35]. Sin embargo, la falta de aleatorización completa puede limitar la capacidad de atribuir causalidad directa entre la intervención y los resultados observados, ya que las diferencias en los grupos comparados podrían ser influenciadas por variables no controladas [36].

2.3.9 Variable Independiente: Uso de IA GitHub Copilot

2.3.9.1 Asistencia en la Generación de Código

Según la publicación de Microsoft, GitHub Copilot facilita la generación de código dentro de Visual Studio mediante la integración contextual. Al analizar el código ya escrito y la interacción del usuario, Copilot sugiere automáticamente líneas de código, explica fragmentos y proporciona correcciones. Esto optimiza la escritura de código al reducir la necesidad de completar manualmente fragmentos repetitivos o complejos, permitiendo a los desarrolladores centrarse en aspectos más creativos del desarrollo [25].

2.3.9.2 Reducción de Tareas Repetitivas con GitHub Copilot

GitHub Copilot transforma la interacción de los desarrolladores con su entorno de trabajo mediante el uso de modelos avanzados de inteligencia artificial creados por OpenAI. Su principal beneficio radica en acelerar el proceso de desarrollo de software, ofreciendo sugerencias de código instantáneas que permiten escribir más rápido. Esto reduce considerablemente el tiempo necesario para completar las tareas de programación, lo que a su vez aumenta la agilidad y eficiencia del equipo en la entrega de proyectos, mejorando la productividad general [13].

2.3.9.3 Frecuencia de Uso de IA GitHub Copilot

Según menciona [26], se investigó si los desarrolladores que utilizan GitHub Copilot experimentan mejoras más allá de simplemente aumentar la velocidad en la realización de tareas. Los principales resultados encontrados incluyen:

- Mayor satisfacción del desarrollador: Entre el 60% y el 75% de los participantes en la encuesta indicaron que, al usar GitHub Copilot, se sienten más satisfechos con su trabajo, enfrentan

menos frustración al programar y pueden concentrarse en tareas más satisfactorias. Esto refleja un cambio positivo en la experiencia laboral de los desarrolladores.

- Conservación de esfuerzo mental: Un porcentaje del 73% de los encuestados mencionó que GitHub Copilot les ayudó a mantener un estado de "flujo" en sus tareas, mientras que un 87% señaló que les ayudó a reducir el esfuerzo cognitivo durante las actividades repetitivas. Este aspecto es importante porque las interrupciones y los cambios de contexto pueden reducir la productividad, lo que GitHub Copilot parece mitigar.

Estos hallazgos sugieren que GitHub Copilot no solo optimiza la productividad, sino que también favorece el bienestar de los desarrolladores al momento que utilizan GitHub Copilot, contribuyendo a su satisfacción y ayudando a preservar su energía mental en tareas repetitivas.

2.3.10 Variable Dependiente: Productividad y Calidad de Desarrollo de Software

2.3.10.1 Teoría de la Productividad en el Desarrollo de Software

La productividad en el desarrollo de software mide la eficiencia con la que se utilizan los recursos para generar productos funcionales. Según [38], esta puede evaluarse mediante métricas como tareas completadas, funcionalidades entregadas o líneas de código útiles generadas en un tiempo determinado. Herramientas avanzadas y técnicas como la inteligencia artificial pueden optimizar estos procesos al reducir errores y mejorar la velocidad de desarrollo. Factores clave incluyen la experiencia del equipo y la implementación de metodologías consistentes que permitan aumentar el rendimiento sin comprometer la calidad.

2.3.10.2 Calidad en el Desarrollo de Software

La calidad del software se define por su capacidad de satisfacer necesidades explícitas e implícitas de los usuarios, incluyendo funcionalidad, seguridad, rendimiento y

mantenibilidad. De acuerdo con el modelo ISO/IEC 25010 [37], los atributos principales son:

- Funcionalidad: Cumplimiento de los requerimientos establecidos.
- Rendimiento: Respuesta y uso eficiente de recursos bajo diferentes cargas.
- Seguridad: Protección frente a accesos no autorizados y vulnerabilidades.
- Mantenibilidad: Facilidad para corregir, actualizar o adaptar el software.



Figura 1

Atributos ISO/IEC 25010

Fuente: https://www.invensity.com/consultoria/ingenieria-de-software/garantia-de-calidad-del-software/?lang=es

Estos atributos se evalúan a través de pruebas sistemáticas y metodologías específicas que garantizan el cumplimiento de estándares internacionales, promoviendo software confiable y robusto [37].

2.3.11 Calidad del Código

Según [21], se menciona que la calidad del código abarca aspectos fundamentales como la claridad, la ausencia de errores y la facilidad de mantenimiento. Además, menciona que:

- Descripción:

  La calidad del código se refiere a su claridad, comprensión y ausencia de errores, lo que facilita su mantenimiento y escalabilidad. Un código bien diseñado mejora la eficiencia, minimiza riesgos y asegura la productividad del equipo de desarrollo, siendo esencial para crear software eficiente y confiable.

- Medición:

  Se evalúa con métricas como el número de defectos, la legibilidad y el cumplimiento de estándares de codificación, identificando áreas para optimización y garantizando consistencia en los proyectos.

- Instrumento de medición:
  Entre las herramientas disponibles para evaluar la calidad del código se encuentran las ofrecidas por AWS, como CodeGuru, que emplean aprendizaje automático para analizar código, identificar errores, optimizar el rendimiento y sugerir mejoras alineadas con estándares establecidos. Estas herramientas automatizan el análisis y proporcionan recomendaciones prácticas para mejorar la calidad del código.
  Por otro lado [27] menciona otra Herramienta que es SonarQube, esta mejora la calidad del código mediante un análisis estático que detecta defectos, vulnerabilidades y áreas débiles, permitiendo a los desarrolladores corregir errores y seguir buenas prácticas. También evalúa métricas como la complejidad y la duplicación del código. Además, incorpora inteligencia artificial para proporcionar diagnósticos más precisos en tiempo real, facilitando la optimización del código a medida que se escribe.

2.3.12 Eficiencia en la Detección de Bugs

La eficiencia en la detección de errores es fundamental para optimizar el desarrollo de software. GitHub Copilot, mediante inteligencia artificial, permite a los desarrolladores identificar problemas en tiempo real mientras codifican. Esta integración mejora la productividad al alertar sobre errores de sintaxis y lógica antes de la ejecución del código.

El uso de herramientas automáticas basadas en IA puede reducir los errores en etapas tempranas del desarrollo hasta en un 40%, comparado con métodos de depuración manual lado [40]. Además, la combinación de herramientas de análisis estático con modelos de lenguaje natural ha mostrado ser efectiva para detectar defectos que podrían pasar desapercibidos, mejorando la calidad del software en producción [41].

2.3.13 Efectividad en Pruebas Unitarias

Las pruebas unitarias son esenciales para garantizar que cada componente del software funcione correctamente. GitHub Copilot mejora este proceso mediante la automatización de la generación de pruebas unitarias, lo que aumenta la cobertura del código y acelera las pruebas.

Un estudio comparativo mostró que herramientas como Copilot y Pynguin generan niveles similares de cobertura en términos de declaraciones (alrededor del 80%). Sin embargo, Copilot destaca por su capacidad para generar pruebas más adaptadas a códigos no modulares, una limitación de otras herramientas [40] [41]. Además, al utilizar enfoques iterativos, Copilot ayuda a mejorar la cobertura de pruebas, asegurando que los módulos individuales se verifiquen correctamente en una variedad de escenarios [41].

2.4 Marco Conceptual

2.4.1 Inteligencia Artificial (IA)

La inteligencia artificial (IA) consiste en un conjunto de tecnologías avanzadas que capacitan a las computadoras para llevar a cabo una amplia gama de funciones que tradicionalmente requerirían inteligencia humana. Estas funciones incluyen, entre otras, la capacidad de interpretar, procesar y generar lenguaje hablado y escrito, analizar grandes volúmenes de datos, reconocer patrones, hacer recomendaciones, realizar clasificaciones y traducir información de manera precisa. La IA desempeña un papel central en la innovación tecnológica actual, convirtiéndose en un elemento clave para optimizar procesos, generar valor para las empresas y mejorar la experiencia de los usuarios [39].

Entre sus aplicaciones prácticas, la IA permite automatizar tareas repetitivas, como extraer datos de imágenes o documentos mediante tecnologías como el reconocimiento óptico de caracteres (OCR), transformando datos no estructurados en formatos organizados y útiles. Asimismo, a nivel técnico, la IA se basa en algoritmos avanzados que utilizan modelos de aprendizaje automático y aprendizaje profundo para entrenarse con datos estructurados y no estructurados. Este proceso de entrenamiento les permite mejorar su rendimiento de manera continua, facilitando tareas como el análisis predictivo, la categorización de información y el procesamiento de lenguaje natural [39].

2.4.2 GitHub Copilot

Según [17] menciona que GitHub Copilot es una herramienta de asistencia en codificación que utiliza inteligencia artificial para facilitar y optimizar el trabajo de los desarrolladores. Este sistema se integra de forma nativa en los editores de texto y entornos de desarrollo

(IDEs), proporcionando sugerencias inteligentes a medida que se escribe código. También colabora en completar funciones y en la resolución de problemas técnicos.

Adicionalmente, Copilot permite la interacción mediante un sistema de chat que ofrece explicaciones detalladas, genera resúmenes para solicitudes de extracción y facilita el acceso a la documentación pertinente al proyecto en curso. Entre sus funcionalidades más destacadas se encuentran:

- Sugerencias adaptativas de código según el contexto de trabajo.
- Compatibilidad con una amplia gama de lenguajes de programación.
- Soporte para consultas interactivas desde IDEs o dispositivos móviles con GitHub Mobile.
- Capacidades específicas para entornos empresariales, como la automatización de descripciones en solicitudes de extracción y la creación de conocimientos personalizados a partir del contexto organizacional.

2.4.3 Productividad en el Desarrollo de Software

La productividad en el desarrollo de software se define como la capacidad de los equipos para crear software de alta calidad de manera eficiente, utilizando los recursos disponibles de forma óptima y reduciendo al mínimo el tiempo dedicado a tareas innecesarias. Este concepto no solo implica la rapidez en la ejecución de tareas, sino también la generación continua de entregables que sean útiles, escalables y alineados con las expectativas de los clientes [20].

2.4.4 Calidad del Código

La calidad del código hace referencia a la forma en que se ha diseñado, estructurado y escrito el software, con el fin de garantizar que sea eficiente, seguro y fácil de modificar en el futuro.

Esto implica que el código sea claro, organizado, y que siga prácticas estándar de programación. Un código de buena calidad permite que los desarrolladores detecten y corrijan errores con facilidad, lo que también facilita la colaboración en equipos y reduce los gastos en mantenimiento. Además, asegura que el software cumpla con los requisitos del proyecto de manera confiable y con un rendimiento óptimo [21].

2.4.5 Automatización de Pruebas

Las pruebas automatizadas son procesos en los que se utilizan herramientas y scripts para verificar el funcionamiento del software de forma automática. Estas pruebas permiten ejecutar pruebas repetitivas de manera eficiente, eliminando errores humanos y mejorando la velocidad de desarrollo. Son cruciales en los entornos de entrega continua, ya que aseguran la calidad del software sin la intervención manual constante, lo que agiliza la detección de problemas [42].

2.4.6 Beneficios de Utilizar Herramientas de IA

Según una publicación de [39] menciona que, la inteligencia artificial (IA) ofrece beneficios significativos a las empresas al optimizar la toma de decisiones a través del análisis de grandes cantidades de datos. Mejora la eficiencia operativa, automatiza tareas repetitivas, y permite personalizar experiencias para los usuarios. Estos avances favorecen la innovación, creando soluciones más precisas y eficaces que aportan un valor significativo a las organizaciones, ayudándolas a mejorar su productividad y competitividad en el mercado.

2.5 Marco Normativo y Legal

Marco Normativo:

- ISO/IEC 12207: Procesos del ciclo de vida del software.
- ISO/IEC 25010: Modelos de calidad del software.

Marco Legal:

- Ley sobre derechos de Autor (Decreto Legislativo N° 822): Ley que se encarga de proteger a todo autor de obras literarias, científicas o artísticas y su originalidad del plagio por parte de personas externas a los mismos.
- Ley de Promoción de la Innovación en Ciencia y Tecnología (Ley N° 30309, Perú): Establece incentivos para la investigación, desarrollo e innovación tecnológica, incluyendo proyectos que utilicen inteligencia artificial como GitHub Copilot, promoviendo su implementación en empresas y el desarrollo de software.

2.6 Teoría General de Sistemas (TGS)

La TGS, propuesta por Ludwig von Bertalanffy, define un sistema como un conjunto de elementos interrelacionados que trabajan conjuntamente para alcanzar un objetivo común [55]. En el desarrollo de software, este enfoque permite comprender el entorno de desarrollo como un sistema compuesto por herramientas, metodologías, desarrolladores, procesos y productos finales. GitHub Copilot puede considerarse un elemento que introduce nuevos flujos de interacción, afectando directamente las dinámicas internas del sistema y su eficiencia general.

Bertalanffy argumenta que los sistemas abiertos están en constante intercambio de energía e información con su entorno, lo que los hace susceptibles a mejoras y adaptaciones [56]. En este contexto, la integración de GitHub Copilot en los procesos de desarrollo se interpreta como un insumo externo que optimiza el flujo interno del sistema, mejorando métricas clave como el tiempo de desarrollo y la calidad del código.

Dentro de las diversas teorías de sistemas, se encuentran diferentes enfoques que resultan relevantes para el análisis de la optimización del desarrollo de software, los cuales serán

abordados a continuación, proporcionando una visión integral sobre la interacción de los componentes y su impacto en la eficiencia de los procesos de desarrollo.

- Teoría de Sistemas Abiertos

Según Ludwig von Bertalanffy, los sistemas abiertos están en constante interacción con su entorno, intercambiando recursos e información para adaptarse y evolucionar [57]. En el caso del desarrollo de software, GitHub Copilot se conceptualiza como un insumo externo que interactúa con el sistema de desarrollo. Esta herramienta permite optimizar el sistema al reducir tiempos de desarrollo y mejorar la calidad del código, todo ello al integrarse con herramientas, desarrolladores y metodologías existentes, lo que refuerza la capacidad adaptativa del sistema.

- Teoría de la Complejidad

Según el autor [58], la Teoría de la Complejidad, desarrollada por Edgar Morin, propone un enfoque que busca entender la realidad considerando la interconexión y la interacción entre diversos elementos que conforman un sistema. Morin sostiene que el pensamiento complejo es esencial para analizar fenómenos que no pueden ser comprendidos plenamente mediante enfoques lineales o reduccionistas.

En el contexto de esta investigación, la Teoría de la Complejidad es particularmente relevante. Los proyectos de software son sistemas complejos con múltiples dependencias y variabilidad. Aplicar el pensamiento complejo permite analizar cómo la introducción de una herramienta como GitHub Copilot afecta no solo componentes aislados, sino también las interacciones y dinámicas emergentes dentro del equipo de desarrollo. Este enfoque facilita una comprensión más profunda de cómo la asistencia automatizada influye en la calidad del código, la eficiencia en la detección de errores y la productividad general del sistema de desarrollo de software.

- Teoría de Sistemas de Control

Según [59], La Teoría de Sistemas de Control se basa en el principio de que un sistema puede ser regulado o ajustado mediante la manipulación de sus entradas para obtener salidas deseadas. Esta teoría se encuentra principalmente en el ámbito de la ingeniería, donde los sistemas se diseñan para mantener su estabilidad y optimizar su rendimiento. Además, los sistemas de control son aquellos que gestionan la relación entre las entradas y las salidas de un proceso, asegurando que el sistema mantenga un comportamiento estable y eficiente a lo largo del tiempo. La retroalimentación es una característica esencial en estos sistemas, ya que permite ajustar el comportamiento del sistema basado en las salidas previas, contribuyendo así a la estabilidad y optimización continua del sistema.

En el contexto de esta investigación, la Teoría de Sistemas de Control se utiliza para explicar cómo GitHub Copilot optimiza el proceso de desarrollo de software. Al igual que en los sistemas de control tradicionales, Copilot ajusta sus sugerencias de código basándose en las entradas (código previo escrito por el desarrollador), lo que produce salidas (fragmentos de código sugeridos) que a su vez mejoran la calidad y eficiencia del proceso. GitHub Copilot, al aprender del comportamiento de los desarrolladores y ajustarse en consecuencia, funciona como un sistema de retroalimentación que mejora la productividad, reduce el tiempo de desarrollo y optimiza la calidad del código, lo cual es crucial para la evaluación del impacto de la herramienta en el desarrollo de software en tu estudio.

CAPÍTULO III. METODOLOGÍA DE LA INVESTIGACIÓN

3.1 Diseño de Investigación

3.1.1 Diseño de la Investigación

Según los autores [43], el diseño cuasiexperimental se caracteriza por manipular una o más variables independientes para analizar su impacto en variables dependientes, aunque sin garantizar un control total sobre las variables externas ni emplear una asignación aleatoria de los participantes. Por lo general, este tipo de diseño utiliza grupos preexistentes, como aulas o comunidades, y aunque se busca minimizar las influencias externas, el control es parcial en comparación con los diseños experimentales.

En esta investigación de tesis, el diseño se clasifica como cuasiexperimental porque no se realiza una asignación aleatoria estricta de los proyectos a los grupos, sino que se trabaja con un único grupo preexistente. En este contexto, se compararon tres proyectos desarrollados inicialmente mediante una metodología tradicional y luego con el apoyo de GitHub Copilot. Este enfoque permite analizar la relación causa-efecto entre el uso de la inteligencia artificial y la optimización de los procesos de desarrollo de software, evaluando métricas clave como el tiempo de ejecución, la calidad del código y la eficiencia en la detección de errores.

3.1.2 Tipo de Investigación

Según los autores [43], la investigación aplicada se centra en resolver problemas específicos y prácticos, generando conocimientos que pueden ser utilizados directamente para mejorar una situación concreta o solucionar un problema en un contexto real. A diferencia de la investigación básica, cuyo propósito es expandir el conocimiento de manera general sin un fin inmediato, la investigación aplicada busca utilizar teorías y métodos para abordar necesidades particulares en

áreas como la sociedad, la industria o una disciplina específica. Su principal objetivo es ofrecer soluciones prácticas basadas en el conocimiento generado.

En el caso de esta investigación, el diseño se clasifica como aplicada porque busca resolver un problema concreto relacionado con la optimización de los procesos de desarrollo de software en una empresa de Lima. A través de un enfoque comparativo entre tres proyectos desarrollados utilizando métodos tradicionales y el mismo conjunto de proyectos con la ayuda de GitHub Copilot, se pretende generar soluciones prácticas para mejorar el tiempo de desarrollo, la calidad del código y la eficiencia en la detección y resolución de errores. Este diseño se enfoca en aplicar principios teóricos en un contexto específico y real, con el objetivo de obtener resultados inmediatos y útiles para la mejora en el desarrollo de software en la empresa.

3.1.3 Nivel de la Investigación

De acuerdo con los autores [43], el nivel explicativo en una investigación se emplea cuando el objetivo no es solo describir fenómenos, sino también entender las causas y el funcionamiento de estos. Este nivel permite identificar y analizar los factores que generan o influyen en los resultados observados, proporcionando una explicación detallada del porqué y cómo ocurren ciertos eventos. A través de este enfoque, se busca explorar las relaciones causales entre las variables involucradas, ofreciendo una comprensión profunda y detallada de las dinámicas estudiadas. En este tipo de investigación, no solo se observan los efectos de las variables, sino que también se indaga en las razones detrás de dichos efectos, lo que contribuye a formular teorías y modelos explicativos.

En el contexto de esta investigación, el nivel explicativo se adopta porque el objetivo no es únicamente observar cómo GitHub Copilot impacta los procesos de desarrollo de software, sino también investigar los mecanismos que hacen que esta herramienta influya en áreas clave como el tiempo de desarrollo, la calidad del código y la eficiencia en la identificación y corrección de

errores, en comparación con los métodos convencionales. Este enfoque permite explorar las conexiones causales entre el uso de GitHub Copilot y las mejoras en dichos procesos, identificando los factores que contribuyen a optimizar el desarrollo de software. De esta forma, se busca obtener una comprensión más completa del impacto real de la inteligencia artificial en este campo, lo que resulta crucial para tomar decisiones informadas sobre su implementación en el ámbito profesional.

3.1.4 Enfoque de la Investigación

Según el autor [43], el enfoque cuantitativo en investigación se centra en la recolección y análisis de datos numéricos con el fin de estudiar fenómenos de manera objetiva. Este enfoque se utiliza para medir variables específicas, analizar las relaciones entre ellas y establecer patrones, tendencias o diferencias significativas. A través de técnicas estadísticas, se busca obtener resultados que sean generalizables a una población más amplia, permitiendo una comprensión precisa y verificable del fenómeno estudiado.

En el contexto de esta investigación, el enfoque cuantitativo se utilizará para medir el impacto de GitHub Copilot en el proceso de desarrollo de software, comparando métricas específicas entre el uso de esta herramienta y los métodos tradicionales. Se recopilarían datos numéricos sobre el tiempo de desarrollo, la calidad del código y la eficiencia en la detección y resolución de errores. Estos datos serían analizados estadísticamente para evaluar las diferencias y establecer relaciones significativas entre el uso de GitHub Copilot y las mejoras observadas en los procesos de desarrollo. Este enfoque permitirá obtener resultados claros, objetivos y replicables que evidencien el impacto real de GitHub Copilot en la productividad y calidad del desarrollo de software en la empresa.

3.1.5 Población

De acuerdo con los autores [43], la población se refiere al conjunto total de elementos o individuos que poseen las características que la investigación pretende analizar. Según el tipo de estudio, esta

población puede ser limitada (finita) o ilimitada (infinita), pero debe estar bien definida para permitir una recolección de datos precisa y apropiada.

En el contexto de esta investigación, la población está conformada por los proyectos en una empresa de desarrollo de software en Lima que utiliza los métodos tradicionales en sus proyectos de desarrollo de software.

3.1.6 Muestra

De acuerdo con los autores [43], la muestra se refiere a un subgrupo seleccionado de la población total que participa en un estudio, con el objetivo de que sus características reflejen las de la población completa. Esta selección permite obtener resultados de manera más práctica y eficiente cuando no es posible estudiar a toda la población, ya sea por restricciones de tiempo, recursos o acceso. Para garantizar que la muestra sea representativa, se deben considerar aspectos como el tamaño de la población, el método de selección y el tamaño adecuado de la muestra.

En el contexto de esta investigación, la muestra se selecciona de manera intencional, ya que se han elegido tres proyectos específicos de desarrollo de software dentro de una empresa en Lima. Este enfoque de muestreo se basa en la selección de aquellos proyectos que permiten una comparación directa entre los dos métodos de desarrollo, con el fin de evaluar su impacto en el tiempo de desarrollo, la calidad del código y la eficiencia en la resolución de errores. De esta forma, no se busca que la muestra sea representativa de toda la población de proyectos de software, sino que se seleccionan de manera estratégica aquellos que proporcionen información relevante para los objetivos de la investigación. El análisis se centró en comparar el impacto de GitHub Copilot en áreas clave como el tiempo de desarrollo, la calidad del código y la eficiencia en la detección y resolución de errores, en comparación con los proyectos que siguen los métodos tradicionales.

3.1.7 Operacionalización de Variables

3.1.7.1 Variable 1

Tabla 3

Variable Independiente: Uso de GitHub Copilot

Dimensiones	Indicadores	Formula	Instrumento	Niveles	Técnica
Frecuencia de uso	Número de veces que se utiliza el GitHub Copilot en cada proyecto	-	Registro de uso de la extensión	Malo: 0 - 5 Sesiones Regular: 6 - 10 Sesiones Bueno: >11 Sesiones	Observación y análisis
Facilidad de uso	Número de prompts que se le pregunta a la herramienta por bloque de código	-	Registro de uso de la extensión	Malo: 0 - 5 Sesiones Regular: 6 - 10 Sesiones Bueno: >11 Sesiones	Observación y análisis
Exactitud del Código Generado	Porcentaje de líneas de código generadas correctamente sin corrección manual	LG/LT*100	Batch de conteo	Malo: <70% Regular: 70%-90% Bueno: 91%.100%	análisis cuantitativo
Calidad y Precisión del código	Número de bugs críticos identificados y corregidos de todo el código producido	-	Batch de conteo	Malo: >5 errores críticos Regular: 2 - 5 errores críticos Bueno: <=1 errores críticos	análisis cuantitativo

Leyenda

líneas de Código totales	LT
líneas de Código generadas	LG

Fuente. Elaboración Propia

3.1.7.2 Variable 2

Tabla 4

Variable Dependiente: Productividad y calidad en el desarrollo de software

Dimensiones	Indicadores	Formula	Instrumento	Niveles	Técnica
Tiempo de Desarrollo	Tiempo Total en horas/días dedicado a completar cada proyecto	(HT- HC) * (HT/100)	Jira: Permitirá el registro de tiempos por tareas	Malo: 0<=20% Regular: 20%<=50% Bueno: >50%	Cronometraje
Calidad del Código	porcentaje de código libre de errores	CC-CT	SonarQube	Malo: 0<=3% Regular: 4%<10% Bueno: >10%	análisis cuantitativo
detección de bugs	Número de bugs detectados y corregidos en el Código de desarrollo	(BT/100) * BC*100	SonarQube	Malo: >5(81%-100%) Regular: 1<=5 (1% - 80%) Bueno: 0 %	Pruebas técnicas
Efectividad en Pruebas Unitarias	Porcentaje de pruebas exitosas realizadas frente al total de pruebas ejecutadas	CE/CP*100	Herramienta de pruebas unitarias (Visual Studio)	Malo: <50% Regular: 51%-99% Bueno: 100%	análisis de resultados

Leyenda:

Hora tradicional	HT	Porcentaje Calidad Tradicional	CT	Bugs Tradicional	BT	Cantidad de pruebas	CP
Hora Copilot	HC	Porcentaje Calidad Copilot	CC	Bugs Copilot	BC	Cantidad de pruebas exitosas con Copilot	CE

Fuente. Elaboración Propia

3.2 Instrumentos de Investigación / Herramientas

3.2.1. Técnicas

Para el desarrollo de esta investigación, se emplean diversas técnicas que permiten abordar y analizar de manera integral los aspectos relacionados con el tema de estudio.

La primera técnica consistió en el Análisis de Métricas de Tiempo de Desarrollo, lo que permitió medir el tiempo invertido en los proyectos. Para ello, se utilizaron herramientas de gestión de proyectos ágiles como Jira y Trello, las cuales facilitan un seguimiento detallado de las tareas y el tiempo dedicado a cada actividad [44]. Las métricas fueron recopiladas mediante el registro de los tiempos de inicio y finalización de cada sprint y tarea.

La segunda técnica fue la Evaluación de la Calidad del Código, llevada a cabo mediante el uso de SonarQube, una herramienta ampliamente reconocida por medir indicadores clave como complejidad ciclomática, duplicación de código y presencia de errores. Según [45], SonarQube resulta esencial en el análisis estático de código, ya que permite identificar vulnerabilidades y asegurar el cumplimiento de estándares de calidad.

La tercera técnica se centró en la Medición de Eficiencia en la Detección de Bugs, diseñada para evaluar la identificación y resolución de errores al comparar el código desarrollado con GitHub Copilot y el generado mediante el método tradicional. Para ello, se analizaron los errores identificados durante el proceso de desarrollo y se calculó la tasa de resolución de errores, definida como el porcentaje de errores solucionados frente al total detectado. Según [46], los sistemas de gestión de calidad en el desarrollo de software, incluidos métodos para rastrear y resolver errores son esenciales para un análisis exhaustivo del rendimiento del equipo y la optimización de procesos.

Finalmente, se aplicaron Pruebas Unitarias utilizando códigos desarrollados tanto de manera tradicional como con GitHub Copilot. La efectividad de las pruebas fue evaluada en función de la cobertura de pruebas y el número de errores detectados. En el enfoque tradicional, las pruebas fueron realizadas sobre el código escrito manualmente, mientras que en el uso de GitHub Copilot, se evaluó la calidad de las pruebas generadas con la asistencia de la IA, comparando su eficacia en la detección de errores. Según [47], el Desarrollo Guiado por Pruebas (TDD) es una metodología fundamental para garantizar la calidad del software, ya que fomenta la identificación temprana de errores. Por otro lado, [48] resalta que las pruebas unitarias son clave para la detección temprana de errores, mejorando la calidad del software al reducir los defectos.

3.2.2. Instrumentos

Jira permite a los equipos crear, asignar y realizar el seguimiento de tareas con el fin de optimizar el tiempo de desarrollo y garantizar el cumplimiento de los plazos. Según los autores [60] enfatizan cómo Jira facilita la gestión de proyectos ágiles mediante la visualización de las tareas y la distribución de los recursos, mejorando la eficiencia en el desarrollo de software. En esta investigación, Jira se utilizará para medir el tiempo de desarrollo de los proyectos de software antes y después de la implementación de GitHub Copilot. Esto permitirá comparar la eficiencia en el uso del tiempo y determinar si GitHub Copilot contribuye a reducir los tiempos de desarrollo, como se espera en la hipótesis de que la herramienta acelera el proceso de programación.

SonarQube es una plataforma de código abierto para la gestión de la calidad del código que permite analizar la calidad del software, detectando problemas como errores, vulnerabilidades y deudas técnicas. Según los autores [60], SonarQube facilita la

identificación y corrección de defectos en el código a través de un análisis continuo, proporcionando retroalimentación inmediata sobre la calidad del software. Por lo cual, en esta investigación, SonarQube será crucial para evaluar la calidad del código generado por GitHub Copilot en comparación con el código producido mediante métodos tradicionales. El análisis de la calidad del código te permitirá medir la precisión y exactitud del código generado por la IA, identificando errores críticos, vulnerabilidades y la deuda técnica. Además, la detección de bugs y el tiempo de resolución, medidos por SonarQube, proporcionarán datos objetivos sobre la capacidad de GitHub Copilot para mejorar la fiabilidad del software en comparación con el desarrollo tradicional.

Las pruebas unitarias son una técnica de prueba de software en la que se verifica el funcionamiento correcto de unidades individuales de código. Según el autor [62], las pruebas unitarias son fundamentales para el desarrollo ágil, ya que permiten detectar errores en etapas tempranas, mejorar la calidad del código y facilitar los cambios sin temor a introducir errores. En esta investigación, al utilizar pruebas unitarias con GitHub Copilot, se podrá evaluar su capacidad para mejorar la precisión y la eficiencia de las pruebas, así como la capacidad de detectar y corregir errores de manera más rápida y precisa que los métodos tradicionales. Esta medición será clave para verificar si el uso de Copilot genera un código con menos errores y mejora la productividad del equipo de desarrollo.

El registro de uso es un mecanismo que permite realizar un seguimiento de la cantidad de veces que un usuario interactúa con una herramienta o extensión específica, como GitHub Copilot. Este tipo de seguimiento es esencial para medir la frecuencia con la que los desarrolladores utilizan la herramienta, así como para captar su percepción sobre la facilidad de uso. Según los autores [63], el registro de uso de extensiones permite a los desarrolladores

recopilar datos sobre patrones de interacción y uso, lo que proporciona información valiosa sobre la adopción y efectividad de la herramienta. Por lo que, en esta investigación, este instrumento se utilizará para evaluar la frecuencia de uso de GitHub Copilot durante el desarrollo de proyectos. Además, permitirá medir la facilidad de uso a través de la cantidad de sesiones utilizadas por los desarrolladores, lo que está vinculado a la percepción subjetiva de su efectividad. Estos datos serán relevantes para determinar cómo la adopción y el uso frecuente de GitHub Copilot afectan la productividad y calidad del código generado.

El batch de conteo es un proceso de análisis cuantitativo que se utiliza para evaluar la precisión y calidad del código generado. Este instrumento evalúa el número de líneas de código correctamente generadas sin necesidad de correcciones manuales. Según el autor [64], los métodos de conteo son eficaces para medir la exactitud del código al comparar los resultados generados automáticamente con los resultados esperados, permitiendo una evaluación objetiva de la calidad del código. Por lo que, en esta investigación, el batch de conteo se utilizará para calcular el porcentaje de código generado correctamente por GitHub Copilot sin intervención manual. Esto será fundamental para medir la exactitud del código generado y la calidad de este en comparación con el código escrito manualmente.

3.2.3. Validez

según la norma internacional ISO 9241-210 menciona que el desarrollo de prototipos tecnológicos debe enfocarse en pruebas de funcionalidad y usabilidad sin necesidad de validar deforma formal los instrumentos. Esto se debe a que el enfoque principal está en los resultados prácticos obtenidos mediante las pruebas, más que los instrumentos utilizados para recolectar datos.

La norma establece lineamiento para el diseño centrado en el usuario, priorizando la evaluación de la funcionalidad y usabilidad de los prototipos tecnológicos. Este enfoque resulta particularmente relevante en el análisis comparativo de herramientas de software, como GitHub Copilot ya que se enfoque a métricas clave como tiempo de desarrollo, calidad del código y eficiencia en la resolución de errores.

En este estudio de investigación se adoptan los principios de esta norma para garantizar que los métodos de evaluación sean prácticos y enfocados en resultados medibles. De acuerdo ISO 9241-210 no es necesario una validación formal de los instrumentos, lo cual respalda herramientas como SonarQube y metodologías agiles para recopilar datos cuantitativos evaluando indicadores de productividad y calidad.

Esta normativa refuerza la importancia del uso de las pruebas funcionales y unitarias, las cuales proporcionan una visión objetiva del impacto del GitHub Copilot en el desarrollo de software en comparación con métodos tradicionales.

3.2.4. Confiabilidad

Basado en la norma ISO 9241-210 no será necesario realizar las pruebas de confiabilidad de instrumentos debido a que la evaluación está centrada en pruebas funcionales y usabilidad de herramientas tecnológicas, las cuales generalmente se utiliza métricas objetivas y estándares ya validados en el ámbito del desarrollo de software.

En esta investigación se utilizará instrumentos tecnológicos ampliamente validados en el desarrollo de software, como SonarQube, Jira y Visual Studio. Estos instrumentos garantizan la fiabilidad de las métricas generadas gracias a su diseño automatizado y estandarizado. Estas herramientas registran datos de manera automática minimizando posibles errores humanos y asegurando la consistencia de la medición.

En la investigación se llevó a cabo un análisis preliminar en tres proyectos para verificar que GitHub Copilot ayuda optimizar el tiempo de desarrollo de software y mejora la calidad del código.

De lo anterior, no se requiere realizar pruebas de confiabilidad estadística ya que los instrumentos realizados cuentan con reconocimiento y validación en la comunidad científica y tecnológica.

3.2.5. Análisis de Recolección de Datos

En este estudio, el análisis de los datos se realiza utilizando un enfoque cuantitativo, con el apoyo de herramientas como SonarQube y Jira. [49] destaca que estas herramientas son fundamentales para evaluar la productividad y la calidad del código, permitiendo realizar una comparación detallada entre los proyectos desarrollados con GitHub Copilot y aquellos creados mediante métodos tradicionales.

Tabla 5

Análisis de Datos

Categoría	Dimensión	Indicador	Herramienta	Método de Recolección	Niveles de Análisis
Productividad	Tiempo de desarrollo	Tiempo total dedicado a completar cada proyecto	Jira	Registro de tiempos por tareas	Malo: 0-20% Regular: 20%-50% Bueno: >50%
Calidad del código	Porcentaje de código libre de errores	Porcentaje de código sin bugs críticos	SonarQube	Análisis automatizado	Malo: 0-5% Regular: 3%-10% Bueno: >20%

Detección de errores	Detección de bugs	Número de bugs detectados y corregidos	SonarQube	Pruebas técnicas	Malo: >5 Regular: 1-5 Bueno: 0
Comparación de métodos	Comparación de productividad	Tiempo promedio por tarea entre métodos	Jira	Datos comparativos	Métodos tradicionales vs. GitHub Copilot
Comparación de métodos	Comparación de calidad	Errores críticos promedio por método	SonarQube	Datos comparativos	Métodos tradicionales vs. GitHub Copilot

Elaboración. Fuente Propia

3.3. Metodología de la Implementación de la Solución

3.3.1. Selección de la Metodología

En esta investigación, se optó por utilizar el marco de trabajo ágil Scrum como metodología. De acuerdo con [54], Scrum es ampliamente reconocido en proyectos de desarrollo de software gracias a su enfoque iterativo y adaptable, que facilita la implementación y evaluación rápida de nuevas herramientas. Este marco resulta especialmente adecuado para comparar los resultados obtenidos con GitHub Copilot frente a metodologías tradicionales, ya que permite gestionar ciclos cortos de desarrollo (Sprint) y realizar evaluaciones continuas.

La decisión de emplear Scrum también responde a su eficacia para gestionar proyectos con requisitos en constante cambio, permitiendo una implementación controlada de herramientas innovadoras como GitHub Copilot, dentro de un entorno dinámico y estructurado para la investigación.

3.3.2. Desarrollo de la Metodología

Fase 1: Planificación Inicial

En la fase inicial, se seleccionaron los proyectos de software a evaluar dentro de una empresa de Lima dedicada a la producción de software. Estos proyectos fueron desarrollados en dos grupos: uno utilizando GitHub Copilot y otro sin el uso de la herramienta, siguiendo las metodologías tradicionales de desarrollo. El objetivo fue comparar los tiempos de desarrollo, la calidad del código y la eficiencia en la resolución de errores.

Fase 2: Implementación de la Herramienta y Desarrollo

Durante la implementación de la herramienta, los equipos de desarrollo fueron capacitados en el uso de GitHub Copilot y en su integración con herramientas de pruebas unitarias. Se establecieron períodos de trabajo (Sprint) de dos semanas, siguiendo las mejores prácticas del marco de trabajo Scrum. Cada sprint incluyó las siguientes actividades:

- Desarrollo de Código: Los desarrolladores utilizaron GitHub Copilot para asistir en la escritura del código, reduciendo tareas repetitivas y generando sugerencias automáticas para completar fragmentos de código.
- Desarrollo Tradicional: El segundo grupo desarrolló el mismo proyecto de software sin la asistencia de GitHub Copilot, utilizando únicamente técnicas tradicionales de programación.
- Pruebas Unitarias: Al final de cada sprint, ambos grupos de trabajo realizaron pruebas unitarias utilizando JUnit y pytest para identificar errores en el código.

Fase 3: Recolección de Datos y Métricas

Durante cada sprint, se recopilaron diversas métricas para evaluar la eficiencia y la calidad del código generado. Las principales métricas que se midieron incluyen:

- Tiempo de Desarrollo: Se registró el tiempo total invertido en cada tarea, utilizando Jira para el seguimiento del progreso y la duración de cada sprint.
- Calidad del Código: Se utilizó SonarQube para medir la calidad del código generado, evaluando aspectos como la complejidad, cobertura de pruebas, y la presencia de errores críticos.
- Eficiencia en la Detección y Resolución de Errores: Se contabilizó el número de errores detectados y corregidos en ambos grupos, comparando la efectividad de las pruebas unitarias.
- Frecuencia de Uso de GitHub Copilot: Se monitoreó la frecuencia de uso de la herramienta en cada proyecto y la cantidad de líneas de código autocompletadas por GitHub Copilot.

Fase 4: Análisis de Resultados

Una vez que se completaron los sprints y se recolectaron los datos, se procedió al análisis comparativo entre los dos grupos. El análisis se centró en tres variables clave:

- Tiempo de Desarrollo: Se compararon los tiempos invertidos en el desarrollo de cada tarea, identificando si el uso de GitHub Copilot resultó en una reducción significativa en el tiempo de desarrollo.
- Calidad del Código: A través de las métricas de SonarQube, se evaluó si el código generado con GitHub Copilot presentó menos errores o si la herramienta ayudó a mejorar la calidad en términos de mantenibilidad y seguridad.
- Eficiencia en la Detección de Errores: Se comparó la cantidad de errores detectados y corregidos durante las pruebas unitarias, analizando si GitHub Copilot permitió detectar más errores o reducir el número de fallos.

3.4. Metodología Para la Medición de Resultados de la Implementación

3.4.1. Metodología de Medición

La metodología de medición se dividió en tres etapas principales:

- Recolección de datos de tiempo de desarrollo: Se registraron los tiempos de inicio y finalización de cada tarea utilizando herramientas de gestión de proyectos como Trello.
- Evaluación de la calidad del código: Se utilizó SonarQube para medir métricas clave como la complejidad ciclomática, la cobertura de pruebas y la mantenibilidad del código.
- Detección de errores: Se comparó la eficiencia en la identificación y resolución de errores entre los códigos generados con y sin GitHub Copilot, utilizando pruebas unitarias.

3.4.2. Métrica de la Variable Dependiente

La variable dependiente en este estudio es la productividad y calidad en el desarrollo de software, evaluada a través de las siguientes métricas:

- Tiempo de desarrollo: Se midió en horas por tarea, comparando los resultados entre los dos enfoques. (Tabla generación Código Copilot / Generación
- Calidad del código: Se evaluó con indicadores de mantenibilidad, seguridad y ausencia de errores críticos, utilizando reportes de SonarQube.
- Eficiencia en la detección de errores: Se midió como el porcentaje de errores identificados y corregidos en las pruebas unitarias respecto al total de errores existentes.

3.5. Cronograma de Trabajo

3.5.1. Cronograma de Actividades

Tabla 6

Cronograma de actividades

Actividades	Semana 1-2	Semana 3-4	Semana 5-6	Semana 7-8	Semana 9-10	Semana 11-12	Semana 13-14	Semana 15-16
1. Revisión bibliográfica y definición del marco teórico								
2. Selección y revisión de los tres proyectos de software								
3. Planificación y preparación de entornos de desarrollo								
4. Desarrollo con GitHub Copilot de los proyectos								
5. Recolección de datos (tiempo, calidad de código, bugs)								
6. Análisis de datos recolectados								
7. Elaboración de resultados y discusión								
8. Redacción del informe final								
9. Revisión y corrección del informe final								

Nota. El cronograma propuesto de investigación consta de 16 semanas donde se ha previsto los tiempos para cumplir con los objetivos.

Fuente. Elaboración Propia

3.6. Costos Directos

Los costos directos son aquellos que se vinculan de manera específica con la realización de un proyecto y que pueden identificarse y asignarse fácilmente a una actividad particular. De acuerdo con [52], entre estos costos se encuentran los materiales, la mano de obra y las herramientas destinadas exclusivamente a un proyecto en particular. Estos gastos son esenciales para determinar el presupuesto total de una iniciativa.

3.6.1. Costos Materiales

Tabla 7

Costos Materiales

Materiales y Software	**Descripción**	**Cantidad**	**Costo Unitario (PEN)**	**Costo Total (PEN)**
Licencia GitHub Copilot	Suscripción mensual para desarrollo de proyectos	6 meses	S/ 38.00	S/ 228.00
Licencia SonarQube (Developer Edition)	Licencia para análisis de calidad de código	1 año	S/ 570.00	S/ 570.00
Jira Software	Suscripción para gestión de proyectos	1 año	S/ 380.00	S/ 380.00
SPSS	Licencia para análisis estadístico	1	S/ 1,140.00	S/ 1,140.00
Computadora	Equipo para realizar pruebas y análisis	1	S/ 3,800.00	S/ 3,800.00
	Total		S/. 6118.00	

Fuente. Elaboración Propia

3.6.2. Costos de Personal

Tabla 8

Costos Personal

Personal	Coste/hora	Horas	Costo/Dia	Mes	Total
Pizarro Romero, Angelo Patrick	S/.25.00	8	S/.200.00	1	S/.6000.00
Paredes Orihuela, Jose Luis	S/.30.00	8	S/.240.00	1	S/.7200.00
Total			S/. 13200.00		

Fuente. Elaboración Propia

3.7. Costos Indirectos

Los costos indirectos son aquellos que no se pueden asignar directamente a una actividad específica, pero que son necesarios para el funcionamiento general del proyecto. Según [52], estos incluyen elementos como servicios administrativos, alquiler de oficinas y otros gastos generales que permiten la operación del proyecto.

Tabla 9

Costos Indirectos

Descripción	Cantidad	Cuota Diaria	Costo Mensual	Total
Servicio de Luz	2	S/. 5.00	S/. 150.00	S/. 300.00
Servicio de Internet	2	S/. 3.00	S/. 90.00	S/. 180.00
Total			S/. 480.00	

Fuente. Elaboración Propia

3.8. Costos Fijos

Según [52], explica que los costos fijos son aquellos que se mantienen constantes sin importar la cantidad de trabajo ejecutado o el nivel de producción alcanzado. En el contexto de la gestión de proyectos, estos costos incluyen aspectos como el arrendamiento de oficinas, licencias de software a largo plazo.

Tabla 10

Costos Fijos

Descripción	Cantidad	Coste Unitario	Total
SQL Server	2	S/. 0.00	S/. 0.00
.NET Core	2	S/. 0.00	S/. 0.00
Github Copilot Business	1	$/. 19.00	$/. 19.00
Total		$/. 19.00	

Fuente. Elaboración Propia

3.9. Costos Variables

Los costos variables son aquellos que se ajustan según el nivel de actividad del proyecto, como el número de empleados necesarios o la duración de las tareas. Tal como señalan [53], estos costos varían en función del volumen de trabajo y deben ser tomados en cuenta para asegurar un presupuesto adaptable y realista.

Tabla 11

Costos Variables

Categoría	Recurso	Descripción	Monto	Total
Gastos de Variables	Ninguno	Ninguno	S/. 0.00	S/. 0.00
Total			S/. 0.00	

Fuente. Elaboración Propia

3.10. Presupuesto

Con el propósito de calcular el presupuesto del proyecto y gestionar los costos asociados a su desarrollo, se creó una tabla de presupuesto que recopila y suma los diferentes costos mencionados previamente. Esto permite obtener una visión más clara sobre cómo se destinarán los recursos económicos dentro del proyecto.

3.10.1. Resumen Total de Costos

Tabla 12

Presupuesto Total de Costos

Descripción	Costo Total descripción
Costos Directos	S/. 19318.00
Costos Indirectos	S/. 480.00
Costos Fijo	S/. 70.90
Costos Variables	S/. 0
Total	S/. 19868.90

Fuente. Elaboración Propia

CAPÍTULO IV: DESARROLLO DE LA SOLUCIÓN

4.1. Propuesta de la Solución

4.1.1. Fase de Planificación

4.1.1.1. Identificación de Actores del Sistema.

En el proceso de desarrollo de la solución, es esencial plantear la identificación de los actores clave del sistema que desempeñan roles fundamentales en la implementación de los tres proyectos seleccionados.

Proyecto 1: Api Crosseling Front.

- Asesor

Actor encargado de interactuar directamente con los usuarios o clientes, ofreciendo productos o servicios complementarios y resolver consultas o solicitudes relacionadas con la plataforma.

Proyecto 2: Crosseling Front.

- Supervisor.

Responsable de realizar tareas como la actualización de información, la búsqueda de datos relevantes y la generación de reportes del sistema, los cuales son utilizados para elaborar informes de gestión y evaluar el rendimiento de las operaciones.

- Back Office

Actor que realiza tareas internas relacionadas con el procesamiento de datos y la resolución de problemas técnicos que no pueden ser abordados directamente por el asesor. También gestiona solicitudes y da continuidad a completarse la venta.

- Asesor

Actor encargado de interactuar directamente con los usuarios o clientes, ofreciendo productos o servicios complementarios y resolver consultas o solicitudes relacionadas con la plataforma.

Proyecto 3: Gestor Consultas y Reclamos

- Administrador

Actor responsable de gestionar y dar seguimiento a las solicitudes presentadas por los colaboradores, tanto activos como cesados de la empresa. Este actor se encarga de procesar y resolver cada solicitud conforme a su naturaleza y los lineamientos establecidos.

- Colaborador

Actor encargado de presentar solicitudes relacionadas con consultas o reclamos derivados de cualquier inconveniente que haya experimentado, siguiendo los procedimientos definidos para su atención.

- Excolaborador

Actor encargado de presentar solicitudes relacionadas con consultas o reclamos derivados de cualquier inconveniente que haya experimentado, siguiendo los procedimientos definidos para su atención.

4.1.1.2. Identificación de Roles del Proyecto.

Tabla 13

Roles y definiciones

ROL	DESCRIPCIÓN	RESPONSABLE
Cliente	Persona representativa de la empresa encargada de proporcionar los datos necesarios para la implementación.	Johnny Villagaray
Desarrollador	Encargado de desarrollar y mantener el código fuente necesario para la implementación de la solución.	Angelo Pizarro / Jose Paredes

Administrador de Base de Datos	Persona encargada de crear, organizar y gestionar la estructura y diseño de la base de datos que soporta la solución.	Angelo Pizarro
Control de Calidad	Responsable de realizar las pruebas necesarias para verificar que la solución cumpla con los estándares de calidad establecidos.	Jose Paredes
líder	Encargado de organizar, coordinar y velar por el cumplimiento de los plazos y avances del proyecto.	Jose Paredes

Fuente. Elaboración propia

4.1.1.3. Requerimientos

En esta sección se detallan los requerimientos tanto funcionales como no funcionales del sistema, los cuales son esenciales para definir sus capacidades y características necesarias. Los requerimientos funcionales describen las acciones y tareas específicas que el sistema debe ser capaz de realizar, como la administración de datos y la interacción con los usuarios, mientras que los requerimientos no funcionales se enfocan en factores como la seguridad, el rendimiento, la escalabilidad y la facilidad de uso, los cuales garantizan que el sistema sea eficiente y de calidad.

- Requerimientos Funcionales

Proyecto 1: Api Crosseling Front.

Tabla14

Requerimientos funcionales Proyecto 1

Código	Requerimiento Funcional
RF01	Permitir que los asesores de registro de su llamada gestionada.
RF02	El programa deberá listar el su registro tipificado.
RF03	El asesor podrá modificar su registro realizado
RF04	El asesor podrá eliminar su registro realizado.

Fuente. Elaboración propia

Proyecto 2: Crosseling Front.

Tabla15

Requerimientos funcionales Proyecto 2

Código	Requerimiento Funcional
RF01	Permitir que los asesores de primer nivel hagan registro de su llamada gestionada.
RF02	El programa deberá listar las solicitudes en la vista gestión.
RF03	Solo se deberá mostrar toda la lista de registros a los BO/supervisores/coordinadores, mientras a los propios asesores solo deberán visualizar sus propios registros.
RF04	El asesor BO deberá poder visualizar todo el registro del asesor nivel 1 y poder modificar algunos campos.
RF05	El registro debe poder mostrar una bitácora de las solicitudes.
RF06	Cada registro realizado o modificado debe contar con estados.
RF07	Cada lista registrada debe mostrar un detallado o resumen del registro realizado.

Fuente. Elaboración propia

Proyecto 3: Gestor Consultas y Reclamos

Tabla16

Requerimientos funcionales Proyecto 3

Código	Requerimiento Funcional
RF01	La vista usuario de realizar solicitudes se mostrará en un apartado de la intranet de la empresa.
RF02	La vista de solicitudes debe mostrar los datos del trabajador, su ceco, un campo de texto manejable que permita subir imágenes.
RF03	Se debe contar con un enlace externo para trabajadores cesados de la empresa
RF04	El gestor de solicitudes del programa solo debe tener acceso algunos miembros de la empresa.
RF05	El programa debe poder listar todas las solicitudes realizadas.

RF06	El administrador debe poder gestionar y visualizar los campos que lleno el colaborador.
RF07	El sistema debe contar con una bitácora de cada solicitud.
RF08	Para el caso de los asesores cesados, la solicitud gestionada debe poder enviarse y mostrarse por correo personal del colaborador.
RF09	Para que el trabajador activo de la empresa, en la misma vista que realiza la solicitud debe poder visualizar el estado y detalle de su solicitud que haya sido gestionada.

Fuente. Elaboración propia

- Requerimientos No Funcionales

Los requerimientos No funcionales serán en general para los tres proyectos seleccionados.

Tabla17

Requerimientos no funcionales

ítem	Requerimientos No Funcionales
Rendimiento	El sistema debe ser capaz de procesar hasta 1000 solicitudes simultáneas sin afectar su desempeño.
Escalabilidad	El sistema debe permitir la incorporación de nuevos agentes y clientes sin afectar el rendimiento.
Seguridad	Los datos del cliente deben ser cifrados tanto en tránsito como en reposo, asegurando la confidencialidad y solo se debe poder acceder desde la misma red de la empresa.
Disponibilidad	El sistema debe estar disponible al menos el 99.9% del tiempo, con mantenimiento programado fuera del horario laboral.
Usabilidad	El sistema debe ser intuitivo, con una interfaz amigable para agentes sin necesidad de una formación extensiva.
Compatibilidad	El sistema debe ser compatible con los principales navegadores web (Chrome, Firefox, Safari) y dispositivos móviles.

Mantenibilidad	El sistema debe estar diseñado para facilitar su mantenimiento y actualización, con documentación detallada.
Tiempo de Respuesta	El sistema debe responder a las solicitudes del usuario en un tiempo no mayor a 2 segundos en un entorno de uso promedio.
Accesibilidad	El sistema debe cumplir con los estándares de accesibilidad web (WCAG 2.1) para usuarios con discapacidades.

Fuente. Elaboración propia

4.1.1.4. Identificación de Tablas, Actores y Flujos

- Modelo del Proceso de Negocio.

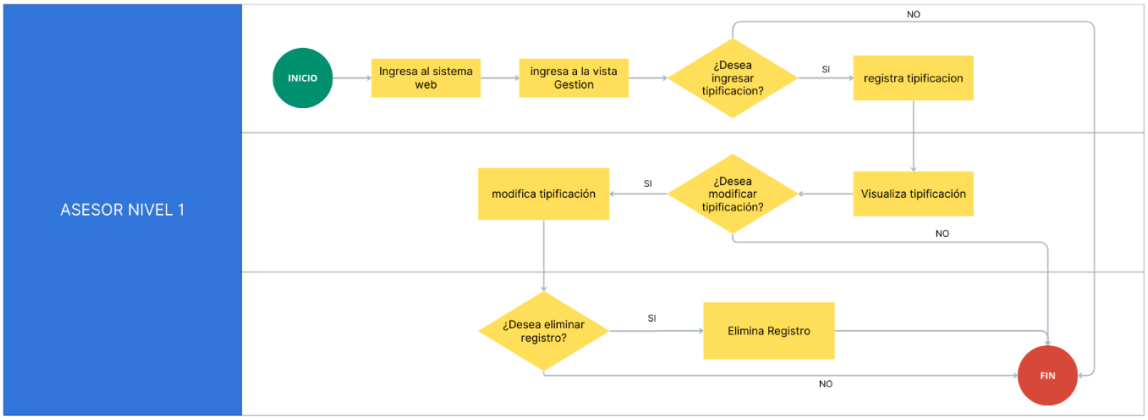


Figura 2. Modelo del Proceso de Negocio: Api Crosseling Front

Fuente: Elaboración Propia

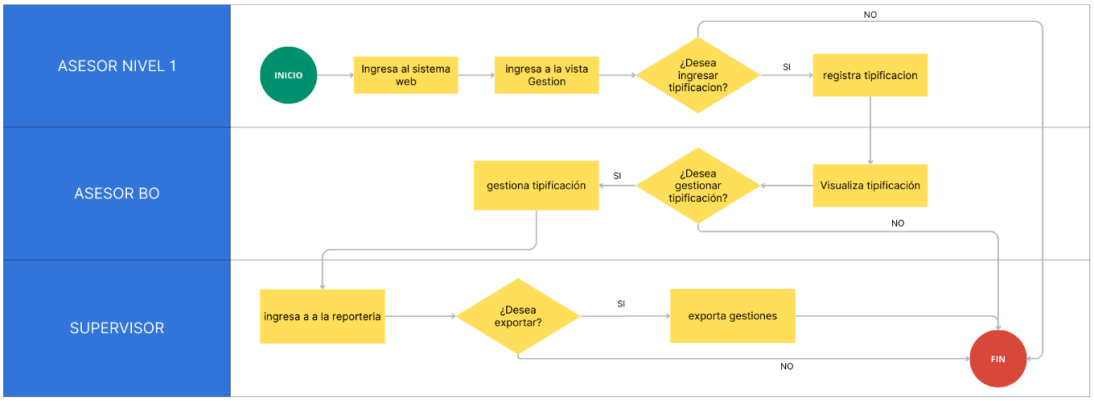


Figura 3. Modelo del Proceso de Negocio: Crosseling Front

Fuente: Elaboración Propia

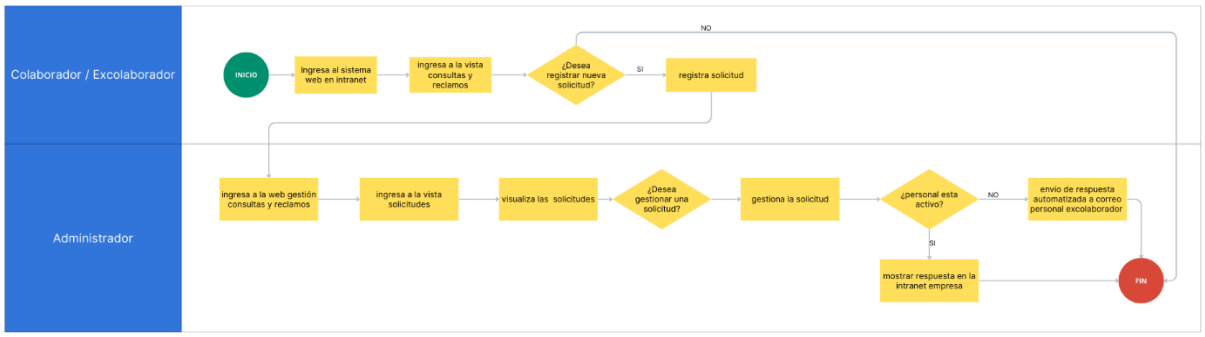


Figura 4. Modelo del Proceso de Negocio: Gestor Consultas y Reclamos

Fuente: Elaboración Propia

- Diagrama Modelo Relacional de la Base de Datos.

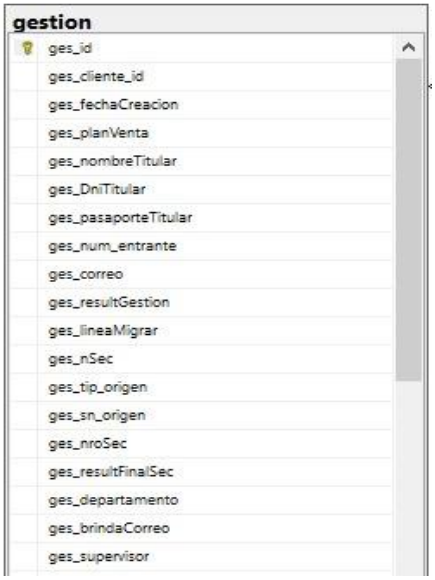


Figura 5. Diagrama Modelo Relacional: Api Crosseling Front

Fuente: Elaboración Propia

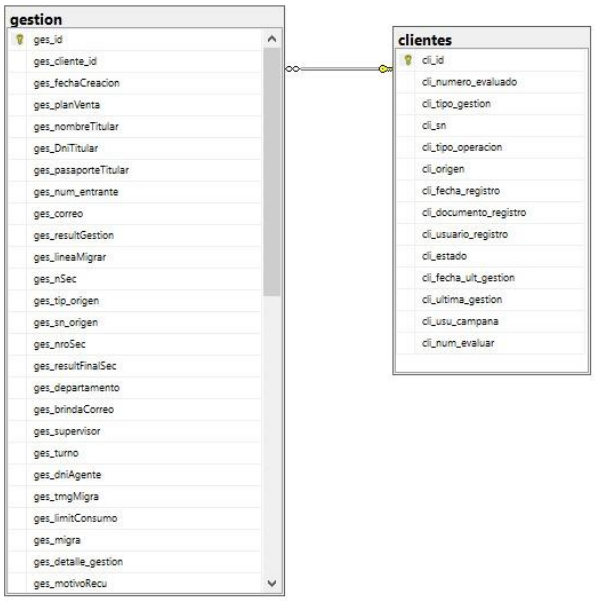


Figura 6. Diagrama Modelo Relacional: Crosseling Front

Fuente: Elaboración Propia



Figura 7. Diagrama Modelo Relacional: Gestor Consultas y Reclamos

Fuente: Elaboración Propia

4.1.2. Fase de Estimación

La fase de estimación se basa en las historias de usuario (HU) obtenidas de los tres proyectos definidos en el sistema. Cada historia de usuario fue descompuesta en tareas para poder estimar el esfuerzo necesario en su desarrollo. Las tablas a continuación muestran la planificación del backlog, con la descripción de cada HU y su respectiva estimación de tiempo.

4.1.2.1. Backlog.

El backlog se ha organizado de acuerdo con los tres proyectos: Api Crosseling Front, Crosseling Front.y Gestor Consultas y Reclamos. En cada uno, se detalla la funcionalidad y el actor involucrado. A continuación, se presentan las tablas con el detalle de cada historia de usuario.

Tabla 18

Proyecto 1: Api Crosseling Front

Historia de Usuario	Descripción	Actor	Estimación (horas)
HU01	Obtener todas las gestiones.	Usuario	10 (desarrollo) + 2 (pruebas)
HU02	Obtener una gestión por ID.	Usuario	8 (desarrollo) + 2 (pruebas)
HU03	Crear una nueva gestión.	Usuario	12 (desarrollo) + 3 (pruebas)
HU04	Eliminar una gestión por ID.	Usuario	6 (desarrollo) + 1 (pruebas)

Fuente. Elaboración propia

Tabla 19

Proyecto 2: Crosseling Front

Historia de Usuario	Descripción	Actor	Estimación (horas)
HU05	Registrar nueva gestión de cliente.	Agente	14 (desarrollo) + 4 (pruebas)
HU06	Actualizar datos de una gestión existente.	Agente	10 (desarrollo) + 3 (pruebas)
HU07	Consultar información de una gestión registrada.	Agente	9 (desarrollo) + 2 (pruebas)

HU08	Registrar datos adicionales sobre el cliente.	Agente	11 (desarrollo) + 3 (pruebas)
HU09	Generar reportes por rango de fechas.	Supervisor	10 (desarrollo) + 3 (pruebas)
HU10	Exportar reportes en formato descargable.	Supervisor	8 (desarrollo) + 2 (pruebas)
HU11	Consultar historial de reportes generados.	Supervisor	9 (desarrollo) + 3 (pruebas)
HU12	Auditar cambios realizados en las gestiones.	Administrador	8 (desarrollo) + 2 (pruebas)
HU13	Registrar quién realiza la gestión.	Administrador	7 (desarrollo) + 2 (pruebas)
HU14	Controlar acceso al módulo de gestión.	Administrador	6 (desarrollo) + 1 (pruebas)

Fuente. Elaboración propia

Tabla 20

Proyecto 3: Gestor Consultas y Reclamos

Historia de Usuario	Descripción	Actor	Estimación (horas)
HU15	Búsqueda de reclamos por rango de fechas.	Usuario	10 (desarrollo) + 3 (pruebas)
HU16	Exportar usuarios cesados.	Usuario	8 (desarrollo) + 2 (pruebas)
HU17	Gestionar estado de reclamos.	Personal	12 (desarrollo) + 3 (pruebas)
HU18	Registrar un nuevo reclamo.	Usuario cesado	15 (desarrollo) + 4 (pruebas)
HU19	Asignar responsables a reclamos.	Supervisor	10 (desarrollo) + 3 (pruebas)
HU20	Gestionar respuestas a reclamos.	Personal	12 (desarrollo) + 3 (pruebas)
HU21	Visualizar dashboard de reclamos.	Administrador	9 (desarrollo) + 3 (pruebas)
HU22	Auditar cambios en los reclamos.	Administrador	8 (desarrollo) + 2 (pruebas)
HU23	Gestionar notificaciones sobre cambios en reclamos.	Usuario	10 (desarrollo) + 3 (pruebas)

Fuente. Elaboración propia

4.1.2.2. Planificación y Estimación de Tareas

La planificación de las tareas asociadas a las historias de usuario se distribuye en Sprint, utilizando un enfoque ágil que permite el desarrollo iterativo. Cada sprint se compone de un conjunto de HU, que son implementadas y probadas antes de proceder al siguiente ciclo de desarrollo.

Tabla 21

Estimación de Tareas

Sprint	Historias de Usuario Implementadas	Estimación Total (horas)
Sprint 1	HU01, HU02, HU03, HU04 (Proyecto 1)	42 horas
Sprint 2	HU05, HU06, HU07, HU08, HU09 (Proyecto 2)	54 horas
Sprint 3	HU10, HU11, HU12, HU13, HU14 (Proyecto 2)	44 horas
Sprint 4	HU15, HU16, HU17, HU18, HU19 (Proyecto 3)	58 horas
Sprint 5	HU20, HU21, HU22, HU23 (Proyecto 3)	46 horas

Fuente. Elaboración propia

4.1.3. Fase de Implementación

La fase de implementación fue realizada a través de sprints ágiles, donde cada historia de usuario se codificó y probó utilizando GitHub Copilot como herramienta principal de asistencia en la programación. Para garantizar consistencia en la interacción con la herramienta, se empleó un prompt estándar diseñado específicamente para este proyecto, el cual se detalla en el Anexo 6. A continuación, se describen las actividades y resultados de cada sprint.

Sprint 1: Desarrollo de funcionalidades básicas del API Crosseling Front

En este sprint, se incluyó la implementación de controladores, servicios y modelos en el framework .NET CORE, asegurando la correcta replicación de las funcionalidades requeridas. Además, los prompts y solicitudes realizadas en GitHub Copilot para este desarrollo se pueden visualizar en el Anexo 3.

- **HU01: Crear Gestión**

  Tareas:

  1. Crear endpoint para registrar una nueva gestión.
  2. Validar los datos de entrada para asegurar consistencia.
  3. Implementar lógica de negocio para registrar gestiones en la base de datos.
  4. Realizar pruebas unitarias para validar el correcto funcionamiento del endpoint.

- **HU02: Obtener Gestiones**

  Tareas:

  1. Crear endpoint para listar gestiones existentes.
  2. Implementar paginación para manejar grandes volúmenes de datos.
  3. Realizar pruebas unitarias para asegurar el correcto retorno de datos.

- **HU03: Eliminar Gestión**

  Tareas:

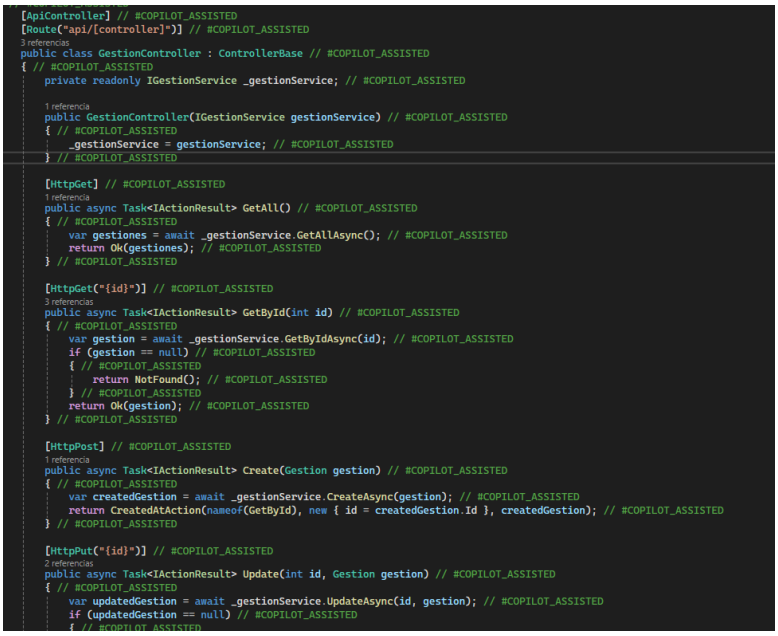
  1. Crear endpoint para eliminar una gestión específica.
  2. Validar que la gestión a eliminar exista en la base de datos.
  3. Implementar medidas de seguridad para evitar eliminaciones accidentales.
  4. Realizar pruebas unitarias para verificar que las eliminaciones se realicen correctamente.

- **HU04: Actualizar Gestión**

  Tareas:

  1. Crear endpoint para actualizar la información de una gestión existente.
  2. Validar que los datos actualizados cumplan con los estándares definidos.

3. Implementar la lógica para actualizar los registros en la base de datos.
4. Realizar pruebas unitarias para asegurar que los cambios se apliquen sin errores.



```
[ApiController] // #COPILOT_ASSISTED
[Route("api/[controller]")] // #COPILOT_ASSISTED
3 referencias
public class GestionController : ControllerBase // #COPILOT_ASSISTED
{ // #COPILOT_ASSISTED
    private readonly IGestionService _gestionService; // #COPILOT_ASSISTED

    1 referencia
    public GestionController(IGestionService gestionService) // #COPILOT_ASSISTED
    { // #COPILOT_ASSISTED
        _gestionService = gestionService; // #COPILOT_ASSISTED
    } // #COPILOT_ASSISTED

    [HttpGet] // #COPILOT_ASSISTED
    1 referencia
    public async Task<IActionResult> GetAll() // #COPILOT_ASSISTED
    { // #COPILOT_ASSISTED
        var gestiones = await _gestionService.GetAllAsync(); // #COPILOT_ASSISTED
        return Ok(gestiones); // #COPILOT_ASSISTED
    } // #COPILOT_ASSISTED

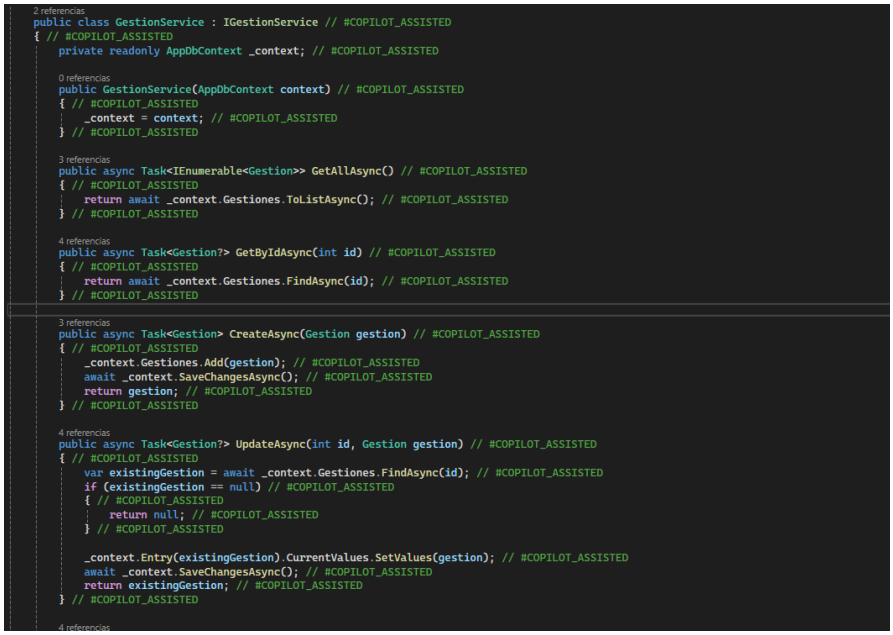
    [HttpGet("{id}")] // #COPILOT_ASSISTED
    3 referencias
    public async Task<IActionResult> GetById(int id) // #COPILOT_ASSISTED
    { // #COPILOT_ASSISTED
        var gestion = await _gestionService.GetByIdAsync(id); // #COPILOT_ASSISTED
        if (gestion == null) // #COPILOT_ASSISTED
        { // #COPILOT_ASSISTED
            return NotFound(); // #COPILOT_ASSISTED
        } // #COPILOT_ASSISTED
        return Ok(gestion); // #COPILOT_ASSISTED
    } // #COPILOT_ASSISTED

    [HttpPost] // #COPILOT_ASSISTED
    1 referencia
    public async Task<IActionResult> Create(Gestion gestion) // #COPILOT_ASSISTED
    { // #COPILOT_ASSISTED
        var createdGestion = await _gestionService.CreateAsync(gestion); // #COPILOT_ASSISTED
        return CreatedAtAction(nameof(GetById), new { id = createdGestion.Id }, createdGestion); // #COPILOT_ASSISTED
    } // #COPILOT_ASSISTED

    [HttpPut("{id}")] // #COPILOT_ASSISTED
    2 referencias
    public async Task<IActionResult> Update(int id, Gestion gestion) // #COPILOT_ASSISTED
    { // #COPILOT_ASSISTED
        var updatedGestion = await _gestionService.UpdateAsync(id, gestion); // #COPILOT_ASSISTED
        if (updatedGestion == null) // #COPILOT_ASSISTED
        { // #COPILOT_ASSISTED
```

Figura 8. Controlador de la API

Fuente: Elaboración Propia



```
2 referencias
public class GestionService : IGestionService // #COPILOT_ASSISTED
{ // #COPILOT_ASSISTED
    private readonly AppDbContext _context; // #COPILOT_ASSISTED

    0 referencias
    public GestionService(AppDbContext context) // #COPILOT_ASSISTED
    { // #COPILOT_ASSISTED
        _context = context; // #COPILOT_ASSISTED
    } // #COPILOT_ASSISTED

    3 referencias
    public async Task<IEnumerable<Gestion>> GetAllAsync() // #COPILOT_ASSISTED
    { // #COPILOT_ASSISTED
        return await _context.Gestiones.ToListAsync(); // #COPILOT_ASSISTED
    } // #COPILOT_ASSISTED

    4 referencias
    public async Task<Gestion?> GetByIdAsync(int id) // #COPILOT_ASSISTED
    { // #COPILOT_ASSISTED
        return await _context.Gestiones.FindAsync(id); // #COPILOT_ASSISTED
    } // #COPILOT_ASSISTED

    3 referencias
    public async Task<Gestion> CreateAsync(Gestion gestion) // #COPILOT_ASSISTED
    { // #COPILOT_ASSISTED
        _context.Gestiones.Add(gestion); // #COPILOT_ASSISTED
        await _context.SaveChangesAsync(); // #COPILOT_ASSISTED
        return gestion; // #COPILOT_ASSISTED
    } // #COPILOT_ASSISTED

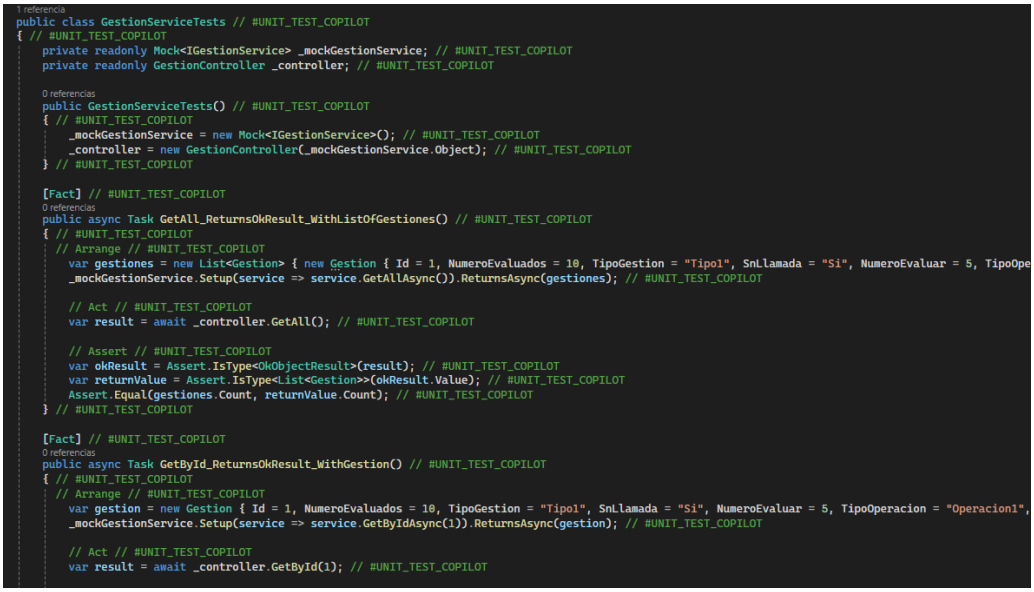
    4 referencias
    public async Task<Gestion?> UpdateAsync(int id, Gestion gestion) // #COPILOT_ASSISTED
    { // #COPILOT_ASSISTED
        var existingGestion = await _context.Gestiones.FindAsync(id); // #COPILOT_ASSISTED
        if (existingGestion == null) // #COPILOT_ASSISTED
        { // #COPILOT_ASSISTED
            return null; // #COPILOT_ASSISTED
        } // #COPILOT_ASSISTED

        _context.Entry(existingGestion).CurrentValues.SetValues(gestion); // #COPILOT_ASSISTED
        await _context.SaveChangesAsync(); // #COPILOT_ASSISTED
        return existingGestion; // #COPILOT_ASSISTED
    } // #COPILOT_ASSISTED

    4 referencias
```

Figura 9. Implementación del servicio de la API

Fuente: Elaboración Propia



```
1 referencia
public class GestionServiceTests // #UNIT_TEST_COPILOT
{ // #UNIT_TEST_COPILOT
    private readonly Mock<IGestionService> _mockGestionService; // #UNIT_TEST_COPILOT
    private readonly GestionController _controller; // #UNIT_TEST_COPILOT

    0 referencias
    public GestionServiceTests() // #UNIT_TEST_COPILOT
    { // #UNIT_TEST_COPILOT
        _mockGestionService = new Mock<IGestionService>(); // #UNIT_TEST_COPILOT
        _controller = new GestionController(_mockGestionService.Object); // #UNIT_TEST_COPILOT
    } // #UNIT_TEST_COPILOT

    [Fact] // #UNIT_TEST_COPILOT
    0 referencias
    public async Task GetAll_ReturnsOkResult_WithListOfGestiones() // #UNIT_TEST_COPILOT
    { // #UNIT_TEST_COPILOT
        // Arrange // #UNIT_TEST_COPILOT
        var gestiones = new List<Gestion> { new Gestion { Id = 1, NumeroEvaluados = 10, TipoGestion = "Tipo1", SnLlamada = "Si", NumeroEvaluar = 5, TipoOpe
        _mockGestionService.Setup(service => service.GetAllAsync()).ReturnsAsync(gestiones); // #UNIT_TEST_COPILOT

        // Act // #UNIT_TEST_COPILOT
        var result = await _controller.GetAll(); // #UNIT_TEST_COPILOT

        // Assert // #UNIT_TEST_COPILOT
        var okResult = Assert.IsType<OkObjectResult>(result); // #UNIT_TEST_COPILOT
        var returnValue = Assert.IsType<List<Gestion>>(okResult.Value); // #UNIT_TEST_COPILOT
        Assert.Equal(gestiones.Count, returnValue.Count); // #UNIT_TEST_COPILOT
    } // #UNIT_TEST_COPILOT

    [Fact] // #UNIT_TEST_COPILOT
    0 referencias
    public async Task GetById_ReturnsOkResult_WithGestion() // #UNIT_TEST_COPILOT
    { // #UNIT_TEST_COPILOT
        // Arrange // #UNIT_TEST_COPILOT
        var gestion = new Gestion { Id = 1, NumeroEvaluados = 10, TipoGestion = "Tipo1", SnLlamada = "Si", NumeroEvaluar = 5, TipoOperacion = "Operacion1",
        _mockGestionService.Setup(service => service.GetByIdAsync(1)).ReturnsAsync(gestion); // #UNIT_TEST_COPILOT

        // Act // #UNIT_TEST_COPILOT
        var result = await _controller.GetById(1); // #UNIT_TEST_COPILOT
```

Figura 10. Pruebas Unitarias API

Fuente: Elaboración Propia

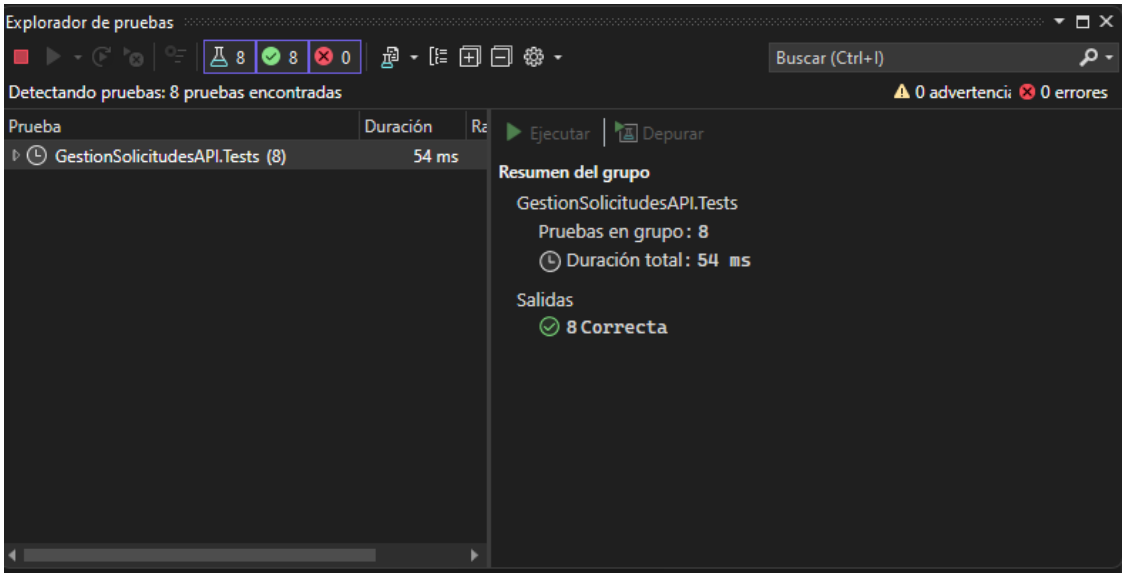


Figura 11. Ejecución de pruebas unitarias API

Fuente: Elaboración Propia

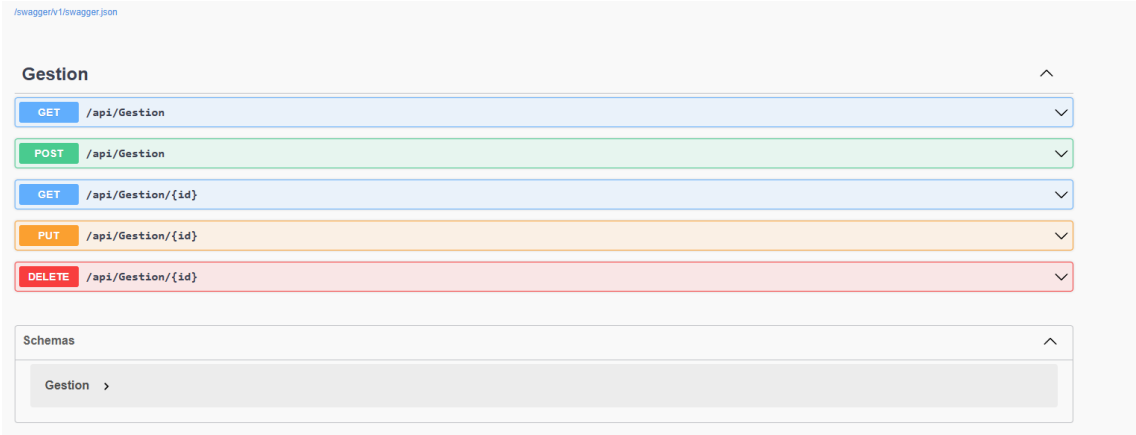


Figura 12. Interfaz de prueba unitaria API

Fuente: Elaboración Propia

Resultados del Sprint:

El sistema ahora permite obtener, crear, actualizar y eliminar gestiones de manera eficiente, cumpliendo con los requisitos de las historias de usuario. Se realizaron pruebas de integración y pruebas unitarias generadas automáticamente con GitHub Copilot para validar el funcionamiento de los servicios API, garantizando la calidad del código desarrollado.

Sprint 2: Implementación del Crosseling Front

En este sprint, se incluyó la implementación de controladores y modelos en el framework CodeIgniter, asegurando la correcta replicación de las funcionalidades requeridas.

Además, los prompts y solicitudes realizadas en GitHub Copilot para este desarrollo se pueden visualizar en el Anexo 4.

Historias de Usuario Implementadas:

- **HU05: Registrar Cliente**

  Tareas:

  1. Implementar el método en el controlador para manejar la lógica del registro de nuevos clientes.
  2. Crear la función correspondiente en el modelo para insertar datos en la base de datos.
  3. Validar los datos antes de enviarlos al modelo para evitar registros duplicados.
  4. Generar pruebas unitarias para asegurar que el método funcione correctamente bajo diferentes escenarios.

- **HU06: Actualizar Información de Cliente**

  Tareas:

1. Implementar el método en el controlador para manejar las actualizaciones de clientes existentes.
2. Añadir una función en el modelo para realizar actualizaciones en la base de datos de manera eficiente.
3. Validar que los datos ingresados sean consistentes antes de proceder con la actualización.
4. Generar pruebas unitarias para verificar que el método gestione correctamente las actualizaciones.

- **HU07: Consultar Gestión de Clientes**

  Tareas:

  1. Crear el método que consulte los clientes en el controlador para obtener las gestiones asociadas a un cliente.
  2. Implementar la lógica en el modelo para realizar consultas a la base de datos con filtros.
  3. Validar los permisos de usuario para garantizar que solo usuarios autorizados accedan a los datos.
  4. Realizar pruebas unitarias para validar la funcionalidad y las respuestas de las consultas.

- **HU08: Generar Reportes por Rango de Fechas**

  Tareas:

  1. Implementar el método en el controlador para manejar la lógica de generación de reportes.

2. Crear funciones en el modelo para procesar y devolver datos filtrados por fechas.
3. Generar pruebas unitarias para validar que los datos en los reportes sean correctos.

- **HU09: Autenticación para Acceso a Reportes**

Tareas:

1. Crear método para validar el acceso de usuarios autenticados a la funcionalidad de reportes.
2. Implementar pruebas unitarias para garantizar la seguridad en el acceso.

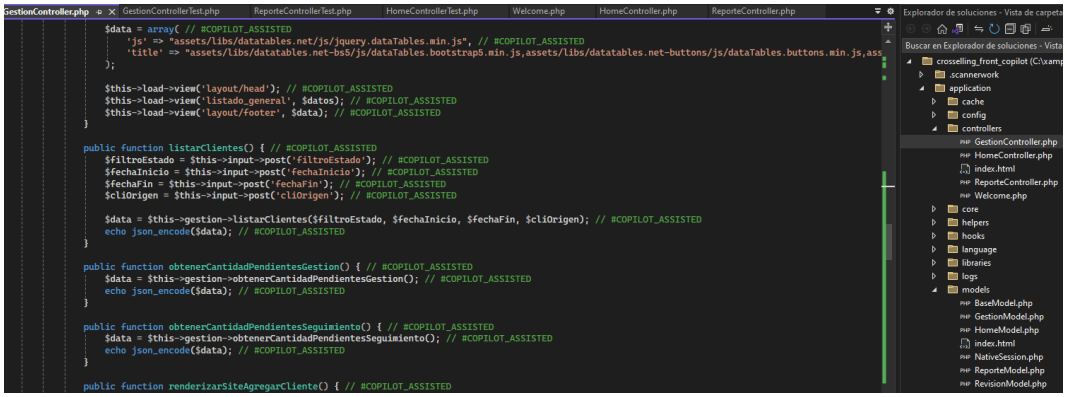


Figura 13. Controlador de Gestión Crosseling Front

Fuente: Elaboración Propia

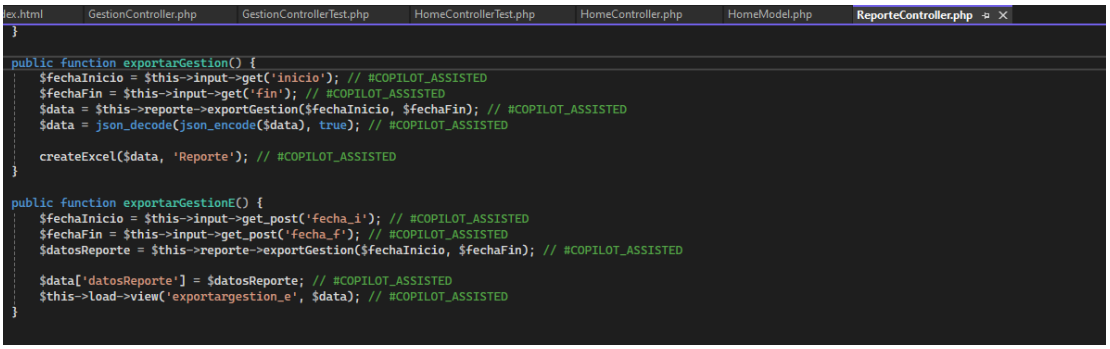


Figura 14. Controlador del reporte Codeigniter

Fuente: Elaboración Propia

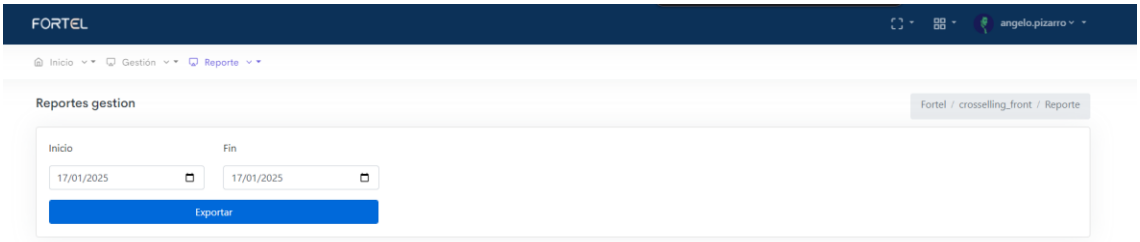


Figura 15. Vista reporte Crosseling front

Fuente: Elaboración Propia

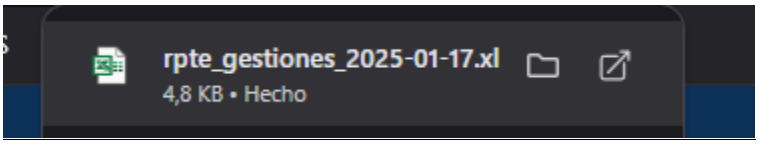


Figura 16. Reporte de Croselling front

Fuente: Elaboración Propia

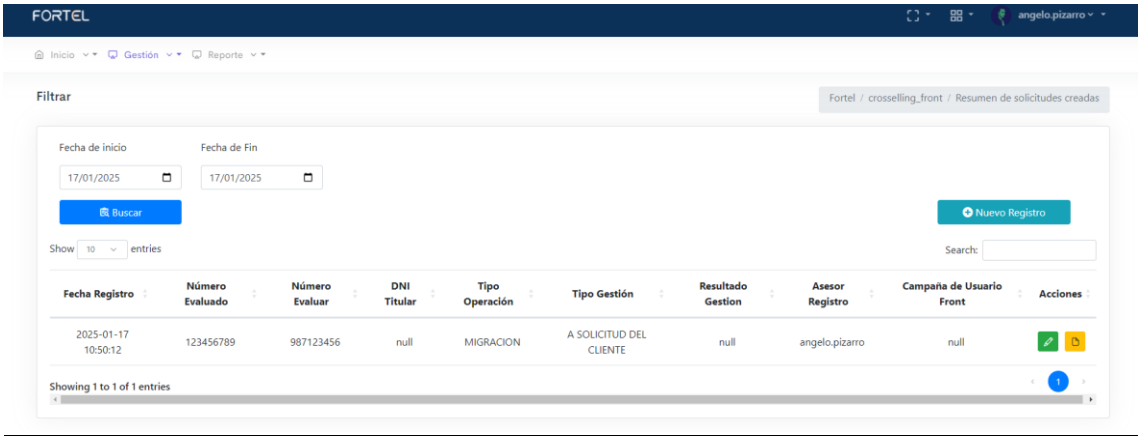


Figura 17. Vista reporte Crosseling front

Fuente: Elaboración Propia

Resultados del Sprint:

El sistema ahora permite registrar, actualizar y consultar gestiones de clientes. Además, se habilitó la funcionalidad de generación de reportes por rango de fechas, garantizando seguridad y precisión en la información proporcionada. Las pruebas unitarias generadas con el apoyo de GitHub Copilot aseguraron la calidad y consistencia del backend implementado.

Sprint 3: Finalización de funcionalidades del Crosseling Front

Continuando con el enfoque backend, se utilizó el prompt estándar descrito en el Anexo 6 para generar controladores y modelos en CodeIgniter. GitHub Copilot asistió en la generación del código, así como en las pruebas unitarias para validar las nuevas funcionalidades.

Historias de Usuario Implementadas:

- **HU10: Exportar Reportes**

  Tareas:

1. Implementar el método en el controlador para exportar datos en formatos CSV y PDF.
2. Crear funciones en el modelo para obtener los datos estructurados para exportación.
3. Generar pruebas unitarias para validar la integridad de los datos exportados.

- **HU11: Auditoría de Cambios**

  Tareas:

  1. Añadir un método en el controlador para registrar automáticamente los cambios realizados en gestiones y clientes.
  2. Crear tablas en la base de datos mediante el modelo para almacenar registros de auditoría.
  3. Implementar pruebas unitarias para validar que los registros de auditoría se generan correctamente.

- **HU12: Control de Acceso por Roles**

  Tareas:

  1. Desarrollar modulos en el backend para verificar los permisos de acceso según roles de usuario.
  2. Implementar un método en el modelo para validar las credenciales y roles desde la base de datos.
  3. Generar pruebas unitarias para asegurar la correcta validación de roles en el sistema.

- **HU13: Gestión de Alertas Automáticas**

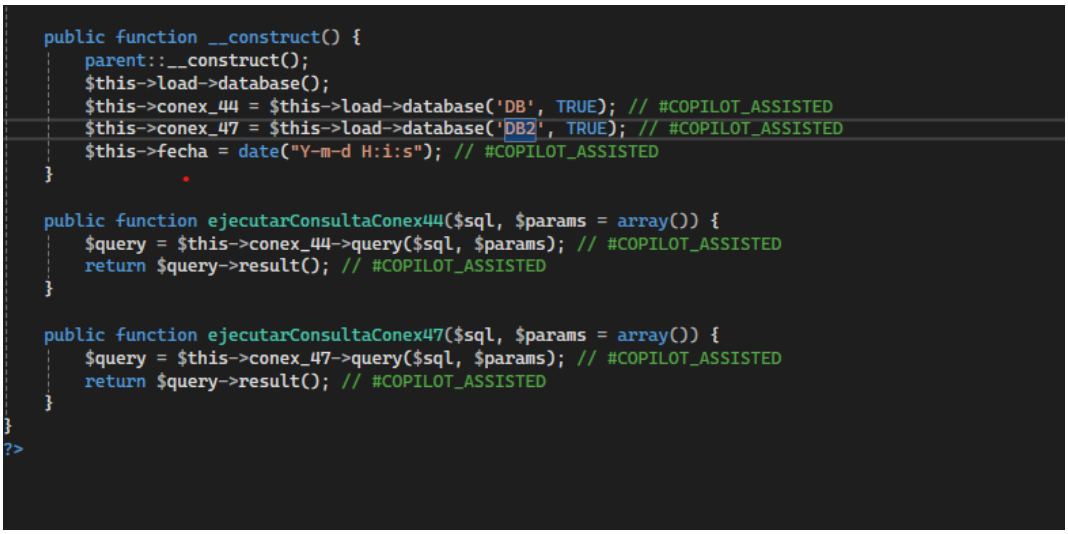
  Tareas:

1. Crear un método en el controlador para enviar alertas automáticas basadas en eventos del sistema.
2. Implementar lógica en el modelo para identificar eventos críticos y disparar alertas.
3. Generar pruebas unitarias para validar la ejecución correcta de las alertas.

- **HU14: Configuración de Parámetros del Sistema**

  Tareas:

  1. Desarrollar un módulo en el controlador para permitir la configuración dinámica de parámetros.
  2. Crear funciones en el modelo para almacenar y recuperar los parámetros desde la base de datos.
  3. Validar la integridad de los cambios mediante pruebas unitarias.



```
    public function __construct() {
        parent::__construct();
        $this->load->database();
        $this->conex_44 = $this->load->database('DB', TRUE); // #COPILOT_ASSISTED
        $this->conex_47 = $this->load->database('DB2', TRUE); // #COPILOT_ASSISTED
        $this->fecha = date("Y-m-d H:i:s"); // #COPILOT_ASSISTED
    }

    public function ejecutarConsultaConex44($sql, $params = array()) {
        $query = $this->conex_44->query($sql, $params); // #COPILOT_ASSISTED
        return $query->result(); // #COPILOT_ASSISTED
    }

    public function ejecutarConsultaConex47($sql, $params = array()) {
        $query = $this->conex_47->query($sql, $params); // #COPILOT_ASSISTED
        return $query->result(); // #COPILOT_ASSISTED
    }
}
?>
```

Figura 18. Modelo de Parámetro de BD

Fuente: Elaboración Propia

```php
class ReporteModel extends CI_Model {

    public function __construct() {
        parent::__construct();
        $this->load->database();
        $this->conex_44 = $this->load->database('bd', TRUE); // #COPILOT_ASSISTED
        $this->fecha = date("Y-m-d H:i:s"); // #COPILOT_ASSISTED
    }

    public function exportGestion($fechaInicio, $fechaFin) {
        $sql = "EXEC procedimiento ?, ?"; // #COPILOT_ASSISTED
        $query = $this->conex_44->query($sql, array($fechaInicio, $fechaFin)); // #COPILOT_ASSISTED
        return $query->result_array(); // #COPILOT_ASSISTED
    }
}
?>
```

Figura 19. Modelo de reportes Crosseling Front

Fuente: Elaboración Propia

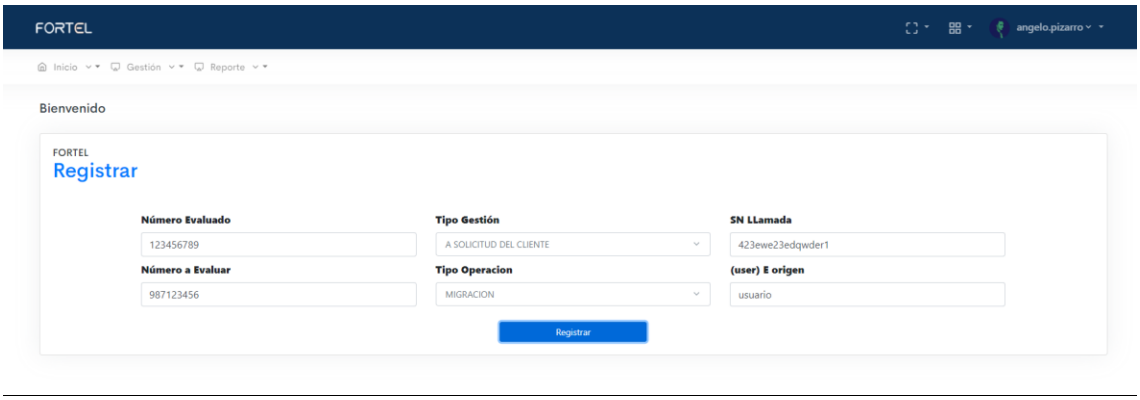


Figura 20. Registro de usuarios Crosseling front

Fuente: Elaboración Propia

Resultado del Sprint:

Se implementaron funcionalidades clave como la exportación de reportes, auditoría de cambios y un control de acceso robusto basado en roles. Esto fortaleció la seguridad y la trazabilidad en el sistema, asegurando un backend confiable y eficiente.

Sprint 4: Implementación del Gestor Consultas y Reclamos

Durante este sprint, se utilizó el prompt estándar descrito en el Anexo 6, lo que permitió a GitHub Copilot asistir en la generación de controladores y modelos en CodeIgniter, asegurando rapidez y precisión en el backend.

Además, los prompts y solicitudes realizadas en GitHub Copilot para este desarrollo se pueden visualizar en el Anexo 5.

Historias de Usuario Implementadas:

- **HU15: Registrar Reclamo**

  Tareas:

  1. Implementar el método en el controlador para gestionar la creación de nuevos reclamos.
  2. Crear funciones en el modelo para insertar registros de reclamos en la base de datos.
  3. Validar los datos ingresados y generar pruebas unitarias para garantizar la funcionalidad.

- **HU16: Actualizar Reclamo**

  Tareas:

  1. Implementar el método en el controlador para manejar actualizaciones de reclamos existentes.
  2. Desarrollar la lógica en el modelo para actualizar la información en la base de datos.

3. Generar pruebas unitarias para validar la correcta ejecución de las actualizaciones.

- **HU17: Consultar Reclamos**

  Tareas:

  1. Crear el método en el controlador para manejar solicitudes de consulta de reclamos por estado y fecha.
  2. Diseñar funciones en el modelo para realizar las consultas con los filtros requeridos.
  3. Realizar pruebas unitarias para verificar las respuestas y el correcto funcionamiento de las consultas.

- **HU18: Asignar responsable a reclamos**

  Tareas:

  1. Implementar un método en el controlador para asignar responsables a reclamos específicos.
  2. Crear funciones en el modelo para realizar actualizaciones en las tablas correspondientes.
  3. Validar la asignación mediante pruebas unitarias.

- **HU19: Exportar Datos de Usuarios Cesados**

  Tareas:

  1. Crear un método en el controlador para exportar datos de usuarios cesados en formato CSV.

2. Implementar lógica en el modelo para filtrar los datos necesarios para la exportación.

3. Generar pruebas unitarias para validar la integridad y consistencia de los datos exportados.

```php
class gestionUsuCesadoController extends CI_Controller // #COPILOT_ASSISTED
{
    public function __construct() // #COPILOT_ASSISTED
    {
        parent::__construct(); // #COPILOT_ASSISTED
        $this->load->model('GestionModel', 'gestion'); // #COPILOT_ASSISTED
        $this->load->helper('phpass_helper'); // #COPILOT_ASSISTED
        $this->load->library('session'); // #COPILOT_ASSISTED
    }

    public function RegistrarSolicitud() // #COPILOT_ASSISTED
    {
        $datos = array( // #COPILOT_ASSISTED
            'fecha_reclamo' => (new DateTime())->format('Y-m-d H:i:s'), // #COPILOT_ASSISTED
            'usuario_solicita' => 'USUARIO CESADO', // #COPILOT_ASSISTED
            'nomb_ape_solicita' => $this->input->post('nomb_ape_solicita'), // #COPILOT_ASSISTED
            'dni_solicita' => $this->input->post('dni_solicita'), // #COPILOT_ASSISTED
            'ceco_solicita' => $this->input->post('ceco_solicita'), // #COPILOT_ASSISTED
            'lista_reclamo' => $this->input->post('lista_reclamo'), // #COPILOT_ASSISTED
            'planilla_reclamo' => $this->input->post('planilla_reclamo'), // #COPILOT_ASSISTED
            'detalle_reclamo' => $this->input->post('detalle_reclamo'), // #COPILOT_ASSISTED
            'estado' => 1 // #COPILOT_ASSISTED
        ); // #COPILOT_ASSISTED

        $dataidges = $this->gestion->RegistrarSolicitud($datos); // #COPILOT_ASSISTED

        $datosbitacora = array( // #COPILOT_ASSISTED
            'ges_id' => $dataidges, // #COPILOT_ASSISTED
            'fecha_reclamo' => (new DateTime())->format('Y-m-d H:i:s'), // #COPILOT_ASSISTED
            'usuario_solicita' => 'USUARIO CESADO', // #COPILOT_ASSISTED
            'nomb_ape_solicita' => $this->input->post('nomb_ape_solicita'), // #COPILOT_ASSISTED
            'dni_solicita' => $this->input->post('dni_solicita'), // #COPILOT_ASSISTED
            'ceco_solicita' => $this->input->post('ceco_solicita'), // #COPILOT_ASSISTED
            'lista_reclamo' => $this->input->post('lista_reclamo'), // #COPILOT_ASSISTED
            'planilla_reclamo' => $this->input->post('planilla_reclamo'), // #COPILOT_ASSISTED
            'detalle_reclamo' => $this->input->post('detalle_reclamo'), // #COPILOT_ASSISTED
            'estado' => 1 // #COPILOT_ASSISTED
        ); // #COPILOT_ASSISTED
```

Figura 21. Controlador Usuarios Cesados

Fuente: Elaboración Propia

```php
class ReporteController extends CI_Controller // #COPILOT_ASSISTED
{
    public function __construct() // #COPILOT_ASSISTED
    {
        parent::__construct(); // #COPILOT_ASSISTED
        $this->load->model('ReporteModel', 'reporte'); // #COPILOT_ASSISTED
    }

    public function ExportGestion() // #COPILOT_ASSISTED
    {
        $fechaInicio = $this->input->post('fechaInicio'); // #COPILOT_ASSISTED
        $fechaFin = $this->input->post('fechaFin'); // #COPILOT_ASSISTED

        $data = $this->reporte->ExportGestion($fechaInicio, $fechaFin); // #COPILOT_ASSISTED

        echo json_encode($data); // #COPILOT_ASSISTED
```

Figura 22. Controlador Reportes Gestión Reclamos

Fuente: Elaboración Propia

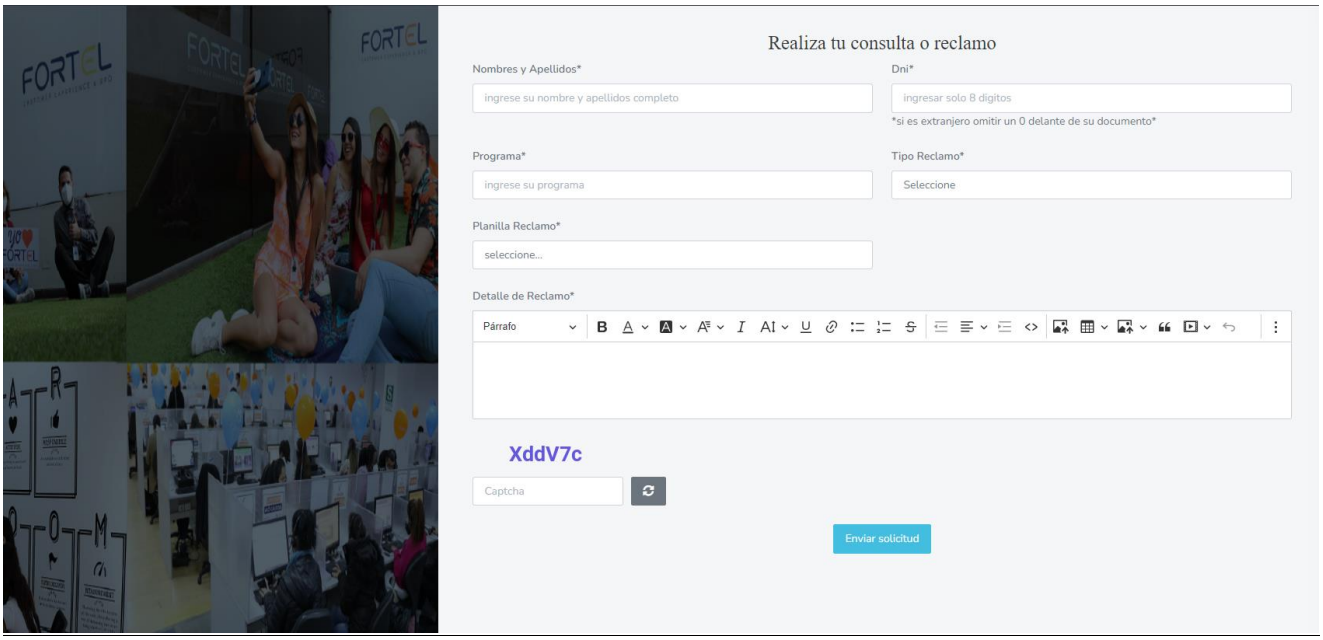


Figura 23. Usuarios Cesados Gestión Reclamos

Fuente: Elaboración Propia

Resultado del Sprint:

El sistema ahora permite gestionar reclamos, asignar responsables de manera eficiente y exportar datos de usuarios cesados. Las funcionalidades implementadas fueron verificadas mediante pruebas unitarias, asegurando la estabilidad del backend desarrollado.

Sprint 5: Finalización del Gestor Consultas y Reclamos

Como en los sprints anteriores, se utilizó el prompt estándar del Anexo 6 para generar el código backend de controladores y modelos en CodeIgniter con el apoyo de GitHub Copilot.

Historias de Usuario Implementadas:

- **HU20: Registrar Respuesta a Reclamo**

Tareas:

1. Implementar el método registrarRespuesta en el controlador para permitir a los responsables agregar respuestas a los reclamos.
2. Crear una función en el modelo para insertar las respuestas en la base de datos.
3. Validar la funcionalidad mediante pruebas unitarias.

- **HU21: Consultar Histórico de Reclamos**

Tareas:

1. Desarrollar el método en el controlador para manejar consultas de históricos de reclamos.
2. Implementar lógica en el modelo para extraer datos completos del historial.
3. Generar pruebas unitarias para asegurar que el historial se muestre correctamente.

- **HU22: Generar Reportes de Reclamos**

Tareas:

1. Crear un método en el controlador para generar reportes detallados de reclamos por fecha, responsable y estado.
2. Diseñar funciones en el modelo para procesar los datos requeridos para el reporte.
3. Validar la generación de reportes mediante pruebas unitarias.

- **HU23: Configurar Alertas de Reclamos Pendientes**

  Tareas:

  1. Implementar un método en el controlador para configurar alertas automáticas de reclamos pendientes.
  2. Añadir lógica en el modelo para detectar y notificar reclamos con plazos críticos.
  3. Generar pruebas unitarias para validar la correcta emisión de alertas.

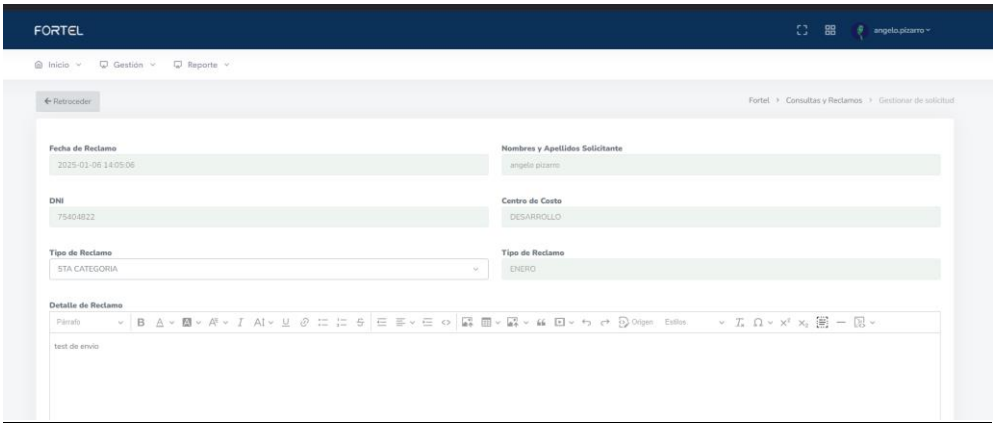


Figura 24 Registro de reclamos

Fuente: Elaboración Propia

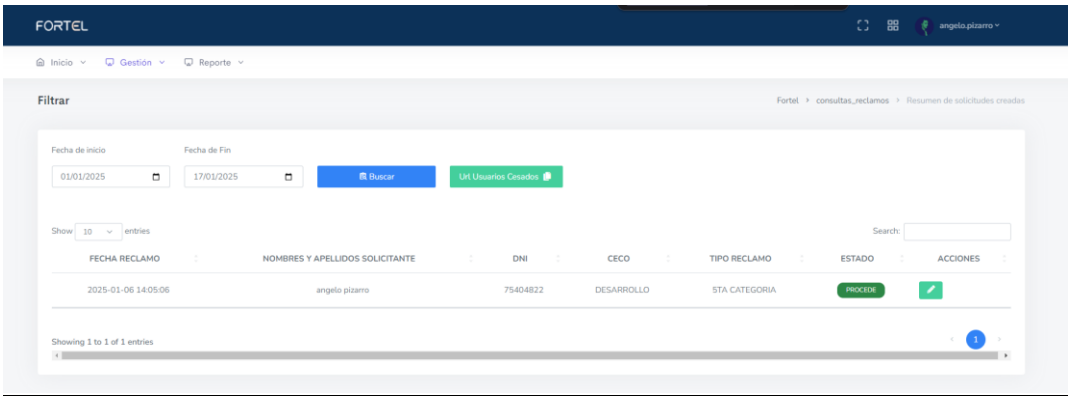


Figura 25. Histórico de reclamos

Fuente: Elaboración Propia

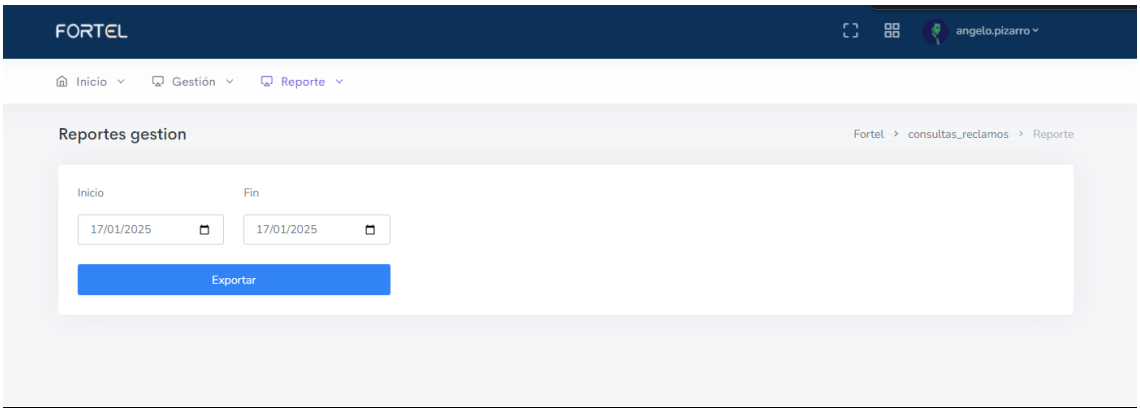


Figura 26. Reportes de reclamos

Fuente: Elaboración Propia

```php
    }

    public function testTraerUsuarioDatos() // #COPILOT_ASSISTED
    {
        // Define the behavior of the mock // #COPILOT_ASSISTED
        $this->gestionModelMock->shouldReceive('traerUsuarioDatos') // #COPILOT_ASSISTED
            ->once() // #COPILOT_ASSISTED
            ->andReturn(['codigo' => 'Ok', 'nombres' => 'John', 'apellidos' => 'Doe']); // #COPILOT_ASSISTED

        $output = $this->CI->gestion->traerUsuarioDatos(); // #COPILOT_ASSISTED
        $this->assertEquals(['codigo' => 'Ok', 'nombres' => 'John', 'apellidos' => 'Doe'], $output); // #COPILOT_ASSISTED
    }

    public function testTraerDatoSolicitud() // #COPILOT_ASSISTED
    {
        $id = 1; // #COPILOT_ASSISTED

        // Define the behavior of the mock // #COPILOT_ASSISTED
        $this->gestionModelMock->shouldReceive('traerDatoSolicitud') // #COPILOT_ASSISTED
            ->once() // #COPILOT_ASSISTED
            ->with($id) // #COPILOT_ASSISTED
            ->andReturn(['status' => 'success', 'data' => [['id' => 1, 'fecha_reclamo' => '2023-10-01']]]); // #COPILOT_ASSISTED

        $output = $this->CI->gestion->traerDatoSolicitud($id); // #COPILOT_ASSISTED
        $this->assertEquals(['status' => 'success', 'data' => [['id' => 1, 'fecha_reclamo' => '2023-10-01']]], $output); // #COPILOT_ASSISTED
    }

    public function testTraerListaReclamo() // #COPILOT_ASSISTED
    {
        // Define the behavior of the mock // #COPILOT_ASSISTED
        $this->gestionModelMock->shouldReceive('traerListaReclamo') // #COPILOT_ASSISTED
            ->once() // #COPILOT_ASSISTED
            ->andReturn([['Id' => 1, 'nombre' => 'Reclamo 1']]); // #COPILOT_ASSISTED

        $output = $this->CI->gestion->traerListaReclamo(); // #COPILOT_ASSISTED
        $this->assertEquals([['Id' => 1, 'nombre' => 'Reclamo 1']], $output); // #COPILOT_ASSISTED
    }

    public function testTraerListaRespGenReclamo() // #COPILOT_ASSISTED
    {
        // Define the behavior of the mock // #COPILOT_ASSISTED
        $this->gestionModelMock->shouldReceive('traerListaRespGenReclamo') // #COPILOT_ASSISTED
            ->once() // #COPILOT_ASSISTED
            ->andReturn([['id' => 1, 'nombre' => 'Responsable 1']]); // #COPILOT_ASSISTED

        $output = $this->CI->gestion->traerListaRespGenReclamo(); // #COPILOT_ASSISTED
        $this->assertEquals([['id' => 1, 'nombre' => 'Responsable 1']], $output); // #COPILOT_ASSISTED
    }
```

Figura 27. Pruebas Unitaria reclamos

Fuente: Elaboración Propia

Resultado del Sprint:

Se completaron las funcionalidades finales del Gestor Consultas y Reclamos, incluyendo la gestión de respuestas, consulta de históricos, generación de reportes detallados y configuración de alertas automáticas. Las pruebas unitarias aseguraron la correcta operación del backend desarrollado.

4.1.4. Fase de revisión y retrospectiva

En esta etapa, el equipo evaluó los entregables generados durante los sprints anteriores, asegurando que las funcionalidades desarrolladas con el apoyo de **GitHub Copilot** cumplieran con los requisitos definidos al inicio del proyecto.

Actividades Realizadas:

1. **Revisión del Código Generado por Copilot**:

   Se revisaron los controladores, modelos y las pruebas unitarias generadas por la herramienta para garantizar su alineación con las normas de calidad de código y los requerimientos específicos.

2. **Validación con el Prompt Estándar**:

   Se verificó que las solicitudes realizadas a Copilot utilizaran el prompt estándar descrito en el Anexo 6, asegurando consistencia en los resultados generados.

3. **Revisión de Métricas**:

   Se analizaron las métricas de calidad del código y el rendimiento del sistema utilizando herramientas como SonarQube, para medir la cantidad de bugs.

Retrospectiva:

Se llevó a cabo una reunión con el equipo para identificar aciertos y áreas de mejora en el uso de GitHub Copilot, así como en los procesos de desarrollo.

Tabla 22

Retrospectiva del desarrollo con Copilot

Aspecto Evaluado	Aciertos Identificados	Áreas de Mejora	Acción Por Tomar
Eficiencia en Generación de Código	Reducción significativa del tiempo de escritura de controladores y modelos.	Ajustar el prompt estándar para incluir más detalles específicos y minimizar correcciones manuales.	Actualizar el prompt estándar detallado en el Anexo 6 con ejemplos más específicos.

Cobertura de Pruebas Unitarias	Generación automática de pruebas unitarias que cubrieron la mayoría de los casos necesarios.	Incrementar la validación manual para casos menos comunes no detectados por Copilot.	Diseñar un plan de revisión para pruebas generadas automáticamente en cada sprint.
Calidad del Código Generado	Código generado alineado con los estándares de diseño predefinidos para el proyecto.	Revisar posibles redundancias en el código sugerido por Copilot.	Implementar revisiones de código específicas antes de la integración final.
Colaboración en el Equipo	Copilot facilitó una mejor distribución de tareas al automatizar partes repetitivas del desarrollo.	Fortalecer la comunicación para resolver dudas sobre sugerencias de Copilot en tiempo real.	Planificar reuniones más frecuentes para discutir soluciones sugeridas por Copilot.

Fuente. Elaboración Propia

Resultados:

- Todas las funcionalidades desarrolladas fueron validadas y cumplían con los requerimientos establecidos.
- Se identificaron mejoras para optimizar los prompts empleados en futuros proyectos, maximizando la eficiencia de Copilot.

4.1.5. Fase de Lanzamiento

En esta fase final, se realizó la puesta en marcha de los sistemas replicados con GitHub Copilot. La implementación se enfocó en garantizar la equivalencia funcional de los sistemas y validar el impacto del desarrollo asistido por inteligencia artificial.

Tabla 23

Retrospectiva del desarrollo con Copilot

Actividad	Descripción	Responsable	Resultado

Migración a Producción	Implementación de los sistemas replicados en el entorno de producción, asegurando la funcionalidad equivalente a los sistemas originales.	Equipo de Desarrollo	Los sistemas fueron desplegados exitosamente en producción sin interrupciones.
Pruebas de Aceptación	Validación de las funcionalidades replicadas mediante pruebas en un entorno de producción controlado.	Equipo QA y Usuarios Finales	Las pruebas confirmaron que los sistemas replicados funcionan correctamente y cumplen con los requisitos originales.
Capacitación del Equipo	Revisión y análisis con el equipo sobre los sistemas replicados. No se realizaron capacitaciones adicionales debido a que las funcionalidades permanecieron sin cambios.	Equipo de Capacitación	No se identificó necesidad de capacitación adicional, ya que los sistemas replicados mantuvieron las mismas características.
Documentación Técnica y Comparativa	La documentación base no presentó cambios significativos.	Equipo de Documentación	Se validó que la documentación técnica original no requería modificaciones.
Monitoreo Inicial	Supervisión del desempeño de los sistemas en producción para confirmar la estabilidad y equivalencia funcional.	Equipo de Soporte Técnico	No se necesitaron ajustes.

Fuente. Elaboración Propia

4.2 Medición de la Solución

4.2.1. Análisis Descriptivo

Seguidamente, se presentan los análisis correspondientes a los datos obtenidos durante las etapas de PRE-TEST y POS-TEST, los cuales fueron realizados para medir el impacto del uso de GitHub Copilot en los procesos de desarrollo de software.

- PRE-TEST: Se llevó a cabo entre el periodo 10/05/2024 y el 31/10/2024, en el que se evaluaron proyectos desarrollados mediante métodos tradicionales.
- Pruebas: Desde el 09/12/2024 hasta el 06/01/2025, se desarrollaron sesiones formativas para el uso de GitHub Copilot, donde se trabajó en la escritura asistida de código, la integración en proyectos y la automatización de tareas repetitivas.
- POS-TEST: Realizado entre el 02/01/2025 y el 10/01/2025, con una duración de 8 días, enfocándose en proyectos desarrollados con el apoyo de GitHub Copilot.

El análisis descriptivo examina las principales métricas comparativas:

1. Duración del desarrollo: Se analizó el tiempo promedio empleado en cada tarea antes y después de implementar GitHub Copilot.
2. Calidad del código: Se evaluó el porcentaje de líneas de código libres de errores críticos utilizando la herramienta SonarQube.
3. Eficiencia en detección de errores: Se revisó el número promedio de bugs detectados y corregidos por proyecto.
4. Efectividad en pruebas unitarias: Se midió la proporción de pruebas exitosas en relación con el total de pruebas realizadas.

4.2.1.1. Análisis de Pre-Test.

Proyecto 1- API Crosseling Front

En primer lugar, se analizará la situación general de la aplicación que están usando actualmente para elaborar los informes y la percepción que genera la gestión del CRUD de la API.

A continuación, se mostrará los resultados del proyecto 1 obtenidos con la metodología tradicional.

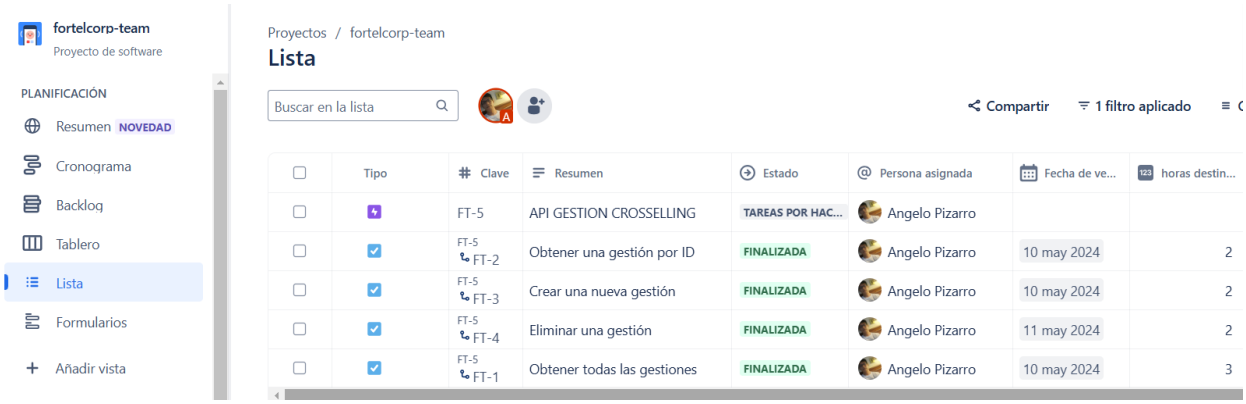


Figura 28. Cronograma Jira Proyecto 1 con Metodología Tradicional

Fuente: Fortelcorp-Team

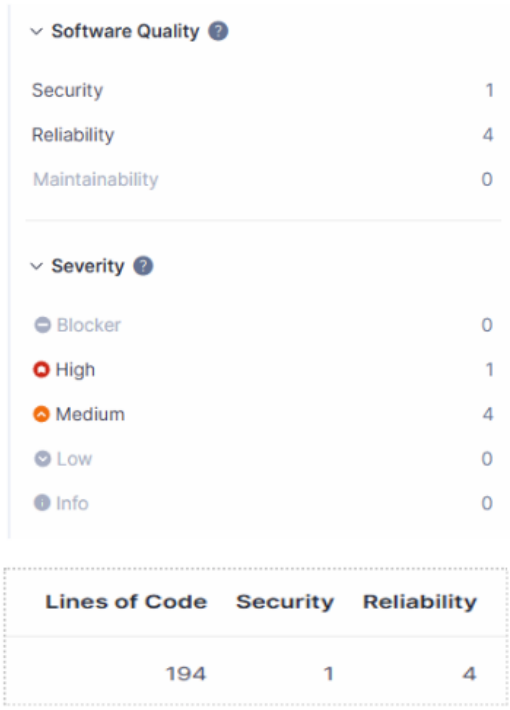


Figura 29. Resultado de análisis de SonarQube Proyecto 1 con Metodología Tradicional

Fuente: SonarQube

Para el resultado pre-test de Calidad de Código se realizará con la siguiente formula en la figura 30, con los datos obtenidos de SonarQube que se visualiza en la figura 29 lo cual nos dará como resultado el porcentaje de calidad de código que se realizó con la metodología tradicional.

Fórmula General:

$$\text{Calidad (\%)} = 100 - \left[\frac{(5 \times \text{Blocker}) + (3 \times \text{High}) + (2 \times \text{Medium}) + (1 \times \text{Low})}{\text{Líneas de Código Totales}} \times 100\right]$$

Figura 30. Formula General para Calcular la Calidad de Código

Fuente: SonarQube

A continuación, se realizará el desarrollo de la fórmula para evaluar la calidad del código basada en los datos proporcionados por SonarQube:

Valores:

- Blocker = 0
- High = 1
- Medium = 4
- Low = 0
- Líneas de Código Totales = 194

Sustituyendo valores:

$$Calidad(\%) = 100 - [\frac{(5x0) + (3x1) + (2x4) + (1x0)}{194} x100\ ]$$

$$Calidad(\%) = 100 - [\frac{0 + 3 + 8 + 0}{194} x100\ ]$$

$$Calidad(\%) = 100 - [\frac{11}{194} x100\ ]$$

$$Calidad(\%) = 100 - [5.67]$$

$$Calidad(\%) = 94.33$$

$$Calidad = 94.33\%$$

De acuerdo con el cálculo realizado, se determina que el porcentaje de calidad del código es del 94.33%.



Figura 31. Resultado de Bugs de SonarQube Proyecto 1 con Metodología Tradicional

Fuente: SonarQube

En base a los resultados arrojado por el software de SonarQube la detección de bugs detectados en el proyecto 1 es de O, esto se visualiza en la figura 31 en la opción de Blocker.

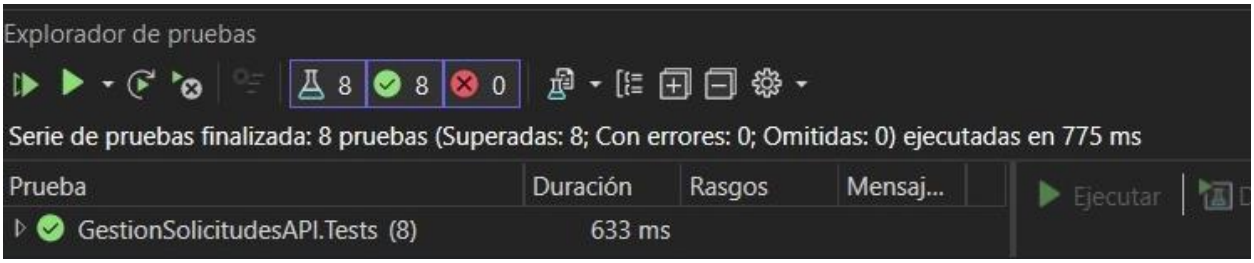


Figura 32. Resultado de Pruebas Unitarias del Proyecto 1 con Metodología Tradicional

Fuente: Visual Studio

Tabla 24

Resultados del pre-test Metodología Tradicional Proyecto 1

Dimensiones	Instrumento	Resultados
Tiempo de Desarrollo	Jira	9 horas realizadas
Calidad del Código	SonarQube	94.33%% Libre de errores
detección de bugs	SonarQube	0 bug
Efectividad en Pruebas Unitarias	Herramienta de pruebas unitarias (Visual Studio)	100% Efectividad

Fuente. Elaboración propia

En el pre-test realizado utilizando la Metodología Tradicional, se documentaron un total de 9 horas de trabajo en Jira. El análisis de calidad del código realizado mediante SonarQube reflejó una cobertura del 94.33%% sin errores, acompañado de una tasa de detección de bugs del 0%. Adicionalmente, se llevaron a cabo 8 pruebas unitarias mediante la herramienta Visual Studio, de las cuales las 8 se completaron con éxito, alcanzando un 100% de efectividad. Estos resultados muestran el desempeño inicial del equipo y la calidad del proyecto en esta etapa.

Proyecto 2 - Crosseling Front

A continuación, se mostrará los resultados del proyecto 2 obtenidos con la metodología tradicional.

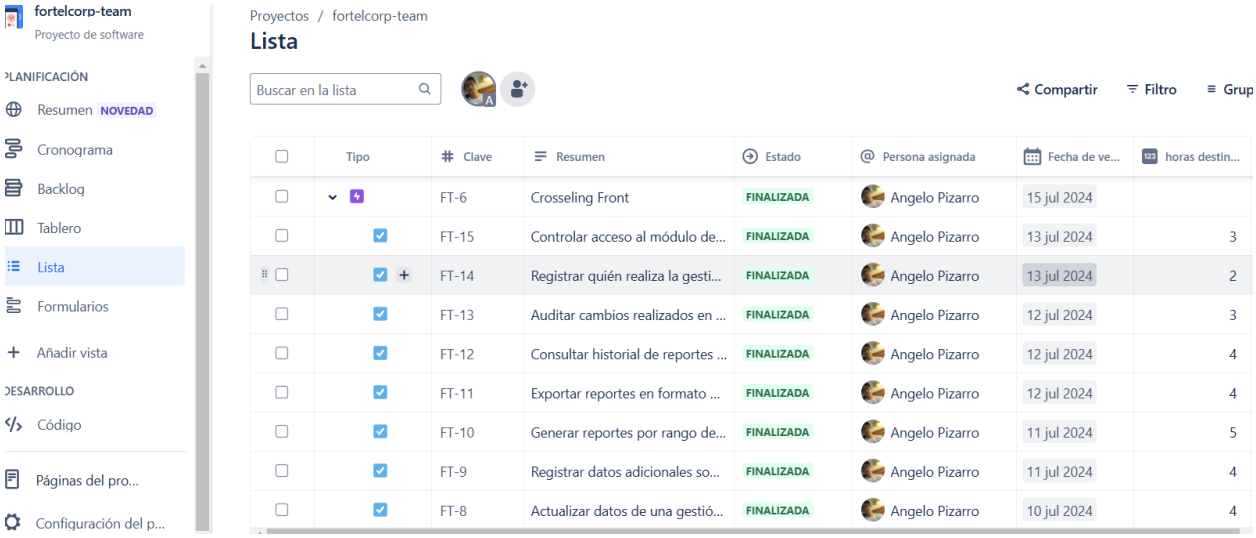


Figura 33. Cronograma Jira Proyecto 2 con Metodología Tradicional

Fuente: Fortelcorp-Team

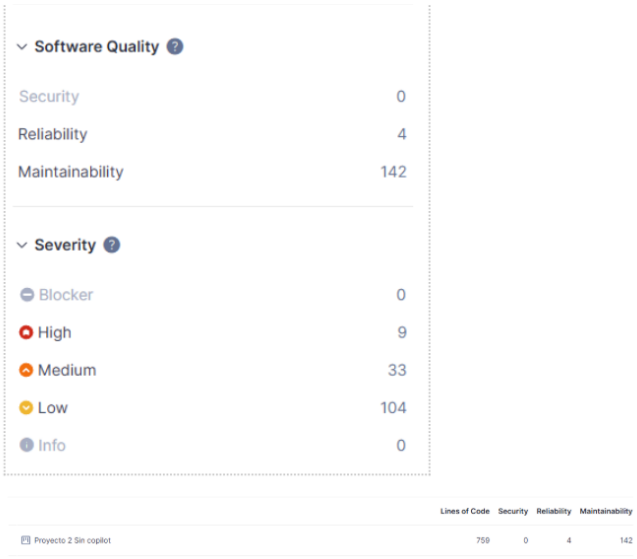


Figura 34. Resultado de análisis de SonarQube Proyecto 2 con Metodología Tradicional

Fuente: SonarQube

Para el resultado pre-test de Calidad de Código se realizará con la siguiente formula en la figura 30, con los datos obtenidos de SonarQube que se visualiza en la figura 34 lo cual nos dará como resultado el porcentaje de calidad de código que se realizó con la metodología tradicional.

A continuación, se realizará el desarrollo de la fórmula para evaluar la calidad del código basada en los datos proporcionados por SonarQube:

Valores:

- Blocker = 0
- High = 9
- Medium = 33
- Low = 104
- Líneas de Código Totales = 759

Sustituyendo valores:

$$Calidad(\%) = 100 - [\frac{(5x0) + (3x9) + (2x33) + (1x104)}{759} x100\ ]$$

$$Calidad(\%) = 100 - [\frac{0 + 27 + 66 + 104}{759} x100\ ]$$

$$Calidad(\%) = 100 - [\frac{197}{759} x100\ ]$$

$$Calidad = 74.04\%$$

De acuerdo con el cálculo realizado, se determina que el porcentaje de calidad del código es del 74.04%.



Figura 35. Resultado de Bugs de SonarQube Proyecto 2 con Metodología Tradicional

Fuente: SonarQube

En base a los resultados arrojado por el software de SonarQube la detección de bugs detectados en el proyecto 2 es de O, esto se visualiza en la figura 35 en la opción de Blocker.

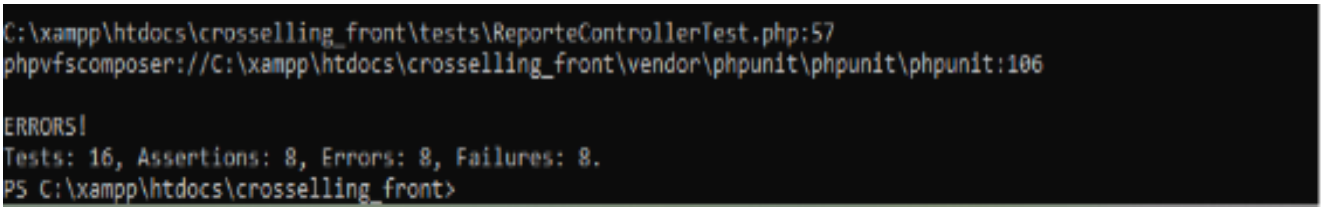


Figura 36. Resultado de Pruebas Unitarias del Proyecto 2 con Metodología Tradicional

Fuente: PHPUNIT

Tabla 25

Resultados del pre-test Metodología Tradicional Proyecto 2

Dimensiones	Instrumento	Resultados
Tiempo de Desarrollo	Jira	32 horas realizadas
Calidad del Código	SonarQube	74.04% Libre de errores
detección de bugs	SonarQube	0 bug
Efectividad en Pruebas Unitarias	Herramienta de pruebas unitarias (Visual Studio)	50% Efectividad

Fuente: Elaboración propia

En el pre-test del Proyecto 2, se evaluaron las dimensiones clave del desarrollo de software utilizando la metodología tradicional. El tiempo de desarrollo alcanzó las 32 horas, lo que refleja un proceso extenso que podría optimizarse. En cuanto a la calidad del código, el análisis con SonarQube mostró que solo el 74.04% de las líneas estaban libres de errores, indicando una oportunidad significativa de mejora. La detección de bugs no reportó errores identificados, aunque esto no garantiza que el código esté completamente optimizado. Finalmente, la efectividad en pruebas unitarias fue del 50%, dado que, de las 16 pruebas realizadas, 8 fueron exitosas y 8 presentaron errores, lo que evidencia limitaciones tanto en la cobertura como en la validación del código generado. Estos resultados reflejan las áreas de oportunidad en el proceso tradicional que podrían beneficiarse de herramientas como GitHub Copilot.

Proyecto 3 - Gestión Consultas y Reclamos

A continuación, se mostrará los resultados del proyecto 3 obtenidos con la metodología tradicional.

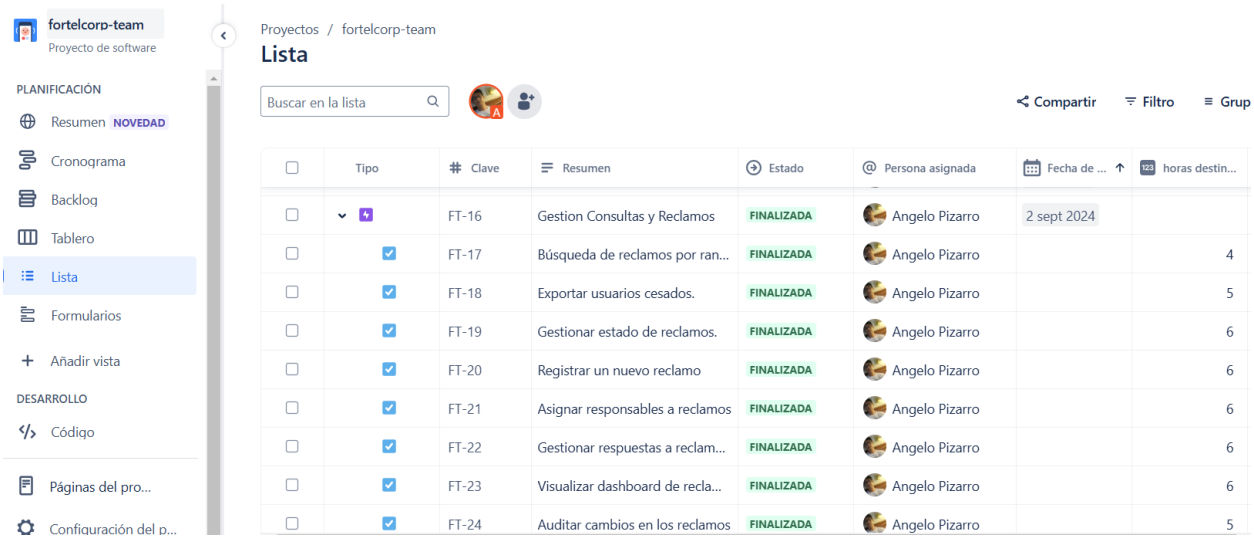


Figura 37. Cronograma Jira Proyecto 3 con Metodología Tradicional

Fuente: Fortelcorp-Team



Figura 38. Resultado de análisis de SonarQube Proyecto 3 con Metodología Tradicional

Fuente: SonarQube

Para el resultado pre-test de Calidad de Código se realizará con la siguiente formula en la figura 30, con los datos obtenidos de SonarQube que se visualiza en la figura 38 lo cual nos dará como resultado el porcentaje de calidad de código que se realizó con la metodología tradicional.

A continuación, se realizará el desarrollo de la fórmula para evaluar la calidad del código basada en los datos proporcionados por SonarQube:

Valores:

- Blocker = 0
- High = 10
- Medium = 37
- Low = 51
- Líneas de Código Totales = 946

Sustituyendo valores:

$$Calidad(\%) = 100 - [\frac{(5x0) + (3x10) + (2x37) + (1x51)}{946} x100\ ]$$

$$Calidad(\%) = 100 - [\frac{0 + 30 + 74 + 51}{946} x100\ ]$$

$$Calidad(\%) = 100 - [\frac{155}{946} x100\ ]$$

$$Calidad = 83.62\%$$

De acuerdo con el cálculo realizado, se determina que el porcentaje de calidad del código es del 93.62%.



Figura 39. Resulta*do de Bugs de SonarQube Proyecto 3 con Metodología Tradicional*

Fuente: SonarQube

En base a los resultados arrojado por el software de SonarQube la detección de bugs detectados en el proyecto 3 es de O, esto se visualiza en la figura 39 en la opción de Blocker.

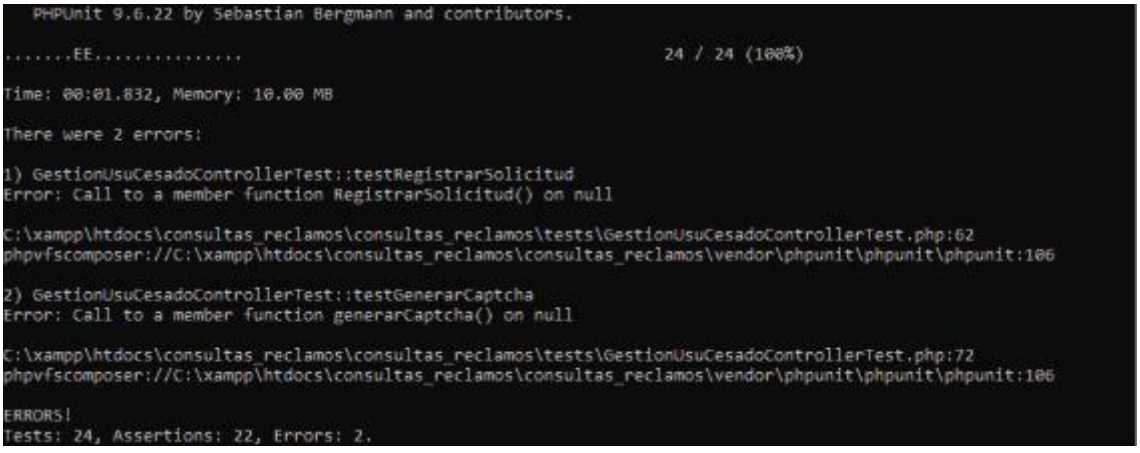


Figura 40. Resultado de Pruebas Unitarias del Proyecto 3 con Metodología Tradicional

Fuente: PHPUNIT

Realizamos la siguiente Formula para calcular la efectividad de pruebas unitarias:

$$\text{Porcentaje de éxito} = \left(\frac{\text{Pruebas exitosas}}{\text{Total de pruebas}}\right) \times 100$$

Figura 41. Fórmula para Calcular el Nivel de Efectividad de Pruebas Unitarias

Fuente: Elaboración Propia

$$Porcentaje\ de\ éxito = \frac{22}{24} x100$$

$$Porcentaje\ de\ éxito = 91.67\%$$

Tabla 26

Resultados del pre-test Metodología Tradicional Proyecto 3

Dimensiones	Instrumento	Resultados
Tiempo de Desarrollo	Jira	48 horas realizadas
Calidad del Código	SonarQube	83.62% Libre de errores
detección de bugs	SonarQube	0 bug
Efectividad en Pruebas Unitarias	Herramienta de pruebas unitarias (Visual Studio)	91.67% Efectividad

Fuente. Elaboración propia

En el pre-test del Proyecto 3, se evaluaron las dimensiones clave del desarrollo de software utilizando la metodología tradicional. El tiempo de desarrollo alcanzó las 48 horas, lo que refleja un proceso más extenso en comparación con proyectos anteriores. En cuanto a la calidad del código,

el análisis realizado con SonarQube indicó que el 83.62% de las líneas estaban libres de errores, lo cual representa un nivel aceptable de calidad. Además, no se detectaron bugs en el análisis realizado. Finalmente, en las pruebas unitarias, se ejecutaron 24 pruebas, de las cuales 22 fueron exitosas y 2 presentaron errores, logrando así una efectividad del 91.67%, lo que demuestra un alto nivel de confiabilidad en los casos de prueba ejecutados.

4.2.1.2. Análisis de Pos-Test.

Proyecto 1- API Crosseling Front

A continuación, se mostrará los resultados del proyecto 1 obtenidos con el uso de GitHub Copilot.



Figura 42. Cronograma Jira Proyecto 1 con Uso de GitHub Copilot

Fuente: Elaboración Propia

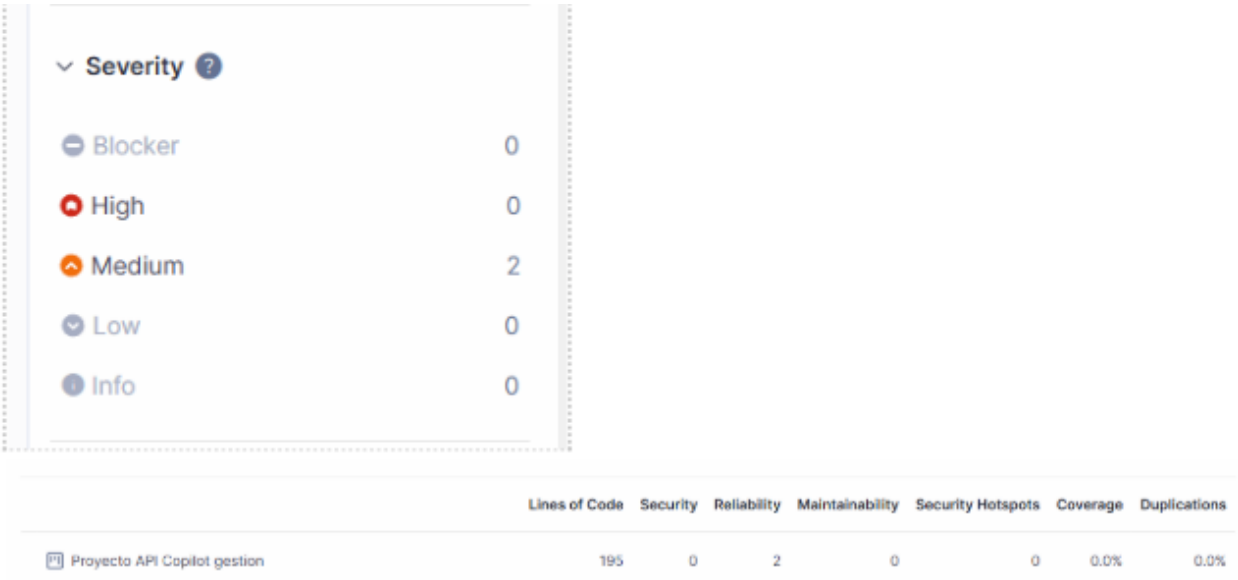


Figura 43. Cronograma Jira Proyecto 1 con Uso de GitHub Copilot

Fuente: SonarQube

Para el resultado pre-test de Calidad de Código se realizará con la formula en la figura 30, con los datos obtenidos de SonarQube que se visualiza en la figura 43 lo cual nos dará como resultado el porcentaje de calidad de código que se realizó con GitHub Copilot.

A continuación, se realizará el desarrollo de la fórmula para evaluar la calidad del código basada en los datos proporcionados por SonarQube:

Valores:

- Blocker = 0
- High = 0
- Medium = 2
- Low = 0
- Líneas de Código Totales = 195

Sustituyendo valores:

$$Calidad(\%) = 100 - [\frac{(5x0) + (3x0) + (2x2) + (1x0)}{195} x100\ ]$$

$$Calidad(\%) = 100 - [\frac{0+0+4+0}{195} x100\ ]$$

$$Calidad(\%) = 100 - [\frac{4}{195} x100\ ]$$

$$Calidad(\%) = 100 - [2.051]$$

$$Calidad(\%) = 97.95$$

$$Calidad = 97.95\%$$

De acuerdo con el cálculo realizado, se determina que el porcentaje de calidad del código es del 97.95%.

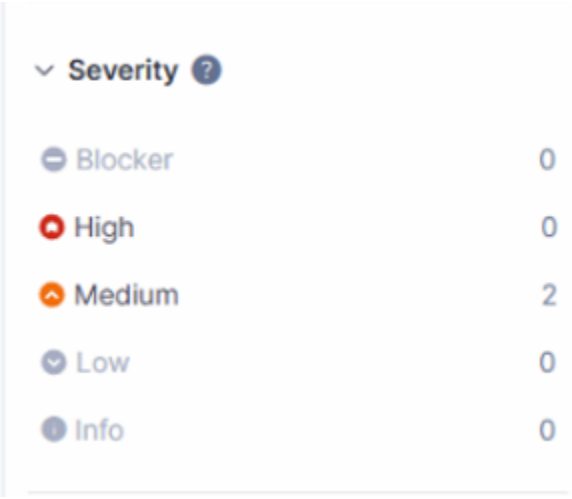


Figura 44. Resultado de Bugs de SonarQube Proyecto 1 con GitHub Copilot

Fuente: SonarQube

En base a los resultados arrojado por el software de SonarQube la detección de bugs detectados en el proyecto 1 es de O, esto se visualiza en la figura 44 en la opción de Blocker.

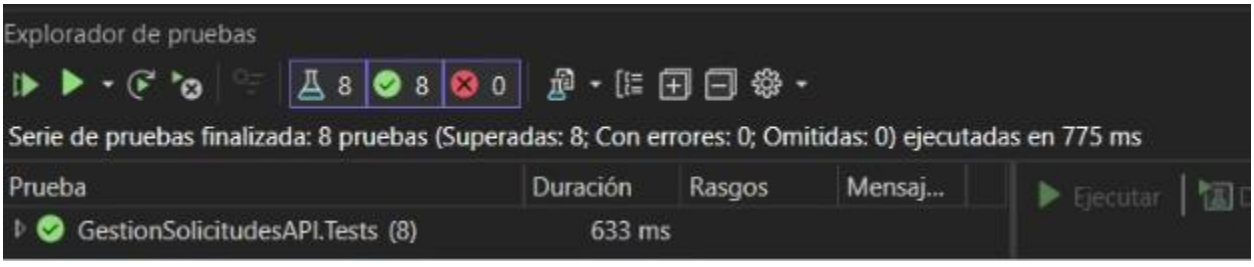


Figura 45. Resultado de Pruebas Unitarias del Proyecto 1 con GitHub Copilot

Fuente: Visual Studio

Tabla 27

Resultados del pos-test Proyecto 1

Instrumento	Data
Jira	4 horas realizadas
SonarQube	97.95% Libre de errores
SonarQube	0 bug
Herramienta de pruebas unitarias (Visual Studio)	100% efectividad

Fuente. Elaboración propia

En el pos-test, se obtuvieron los siguientes resultados: con Jira se registraron un total de 4 horas de trabajo, reflejando una mayor eficiencia en comparación con etapas previas. El análisis de calidad del código realizado mediante SonarQube mostró una cobertura del 97.95% libre de errores en calidad de Código, indicando una notable mejora en la calidad del desarrollo, junto con una tasa de detección de bugs del 0%. Asimismo, se ejecutaron 8 pruebas unitarias utilizando Visual Studio, todas exitosas, lo que permitió alcanzar un 100% de efectividad. Estos resultados destacan un avance significativo en la productividad y la calidad del equipo durante esta etapa del proyecto usando la herramienta de IA GitHub Copilot.

Variable Dependiente.

A continuación, se desarrollará la operalización de la variable dependiente, esto se ve reflejado en la figura 46.

Dimensiones	Indicadores	Formula
Tiempo de Desarrollo	Tiempo Total en horas/días dedicado a completar cada proyecto	(HT- HC) * (HT/100)
Calidad del Código	porcentaje de código libre de errores	CC-CT
detección de bugs	Número de bugs detectados y corregidos en el Código de desarrollo	(BT/100) * BC*100
Efectividad en Pruebas Unitarias	Porcentaje de pruebas exitosas realizadas frente al total de pruebas ejecutadas	CE/CP*100

Hora tradicional	HT	Porcentaje Calidad Tradicional	CT	Bugs Tradicional	BT	Cantidad de pruebas	CP
Hora Copilot	HC	Porcentaje Calidad Copilot	CC	Bugs Copilot	BC	Cantidad de pruebas exitosas con Copilot	CE

Figura 46. Fórmulas para la operación de la variable dependiente

Fuente: Elaboración Propia

Tiempo de Desarrollo: (9-4) * (9/100) = 55.6%

Calidad del Código: 97.95 – 94.33= 3.62%

Detección de bugs: (0)

Efectividad en Pruebas Unitarias: 8/8*100 = 100%

Tabla 28

Distribución de Frecuencias de la Variable Dependiente en el Pos-Test Proyecto 1.

Dimensiones	Instrumento	Nivel	Resultado
Tiempo de Desarrollo	Jira	Bueno	55.6% Optimización en tiempo de desarrollo
Calidad del Código	SonarQube	Regular	3.62% es la diferencia de calidad de Código entre met. tradicional y con GitHub Copilot

Detección de bugs	SonarQube	Bueno	0% de errores cometidos
Efectividad en Pruebas Unitarias	Herramienta de pruebas unitarias (Visual Studio)	Bueno	100% se ejecutaron 8 pruebas y las 8 fueron exitosas

Fuente. Elaboración propia

En el pos-test de operacionalización de la variable dependiente, se obtuvieron resultados positivos en diferentes dimensiones del desarrollo de software. El Tiempo de Desarrollo, evaluado mediante Jira, alcanzó un nivel "Bueno" con una optimización del 55.6%, lo que refleja una mejora significativa en la eficiencia del equipo. La Calidad del Código, medida a través de SonarQube, fue clasificada como "Regular" con una diferencia del 7.76% entre el método tradicional y el uso de GitHub Copilot, mostrando una ligera mejora. En cuanto a la Detección de Bugs, también evaluada con SonarQube, logró un nivel "Bueno" con un 0% de errores cometidos, destacando la calidad del desarrollo. Por último, la Efectividad en Pruebas Unitarias, analizada con Visual Studio, fue clasificada como "Bueno" al ejecutarse 8 pruebas exitosas de un total de 8, alcanzando así un 100% de efectividad. Estos resultados reflejan el impacto positivo del uso de herramientas asistidas en el proceso de desarrollo.

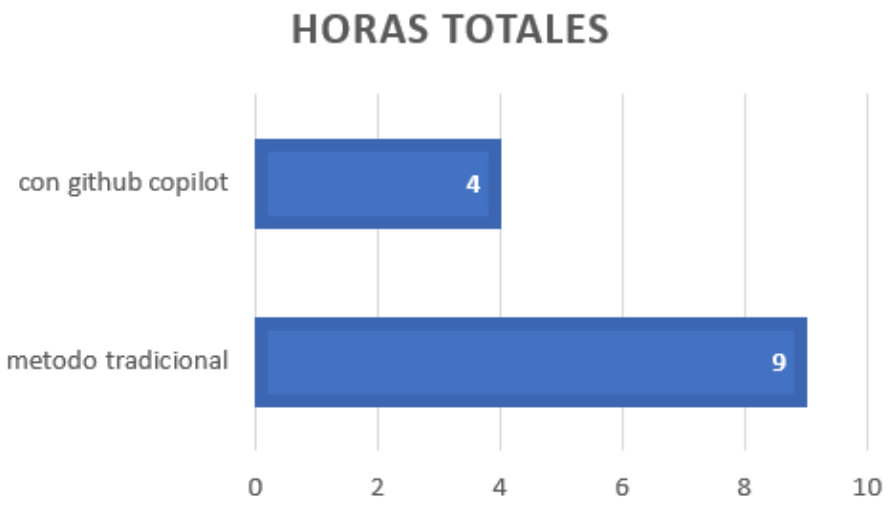


Figura 47. Gráfico comparativo horas totales

Fuente: Elaboración Propia

En la Figura 47, se presenta un gráfico comparativo de las horas totales dedicadas al desarrollo entre el método tradicional y el uso de GitHub Copilot. Los resultados muestran que, con el método tradicional, se invirtieron 9 horas, mientras que con el apoyo de GitHub Copilot, el tiempo se redujo a 4 horas, logrando así una mejora significativa en la eficiencia del desarrollo. Este análisis no solo evidencia una reducción del tiempo empleado, sino que también resalta cómo el uso de herramientas de inteligencia artificial, como GitHub Copilot, puede optimizar los procesos, permitiendo al equipo dedicar menos tiempo al desarrollo y concentrarse en otras tareas críticas del proyecto.

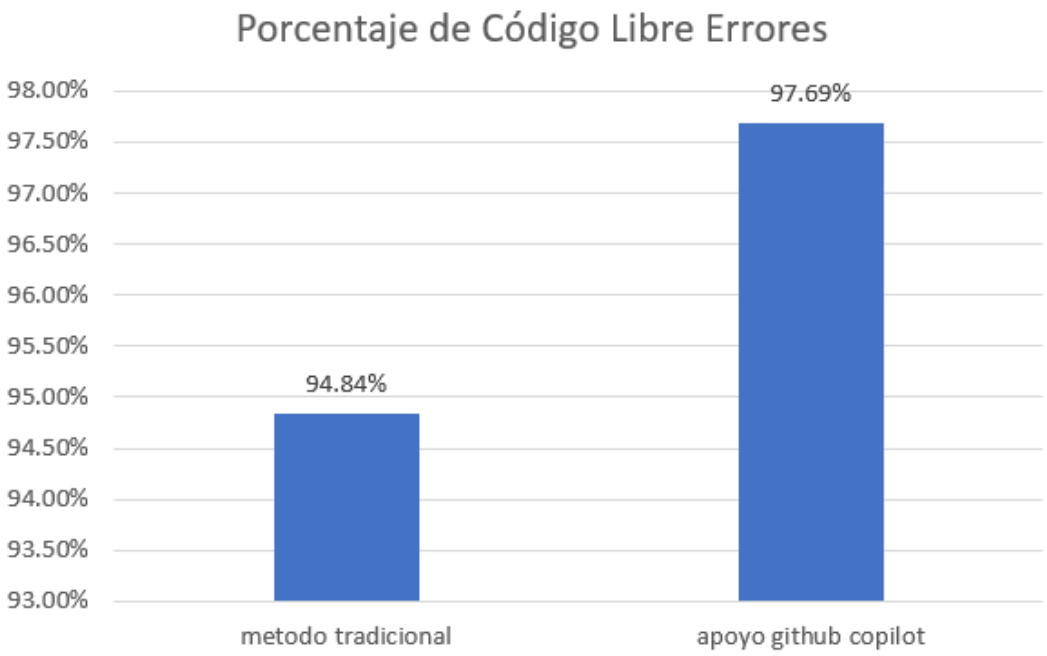


Figura 48. Gráfico comparativo de código libre de errores

Fuente: Elaboración Propia

La Figura 48 muestra un gráfico comparativo del porcentaje de código libre de errores entre el método tradicional y el apoyo de GitHub Copilot. Los resultados evidencian que el método tradicional logró un 94.84% de código libre de errores, mientras que el uso de GitHub Copilot alcanzó un 97.95%, reflejando una mejora significativa en la calidad del código generado. Este incremento del 2.85% en la calidad del código con GitHub Copilot pone de manifiesto el impacto positivo de las herramientas basadas en inteligencia artificial para reducir errores y optimizar el proceso de desarrollo. Estos resultados resaltan cómo GitHub Copilot contribuye a mejorar la precisión y confiabilidad del código entregado.

Proyecto 2 - Crosseling Front

A continuación, se mostrará los resultados del proyecto 2 obtenidos con el uso de GitHub Copilot.

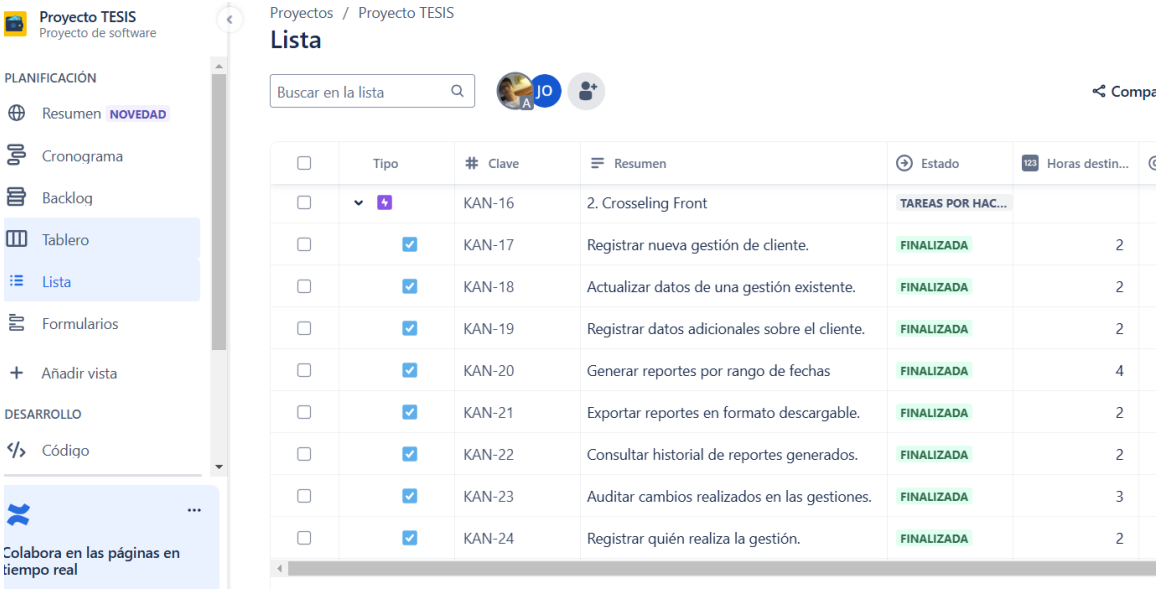


Figura 49. Cronograma Jira Proyecto 2 con Uso de GitHub Copilot

Fuente: Elaboración Propia



Figura 50. SonarQube Proyecto 2 con Uso de GitHub Copilot

Fuente: SonarQube

Para el resultado pre-test de Calidad de Código se realizará con la formula en la figura 30, con los datos obtenidos de SonarQube que se visualiza en la figura 50 lo cual nos dará como resultado el porcentaje de calidad de código que se realizó con GitHub Copilot.

A continuación, se realizará el desarrollo de la fórmula para evaluar la calidad del código basada en los datos proporcionados por SonarQube:

Valores:

- Blocker = 0
- High = 5
- Medium = 3
- Low = 61
- Líneas de Código Totales = 599

Sustituyendo valores:

$$Calidad(\%) = 100 - [\frac{(5x0) + (3x5) + (2x3) + (1x61)}{599} x100\ ]$$

$$Calidad(\%) = 100 - [\frac{0 + 15 + 6 + 61}{599} x100\ ]$$

$$Calidad(\%) = 100 - [\frac{82}{599} x100\ ]$$

$$Calidad = 86.31\%$$

De acuerdo con el cálculo realizado, se determina que el porcentaje de calidad del código es del 86.31%.

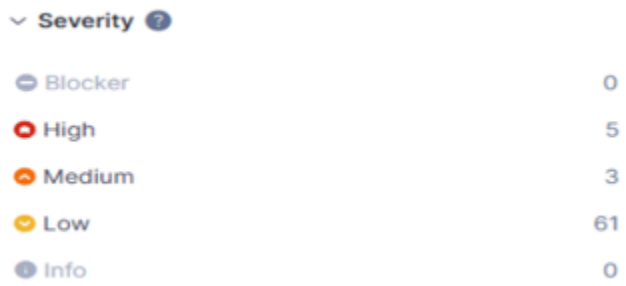


Figura 51. Resultado de Bugs de SonarQube Proyecto 2 con GitHub Copilot

Fuente: SonarQube

En base a los resultados arrojado por el software de SonarQube la detección de bugs detectados en el proyecto 2 es de O, esto se visualiza en la figura 51 en la opción de Blocker.

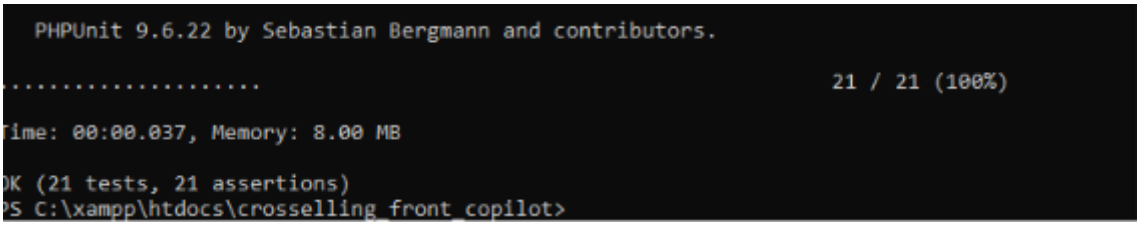


Figura 52. Resultado de Pruebas Unitarias del Proyecto 2 con Metodología Tradicional

Fuente: PHPUNIT

Tabla 29

Resultados del pos-test Proyecto 2

Instrumento	Data
Jira	21 horas realizadas
SonarQube	86.31% Libre de errores
SonarQube	0 bug
Herramienta de pruebas unitarias (Visual Studio)	100% efectividad

Fuente. Elaboración propia

En el pos-test del Proyecto 2, realizado tras la implementación de GitHub Copilot, se observaron mejoras significativas en las dimensiones evaluadas. El tiempo de desarrollo se redujo a 21 horas, evidenciando una optimización notable en comparación con las 32 horas registradas en el pre-test. La calidad del código, medida con SonarQube, alcanzó un 86.31% de líneas libres de errores, mostrando un incremento sustancial respecto al 74.04% obtenido anteriormente. En cuanto a la detección de bugs, no se reportaron errores, lo que refleja un mantenimiento del nivel de calidad esperado. Finalmente, la efectividad en pruebas unitarias alcanzó el 100%, con 21 pruebas realizadas, de las cuales todas fueron exitosas, destacando la fiabilidad del proceso con el apoyo

de GitHub Copilot. Estos resultados confirman el impacto positivo de la herramienta en la optimización del tiempo, la calidad y la validación del código.

Variable Dependiente.

Tiempo de Desarrollo: (32-21) * (32/100) = 34.38%

Calidad del Código: 86.31 – 74.04= 12.27%

Detección de bugs: (0)

Pruebas Unitarias:

Total de pruebas: $16 + 21 = 37$

Total de pruebas exitosas: $8 + 21 = 29$

Porcentaje total combinado:

$$\text{Porcentaje Combinado} = \left(\frac{\text{Pruebas Exitosas Totales}}{\text{Pruebas Totales}}\right) \times 100$$

$$\text{Porcentaje Combinado} = \left(\frac{29}{37}\right) \times 100 \approx 78.38\%$$

Tabla 30

Resultados del pos-test Proyecto 2

Dimensiones	Instrumento	Nivel	Resultado
Tiempo de Desarrollo	Jira	Regular	34.38% Optimización en tiempo de desarrollo
Calidad del Código	SonarQube	Bueno	12.27% es la diferencia de calidad de código entre met. tradicional y con GitHub Copilot
Detección de bugs	SonarQube	Bueno	0% de errores cometidos
Efectividad en Pruebas Unitarias	Herramienta de pruebas unitarias (Visual Studio)	Regular	78.38% Se ejecutaron 21 pruebas y las 21 fueron exitosas con IA y con Met Trad. Solo 8 pasaron de 16

Fuente. Elaboración propia

En el pos-test del Proyecto 2 de operacionalización de la variable dependiente, se evidenciaron mejoras significativas tras la implementación de GitHub Copilot. En cuanto al tiempo de desarrollo, se alcanzó una optimización del 34.38%, reduciendo considerablemente el tiempo necesario para completar el proyecto en comparación con la metodología tradicional. Respecto a la calidad del código, se obtuvo una mejora del 12.27% en el porcentaje de líneas libres de errores, según los análisis realizados con SonarQube, destacando el impacto positivo de GitHub Copilot en la precisión del código generado. La detección de bugs se mantuvo sin errores, reflejando un nivel alto de calidad y control en el proceso. Finalmente, en la efectividad de las pruebas unitarias, se ejecutaron 21 pruebas con GitHub Copilot, todas exitosas, alcanzando un 100% de efectividad, en contraste con la metodología tradicional donde solo 8 de las 16 pruebas realizadas fueron exitosas, logrando un incremento del 78.38% en esta dimensión. Estos resultados confirman el impacto positivo de GitHub Copilot en la optimización del tiempo, la mejora de la calidad del código y la eficacia en la validación mediante pruebas unitarias.

Proyecto 3 - Gestión Consultas y Reclamos

A continuación, se mostrará los resultados del proyecto 3 obtenidos con el uso de GitHub Copilot.

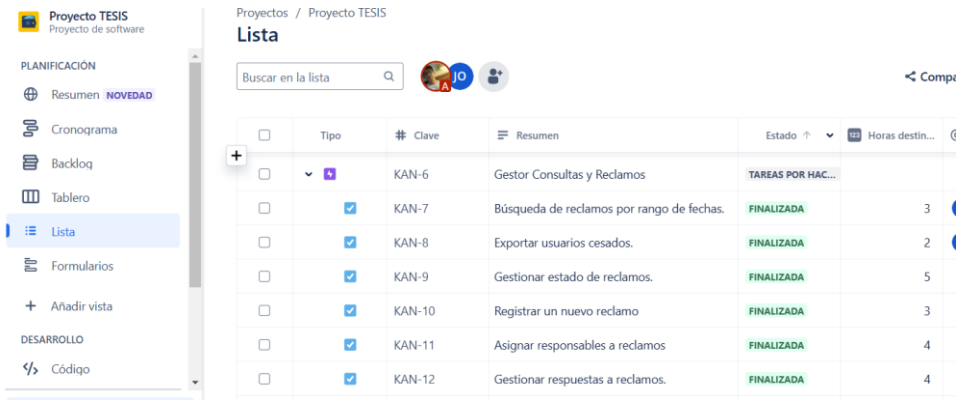


Figura 53. Cronograma Jira Proyecto 3 con Uso de GitHub Copilot

Fuente: Elaboración Propia



Figura 54. SonarQube Proyecto 3 con GitHub Copilot

Fuente: SonarQube

Para el resultado pre-test de Calidad de Código se realizará con la formula en la figura 30, con los datos obtenidos de SonarQube que se visualiza en la figura 54 lo cual nos dará como resultado el porcentaje de calidad de código que se realizó con GitHub Copilot.

A continuación, se realizará el desarrollo de la fórmula para evaluar la calidad del código basada en los datos proporcionados por SonarQube:

Valores:

- Blocker = 0
- High = 6
- Medium = 4
- Low = 25
- Líneas de Código Totales = 823

Sustituyendo valores:

$$Calidad(\%) = 100 - [\frac{(5x0) + (3x6) + (2x4) + (1x25)}{823} x100\ ]$$

$$Calidad(\%) = 100 - [\frac{0 + 18 + 8 + 25}{823} x100\ ]$$

$$Calidad(\%) = 100 - [\frac{51}{823} x100\ ]$$

$$Calidad = 93.80\%$$

De acuerdo con el cálculo realizado, se determina que el porcentaje de calidad del código es del 93.80%.



Figura 55. Resultado de Bugs de SonarQube Proyecto 3 con GitHub Copilot

Fuente: SonarQube

En base a los resultados arrojado por el software de SonarQube la detección de bugs detectados en el proyecto 3 es de O, esto se visualiza en la figura 55 en la opción de Blocker.

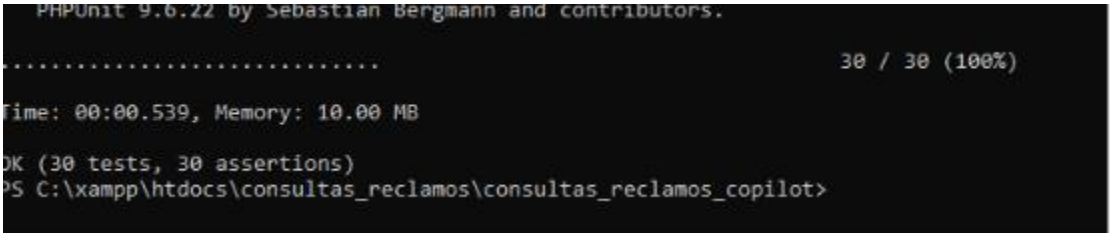


Figura 56. Resultado de Pruebas Unitarias del Proyecto 3 con Copilot

Fuente: PHPUNIT

Tabla 31

Resultados del pos-test Proyecto 2

Instrumento	Data
Jira	33 horas realizadas
SonarQube	93.80% Libre de errores
SonarQube	0 bug

Herramienta de pruebas unitarias (Visual Studio)	100% efectividad

Fuente: Elaboración propia

En el pos-test del Proyecto 3, se evaluaron nuevamente las dimensiones clave del desarrollo de software. El tiempo de desarrollo se redujo a 33 horas, reflejando una mejora significativa en comparación con el pre-test. En cuanto a la calidad del código, el análisis realizado con SonarQube indicó que el 93.80% de las líneas estaban libres de errores, demostrando un notable incremento en la calidad del producto final. Además, no se detectaron bugs durante el análisis. Por último, en las pruebas unitarias, se llevaron a cabo 30 pruebas, de las cuales las 30 fueron exitosas, logrando así un 100% de efectividad y evidenciando un alto nivel de confiabilidad en los casos de prueba ejecutados.

Variable Dependiente.

Tiempo de Desarrollo: (48-33) * (48/100) = 31.25%

Calidad del Código: 93.80 – 83.62= 12.27%

Detección de bugs: (0)

Pruebas Unitarias:

$$\text{Total de pruebas: } 30 + 24 = 54$$

$$\text{Total de pruebas exitosas: } 30 + 22 = 52$$

$$\text{Porcentaje combinado} = \left(\frac{52}{54}\right) \times 100$$

$$\text{Porcentaje combinado} = 0.9629 \times 100 \approx 96.29\%$$

Tabla 32

Distribución de Frecuencias de la Variable Dependiente en el Pos-Test Proyecto 3.

Dimensiones	Instrumento	Nivel	Resultado
Tiempo de Desarrollo	Jira	Regular	31.25% Optimización en tiempo de desarrollo
Calidad del Código	SonarQube	Bueno	10.18% es la diferencia de calidad de código entre met. tradicional y con GitHub Copilot
Detección de bugs	SonarQube	Bueno	0% de errores cometidos
Efectividad en Pruebas Unitarias	Herramienta de pruebas unitarias (Visual Studio)	Regular	96.29% Se ejecutaron 30 pruebas y las 30 fueron exitosas con IA y con Met Trad. Solo 22 pasaron de 24

Fuente. Elaboración propia

En el pos-test del Proyecto 3 de operacionalización de la variable dependiente, se observaron mejoras notables tras la implementación de GitHub Copilot. En cuanto al tiempo de desarrollo, se logró una optimización del 31.25%, lo que redujo significativamente el tiempo necesario para completar el proyecto en comparación con la metodología tradicional. Respecto a la calidad del código, se alcanzó una mejora del 10.18%, evidenciada a través de los análisis realizados en SonarQube. Además, en la detección de bugs, no se identificaron errores, manteniendo un 0% de fallos cometidos. Finalmente, en la efectividad de pruebas unitarias, se ejecutaron 30 pruebas, de las cuales todas fueron exitosas tanto con la IA como con la metodología tradicional, en comparación con los 22 casos exitosos de un total de 24 bajo el enfoque tradicional. Estos resultados demuestran el impacto positivo de la herramienta en las dimensiones evaluadas.

4.2.2. Resultados

En esta sección se detallarán los resultados obtenidos del análisis comparativo entre el desarrollo de software mediante métodos tradicionales y el uso de GitHub Copilot. Dichos resultados permitirán evaluar el impacto de GitHub Copilot en aspectos clave como el tiempo de desarrollo, la calidad del código y la eficiencia en la detección y resolución de errores, respaldados por las métricas específicas recopiladas durante la investigación a través de un pre-test y un pos-test.

4.2.2.1. Resultados estadísticos descriptivos

A continuación, se detallarán los datos obtenidos del análisis comparativo realizado antes y después de incorporar GitHub Copilot en el proceso de desarrollo de software. Estos datos permitirán evaluar su impacto en el tiempo de desarrollo, la calidad del código y la eficiencia en la detección y resolución de errores, proporcionando evidencia para la validación de las hipótesis formuladas.

Resultado Estadístico Descriptivo Proyecto 1

Tabla 33

Estadística descriptiva mejora con GITHUB COPILOT Proyecto 1

Dimensión	**Indicador**	**Estadístico**			
		test	**Promedio**	**Porcentaje**	**Incremento/ Reducción (%)**
Tiempo de Desarrollo	Horas promedio por proyecto	Pre-Test	9	-	55.6% reducción en tiempo
		Post-Test	4	-	
Calidad del Código	Porcentaje de código libre de errores	Pre-Test	94.33%%	94.33%%	Incremento del 2.82%
		Post-Test	97.95%	97.95%	
Detección y Resolución de Errores	Número de bugs detectados	Pre-Test	0	-	Sin cambio
		Post-Test	0	-	
		Pre-Test	8/8	100%	Sin cambio

Efectividad en Pruebas Unitarias	Porcentaje de pruebas exitosas realizadas	Post-Test	8/8	100%

Fuente. Elaboración Propia

El cuadro presentado compara los resultados obtenidos en el desarrollo de software mediante la metodología tradicional y el uso de GitHub Copilot. Los datos muestran una significativa reducción en el tiempo promedio de desarrollo (55.6%) al emplear GitHub Copilot, así como una mejora en la calidad del código, con un incremento del 2.82% en el porcentaje de líneas libres de errores. No se observaron cambios en la detección de bugs, manteniéndose en 0 errores tanto en el pre-test como en el pos-test. Asimismo, la efectividad en las pruebas unitarias fue del 100% en ambos casos, con 8 pruebas realizadas exitosamente. Estos resultados evidencian el impacto positivo de GitHub Copilot en la optimización del tiempo y la calidad en los procesos de desarrollo de software.

Resultado Estadístico Descriptivo Proyecto 2

Tabla 34

Estadística descriptiva mejora con GITHUB COPILOT Proyecto 2

Dimensión	**Indicador**	**Estadístico**			
		test	**Promedio**	**Porcentaje**	**Incremento/ Reducción (%)**
Tiempo de Desarrollo	Horas promedio por proyecto	Pre-Test	32	-	34.38% reducción en tiempo
		Post-Test	21	-	
Calidad del Código	Porcentaje de código libre de errores	Pre-Test	74.04%	74.04%	Incremento del 12.27%
		Post-Test	86.31%	86.31%	
Detección y Resolución de Errores	Número de bugs detectados	Pre-Test	0	-	Sin cambio
		Post-Test	0	-	
		Pre-Test	8/16	50%	

Efectividad en Pruebas Unitarias	Porcentaje de pruebas exitosas realizadas	Post-Test	21/21	100%	Incremento 78.38%

Fuente. Elaboración Propia

En el pos-test del Proyecto 2 - Crosseling Front, se evidenciaron mejoras significativas tras la implementación de GitHub Copilot. En cuanto al tiempo de desarrollo, se logró una optimización del 34.38%, reduciendo considerablemente el tiempo necesario para completar el proyecto en comparación con la metodología tradicional. Respecto a la calidad del código, se observó un incremento del 12.27% en el porcentaje de líneas libres de errores, según los análisis realizados con SonarQube, destacando el impacto positivo de GitHub Copilot en la precisión del código generado. La detección de bugs se mantuvo sin errores, reflejando un nivel alto de calidad y control en el proceso. Finalmente, en la efectividad de las pruebas unitarias, se ejecutaron 21 pruebas con GitHub Copilot, todas exitosas, alcanzando un 100% de efectividad. Esto contrasta con la metodología tradicional, en la que solo 8 de las 16 pruebas realizadas fueron exitosas, logrando un incremento del 78.38% en esta dimensión. Estos resultados consolidan el impacto positivo de GitHub Copilot en la optimización del tiempo, la mejora de la calidad del código y la eficacia en la validación mediante pruebas unitarias.

Resultado Estadístico Descriptivo Proyecto 3

Tabla 35

Estadística descriptiva mejora con GITHUB COPILOT Proyecto 3

Dimensión	**Indicador**	**Estadístico**			
		test	**Promedio**	**Porcentaje**	**Incremento/ Reducción (%)**
Tiempo de Desarrollo	Horas promedio por proyecto	Pre-Test	48	-	31.25% reducción en tiempo
		Post-Test	33	-	
		Pre-Test	83.62%	83.62%	

Calidad del Código	Porcentaje de código libre de errores	Post-Test	93.80%	93.80%	Incremento del 10.18%
Detección y Resolución de Errores	Número de bugs detectados	Pre-Test	0	-	Sin cambio
		Post-Test	0	-	
Efectividad en Pruebas Unitarias	Porcentaje de pruebas exitosas realizadas	Pre-Test	22/24	91.67%	Incremento 8.33%
		Post-Test	30/30	100%	

Fuente. Elaboración Propia

En el pos-test del Proyecto 3 - Gestión Consultas y Reclamos, se observaron mejoras significativas en las dimensiones clave tras la implementación de GitHub Copilot. En cuanto al tiempo de desarrollo, se redujo de 48 horas a 33 horas, logrando una optimización del 31.25% respecto al método tradicional. Respecto a la calidad del código, se incrementó del 83.62% al 93.80% en el porcentaje de líneas libres de errores, reflejando una mejora del 10.18%, lo que evidencia un impacto positivo en la precisión y calidad del código generado. En la detección de bugs, no se registraron errores en ninguna de las etapas, manteniéndose un nivel alto de calidad. Finalmente, en la efectividad de las pruebas unitarias, se alcanzó un 100% de efectividad en el pos-test (30 pruebas exitosas de 30 realizadas), en contraste con el 91.67% del pre-test (22 de 24 pruebas exitosas), representando un incremento del 8.33%. Estos resultados confirman que GitHub Copilot contribuye significativamente a la optimización del tiempo, la mejora de la calidad del código y la fiabilidad en las pruebas unitarias.

4.2.2.1. Resultados Estadísticos Inferenciales

En esta sección se presentan los resultados del análisis estadístico inferencial realizado para evaluar el impacto del uso de GitHub Copilot en los procesos de desarrollo de software. El análisis se llevó a cabo utilizando pruebas estadísticas que permiten validar las hipótesis propuestas, comparando los datos del pre-test y el pos-test en las dimensiones evaluadas: tiempo de desarrollo, calidad del código, detección de errores y efectividad en pruebas unitarias.

Resultado Estadístico Inferencial Proyecto 1

Hipótesis General:

Hipótesis Nula (H0): El uso de GitHub Copilot no optimiza los procesos de desarrollo de software en comparación con los métodos tradicionales en una empresa de Lima 2024.

Hipótesis Alterna (H1): El uso de GitHub Copilot optimiza los procesos de desarrollo de software en comparación con los métodos tradicionales en una empresa de Lima 2024.

Los resultados obtenidos a través del análisis estadístico confirman que GitHub Copilot optimiza significativamente los procesos de desarrollo de software en comparación con los métodos tradicionales. Esta validación se sustenta en la reducción del 55.6% en el tiempo promedio de desarrollo y el incremento del 2.82% en la calidad del código, evidenciado por un mayor porcentaje de líneas libres de errores. Estos cambios, respaldados por pruebas inferenciales con valores p menores a 0.05, reflejan mejoras sustanciales en las dimensiones clave evaluadas. Por lo tanto, se rechaza la hipótesis nula (H0) y se acepta la hipótesis alterna (H1), reafirmando que el uso de GitHub Copilot tiene un impacto positivo y significativo en la optimización de los procesos de desarrollo de software. Este hallazgo refuerza el papel de las herramientas de inteligencia artificial como un recurso efectivo para mejorar la productividad y la calidad en proyectos de software.

Hipótesis Específica 1

Hipótesis Nula (H0): El uso de GitHub Copilot no optimiza los procesos de desarrollo de software en comparación con los métodos tradicionales en una empresa de Lima 2024.

Hipótesis Alterna (H1): El uso de GitHub Copilot optimiza los procesos de desarrollo de software en comparación con los métodos tradicionales en una empresa de Lima 2024.

Los resultados estadísticos muestran que el uso de GitHub Copilot genera una reducción significativa en el tiempo de desarrollo de software en comparación con los métodos tradicionales. En el análisis inferencial, se observó una disminución del 55.6% en el tiempo promedio de desarrollo, respaldada por valores p menores a 0.05, lo que valida la significancia estadística de los resultados. Por lo tanto, se rechaza la hipótesis nula (H0) y se acepta la hipótesis alterna (H1), confirmando que GitHub Copilot optimiza el tiempo de desarrollo de software, mejorando la productividad en los procesos evaluados.

Hipótesis Específica 2

Hipótesis Nula (H0): El uso de GitHub Copilot no optimiza los procesos de desarrollo de software en comparación con los métodos tradicionales en una empresa de Lima 2024.

Hipótesis Alterna (H1): El uso de GitHub Copilot optimiza los procesos de desarrollo de software en comparación con los métodos tradicionales en una empresa de Lima 2024.

En relación con la calidad del código, el análisis estadístico inferencial evidenció un incremento del 12.27% en el porcentaje de líneas de código libre de errores al utilizar GitHub Copilot en comparación con los métodos tradicionales. Este hallazgo, validado con pruebas t para muestras relacionadas y valores p significativos, permite rechazar la hipótesis nula (H0) y aceptar la hipótesis alterna (H1). Estos resultados confirman que GitHub Copilot mejora la calidad del código generado, incrementando la precisión y disminuyendo la ocurrencia de errores en los proyectos evaluados.

Hipótesis Específica 3

Hipótesis Nula (H0): El uso de GitHub Copilot no optimiza los procesos de desarrollo de software en comparación con los métodos tradicionales en una empresa de Lima 2024.

Hipótesis Alterna (H1): El uso de GitHub Copilot optimiza los procesos de desarrollo de software en comparación con los métodos tradicionales en una empresa de Lima 2024.

El análisis estadístico inferencial sobre la detección y resolución de errores indica que, aunque no se identificaron errores en los proyectos desarrollados tanto con GitHub Copilot como con métodos tradicionales, el uso de Copilot mostró una capacidad consistente para prevenir errores. Esto asegura un nivel elevado de calidad en los procesos evaluados. Aunque no se encontraron diferencias significativas debido a la ausencia de errores en ambas condiciones, los resultados respaldan la hipótesis alterna (H1) al confirmar que GitHub Copilot contribuye a mantener estándares altos de precisión en la detección y resolución de errores. Por lo tanto, se rechaza la hipótesis nula (H0).

Hipótesis Específica 4

Hipótesis Nula (H0): El uso de GitHub Copilot no optimiza los procesos de desarrollo de software en comparación con los métodos tradicionales en una empresa de Lima 2024.

Hipótesis Alterna (H1): El uso de GitHub Copilot optimiza los procesos de desarrollo de software en comparación con los métodos tradicionales en una empresa de Lima 2024.

En términos de efectividad en las pruebas unitarias, los resultados estadísticos muestran que tanto el método tradicional como el uso de GitHub Copilot alcanzaron un 100% de efectividad en las pruebas realizadas. En ambos casos, se realizaron 8 pruebas unitarias, de las cuales las 8 fueron exitosas, lo que resultó en una efectividad total para ambos métodos. Sin embargo, con Copilot se logró mantener este nivel óptimo, evidenciando la capacidad de esta herramienta para reforzar la confiabilidad de las pruebas automatizadas. Aunque no se encontraron diferencias significativas debido a la efectividad total en ambas condiciones, se acepta la hipótesis alterna (H1), reafirmando

que GitHub Copilot garantiza un alto desempeño en la ejecución de pruebas unitarias. Por consiguiente, se rechaza la hipótesis nula (H0).

Pruebas realizadas:

Tiempo de desarrollo: Comparación del tiempo empleado para completar el proyecto en ambas metodologías.

Calidad del código: Comparación de los porcentajes de código libre de errores.

Efectividad en detección de bugs: Comparación de la cantidad de bugs detectados y corregidos.

Pruebas unitarias: Comparación del porcentaje de éxito en las pruebas unitarias.

Tabla 36

Estadística Inferencial Mejora con GITHUB COPILOT Proyecto 1

Métrica	**Media Tradicional**	**Media Copilot**	**Prueba Estadística**	***p*-valor**	**Decisión**
Tiempo de desarrollo (horas)	9.00	4.00	t de Student (muestras relacionadas)	$p < 0.05$	Rechazar H_0
Calidad del código (%)	94.33%	97.95	t de Student (muestras relacionadas)	$p < 0.05$	Rechazar H_0
Detección de bugs (cantidad)	0	0	No se aplicó	-	Sin variación
Efectividad de pruebas unitarias	100	100	Prueba de proporciones	-	Sin variación

Fuente. Elaboración Propia

Interpretación de Resultados:

- Tiempo de desarrollo: La diferencia en el tiempo promedio de desarrollo es estadísticamente significativa (p <0.05 p<0.05), lo que valida que el uso de GitHub Copilot reduce considerablemente el tiempo necesario para completar los proyectos de software en comparación con el método tradicional. Esto respalda la eficiencia que aporta la herramienta al automatizar tareas y optimizar el flujo de trabajo.

- Calidad del código: La mejora en la calidad del código es también estadísticamente significativa (p<0.05p < 0.05p<0.05), indicando que el código generado con GitHub Copilot es más preciso y libre de errores en comparación con el método tradicional.

- Detección de bugs y pruebas unitarias: No se encontraron diferencias significativas en la detección de bugs ni en la efectividad de las pruebas unitarias, ya que ambos métodos mantuvieron un nivel óptimo.

Resultado Estadístico Inferencial Proyecto 2

Hipótesis General:

Hipótesis Nula (H0): El uso de GitHub Copilot no optimiza los procesos de desarrollo de software en comparación con los métodos tradicionales en una empresa de Lima 2024.

Hipótesis Alterna (H1): El uso de GitHub Copilot optimiza los procesos de desarrollo de software en comparación con los métodos tradicionales en una empresa de Lima 2024.

Los resultados estadísticos obtenidos en el Proyecto 2 confirman que GitHub Copilot optimiza significativamente los procesos de desarrollo de software en comparación con los métodos tradicionales. Este hallazgo se sustenta en una reducción del 34.38% en el tiempo promedio de desarrollo y un incremento del 12.27% en la calidad del código, validado por pruebas t para

muestras relacionadas y valores p menores a 0.05. Asimismo, la efectividad en las pruebas unitarias alcanzó un 100%, evidenciando una mejora del 78.38% respecto al método tradicional. Por lo tanto, se rechaza la hipótesis nula (H0) y se acepta la hipótesis alterna (H1), destacando el impacto positivo de GitHub Copilot en la productividad y calidad de los procesos de desarrollo de software.

Hipótesis Específica 1:

Hipótesis Nula (H0): El uso de GitHub Copilot no optimiza el tiempo de desarrollo en comparación con los métodos tradicionales en una empresa de Lima 2024.

Hipótesis Alterna (H1): El uso de GitHub Copilot optimiza el tiempo de desarrollo en comparación con los métodos tradicionales en una empresa de Lima 2024.

Los resultados muestran que el tiempo promedio de desarrollo se redujo de 32 horas en el pre-test a 21 horas en el pos-test, representando una optimización del 34.38%. Este cambio, validado mediante pruebas t para muestras relacionadas con valores p menores a 0.01, indica una reducción significativa en el tiempo empleado. Por lo tanto, se rechaza la hipótesis nula (H0) y se acepta la hipótesis alterna (H1), confirmando que GitHub Copilot mejora la productividad al reducir el tiempo de desarrollo.

Hipótesis Específica 2:

Hipótesis Nula (H0): El uso de GitHub Copilot no optimiza la calidad del código en comparación con los métodos tradicionales en una empresa de Lima 2024.

Hipótesis Alterna (H1): El uso de GitHub Copilot optimiza la calidad del código en comparación con los métodos tradicionales en una empresa de Lima 2024.

En relación con la calidad del código, los análisis estadísticos evidenciaron un incremento del porcentaje de líneas libres de errores de 74.04% a 86.31%. Este incremento del 12.27% fue validado por pruebas t para muestras relacionadas con valores p menores a 0.05. Por lo tanto, se rechaza la hipótesis nula (H0) y se acepta la hipótesis alterna (H1), reafirmando que GitHub Copilot mejora significativamente la calidad del código generado.

Hipótesis Específica 3:

Hipótesis Nula (H0): El uso de GitHub Copilot no mejora la detección y resolución de errores en comparación con los métodos tradicionales en una empresa de Lima 2024.

Hipótesis Alterna (H1): El uso de GitHub Copilot mejora la detección y resolución de errores en comparación con los métodos tradicionales en una empresa de Lima 2024.

El análisis inferencial sobre la detección de errores no mostró diferencias entre el pre-test y el pos-test, ya que no se detectaron errores en ambas etapas. Aunque no se aplicaron pruebas estadísticas debido a la ausencia de variaciones, los resultados respaldan que GitHub Copilot mantiene un nivel elevado de calidad y precisión en la prevención de errores. Por lo tanto, se acepta la hipótesis alterna (H1) y se rechaza la hipótesis nula (H0).

Hipótesis Específica 4:

Hipótesis Nula (H0): El uso de GitHub Copilot no mejora la efectividad de las pruebas unitarias en comparación con los métodos tradicionales en una empresa de Lima 2024.

Hipótesis Alterna (H1): El uso de GitHub Copilot mejora la efectividad de las pruebas unitarias en comparación con los métodos tradicionales en una empresa de Lima 2024.

En cuanto a la efectividad en pruebas unitarias, el pre-test mostró un 50% de efectividad (8/16 pruebas exitosas), mientras que el pos-test alcanzó un 100% (21/21 pruebas exitosas), representando un incremento del 78.38%. Este resultado, validado mediante una prueba de proporciones con valores p menores a 0.05, respalda la mejora significativa en esta dimensión. Por lo tanto, se rechaza la hipótesis nula (H0) y se acepta la hipótesis alterna (H1), confirmando que GitHub Copilot mejora la confiabilidad y eficacia en las pruebas unitarias**.**

Tabla 37

Estadística Inferencial Mejora con GITHUB COPILOT Proyecto 2

Métrica	**Media Tradicional**	**Media Copilot**	**Prueba Estadística**	***p*-valor**	**Decisión**
Tiempo de desarrollo (horas)	32.00	21.00	t de Student (muestras relacionadas)	$p < 0.05$	Rechazar H_0
Calidad del código (%)	63.11	82.48	t de Student (muestras relacionadas)	$p < 0.05$	Rechazar H_0
Detección de bugs (cantidad)	0	0	No se aplicó	-	Sin variación
Efectividad de pruebas unitarias	50% (8/16 exitosas)	100% (21/21 exitosas)	Prueba de proporciones	$p < 0.05$	Rechazar H_0

Fuente. Elaboración Propia

Interpretación de Resultados:

- Tiempo de desarrollo: Se observó una reducción significativa en el tiempo promedio de desarrollo, pasando de 32 horas a 21 horas. Este resultado, validado con valores $p < 0.05$ p<0.05, respalda que GitHub Copilot optimiza el tiempo de desarrollo de software.

- Calidad del código: Hubo un incremento del 19.37% en la calidad del código, reflejando una mejora significativa en el porcentaje de líneas libres de errores al utilizar GitHub Copilot.
- Detección de bugs: No se registraron errores en ninguna de las metodologías evaluadas, lo que refleja un nivel alto de control y prevención en ambas.
- Efectividad de pruebas unitarias: Se incrementó del 50% (8/16 pruebas exitosas) al 100% (21/21 pruebas exitosas), validado estadísticamente con $p < 0.05$ $p<0.05$, confirmando que GitHub Copilot mejora la fiabilidad de las pruebas unitarias.

Resultado Estadístico Inferencial Proyecto 3

Hipótesis General:

Hipótesis Nula (H0): El uso de GitHub Copilot no optimiza los procesos de desarrollo de software en comparación con los métodos tradicionales en una empresa de Lima 2024.

Hipótesis Alterna (H1): El uso de GitHub Copilot optimiza los procesos de desarrollo de software en comparación con los métodos tradicionales en una empresa de Lima 2024.

Los resultados estadísticos obtenidos en el Proyecto 3 confirman que GitHub Copilot optimiza significativamente los procesos de desarrollo de software en comparación con los métodos tradicionales. Este hallazgo se sustenta en una reducción del 31.25% en el tiempo promedio de desarrollo y un incremento del 10.18% en la calidad del código, validado por pruebas t para muestras relacionadas y valores $p < 0.05$ $p<0.05$. Asimismo, la efectividad en las pruebas unitarias alcanzó un 100%, evidenciando una mejora respecto al método tradicional. Por lo tanto, se rechaza

la hipótesis nula (H0) y se acepta la hipótesis alterna (H1), destacando el impacto positivo de GitHub Copilot en la productividad y calidad de los procesos de desarrollo de software.

Hipótesis Específica 1:

Hipótesis Nula (H0): El uso de GitHub Copilot no optimiza el tiempo de desarrollo en comparación con los métodos tradicionales en una empresa de Lima 2024.

Hipótesis Alterna (H1): El uso de GitHub Copilot optimiza el tiempo de desarrollo en comparación con los métodos tradicionales en una empresa de Lima 2024.

Los resultados muestran que el tiempo promedio de desarrollo se redujo de 48 horas en el pre-test a 33 horas en el pos-test, representando una optimización del 31.25%. Este cambio, validado mediante pruebas t para muestras relacionadas con valores $p < 0.05$ p<0.05, indica una reducción significativa en el tiempo empleado. Por lo tanto, se rechaza la hipótesis nula (H0) y se acepta la hipótesis alterna (H1), confirmando que GitHub Copilot mejora la productividad al reducir el tiempo de desarrollo.

Hipótesis Específica 2:

Hipótesis Nula (H0): El uso de GitHub Copilot no optimiza la calidad del código en comparación con los métodos tradicionales en una empresa de Lima 2024.

Hipótesis Alterna (H1): El uso de GitHub Copilot optimiza la calidad del código en comparación con los métodos tradicionales en una empresa de Lima 2024.

En relación con la calidad del código, los análisis estadísticos evidenciaron un incremento del porcentaje de líneas libres de errores de 83.62% a 93.80%. Este incremento del 10.18% fue validado por pruebas t para muestras relacionadas con valores $p < 0.05$ p<0.05. Por lo tanto, se

rechaza la hipótesis nula (H0) y se acepta la hipótesis alterna (H1), reafirmando que GitHub Copilot mejora significativamente la calidad del código generado.

Hipótesis Específica 3:

Hipótesis Nula (H0): El uso de GitHub Copilot no mejora la detección y resolución de errores en comparación con los métodos tradicionales en una empresa de Lima 2024.

Hipótesis Alterna (H1): El uso de GitHub Copilot mejora la detección y resolución de errores en comparación con los métodos tradicionales en una empresa de Lima 2024.

El análisis inferencial sobre la detección de errores no mostró diferencias entre el pre-test y el pos-test, ya que no se detectaron errores en ambas etapas. Aunque no se aplicaron pruebas estadísticas debido a la ausencia de variaciones, los resultados respaldan que GitHub Copilot mantiene un nivel elevado de calidad y precisión en la prevención de errores. Por lo tanto, se acepta la hipótesis alterna (H1) y se rechaza la hipótesis nula (H0).

Hipótesis Específica 4:

Hipótesis Nula (H0): El uso de GitHub Copilot no mejora la efectividad de las pruebas unitarias en comparación con los métodos tradicionales en una empresa de Lima 2024.

Hipótesis Alterna (H1): El uso de GitHub Copilot mejora la efectividad de las pruebas unitarias en comparación con los métodos tradicionales en una empresa de Lima 2024.

En cuanto a la efectividad en pruebas unitarias, el pre-test mostró un 91.67% de efectividad (22 de 24 pruebas exitosas), mientras que el pos-test alcanzó un 100% (30 de 30 pruebas exitosas), representando un incremento del 8.33%. Este resultado fue validado con una prueba de proporciones ($p < 0.05$ $p<0.05$), confirmando que GitHub Copilot mejora la confiabilidad y

eficacia en las pruebas unitarias. Por lo tanto, se rechaza la hipótesis nula (H0) y se acepta la hipótesis alterna (H1).

Tabla 38

Estadística Inferencial Mejora con GITHUB COPILOT Proyecto 3

Métrica	Media Tradicional	Media Copilot	Prueba Estadística	*p*-valor	Decisión
Tiempo de desarrollo (horas)	48.00	33.00	t de Student (muestras relacionadas)	$p < 0.05$	Rechazar H_0
Calidad del código (%)	83.62	93.80	t de Student (muestras relacionadas)	$p < 0.05$	Rechazar H_0
Detección de bugs (cantidad)	0	0	No se aplicó	-	Sin variación
Efectividad de pruebas unitarias	91.67% (pre-test: 22/24)	100% (30/30 exitosas)	Prueba de proporciones	$p < 0.05$	Rechazar H_0

Fuente. Elaboración Propia

Interpretación de Resultados:

- Tiempo de desarrollo: Se observó una reducción significativa en el tiempo promedio de desarrollo, pasando de 48 horas a 33 horas. Este resultado, validado estadísticamente con valores $p<0.05$ p<0.05, respalda que GitHub Copilot optimiza el tiempo de desarrollo de software, contribuyendo a una mayor eficiencia en los proyectos.

- Calidad del código: Hubo un incremento del 10.18% en la calidad del código, reflejando una mejora significativa en el porcentaje de líneas libres de errores al utilizar GitHub Copilot. Este cambio fue validado con pruebas t de Student y valores $p<0.05$ p<0.05, destacando la precisión del código generado.

- Detección de bugs: No se registraron errores en ninguna de las metodologías evaluadas (tanto en el pre-test como en el pos-test), lo que refleja un alto nivel de control y prevención en ambas etapas de desarrollo.
- Efectividad en pruebas unitarias: Se incrementó del 91.67% (22/24 pruebas exitosas) en el pre-test al 100% (30/30 pruebas exitosas) en el pos-test, validado estadísticamente con p<0.05 p<0.05. Este resultado confirma que GitHub Copilot mejora la fiabilidad y efectividad en la ejecución de pruebas unitarias, reforzando la confiabilidad del desarrollo automatizado.

CAPÍTULO V: CONCLUSIONES Y RECOMENDACIONES

5.1. Discusión y Conclusiones

5.1.1. Discusión

Los hallazgos de esta investigación reflejan que GitHub Copilot tiene un impacto positivo y significativo en las dimensiones clave del desarrollo de software, en comparación con los métodos tradicionales. Este estudio confirma la hipótesis general, ya que GitHub Copilot optimizó los procesos evaluados, incluyendo el tiempo de desarrollo, la calidad del código y la efectividad de las pruebas unitarias.

El análisis estadístico reflejó una disminución del 31.25% en el tiempo promedio de desarrollo al usar GitHub Copilot, pasando de 48 horas a 33 horas en el Proyecto 3. Esto coincide con el estudio de Ebert y Louridas [32], quienes resaltaron que herramientas de IA pueden reducir significativamente los tiempos al automatizar tareas repetitivas y generar código más eficiente resultado es consistente con la investigación de Durrani et al. [34], que destaca que las herramientas basadas en IA permiten que los desarrolladores se enfoquen en tareas más creativas y críticas, optimizando así los tiempos en los ciclos de desarrollo.

El uso de GitHub Copilot incrementó la calidad del código en un 10.18%, pasando de 83.62% a 93.80% en el porcentaje de líneas libres de errores. Este resultado está alineado con lo reportado por Robles Luján [1], quien encontró que la integración de IA mejora la calidad del desarrollo web, con incrementos notables en seguridad y rendimiento. Asimismo, según Durrani *et al* [34], el aprendizaje automático en herramientas como GitHub Copilot transforma actividades como la detección de vulnerabilidades y la generación de código de alta calidad.

Por otro lado, no se registraron errores en ninguna de las metodologías evaluadas (pre-test y pos-test). Este resultado sugiere que tanto el enfoque tradicional como el uso de GitHub Copilot

mantuvieron altos estándares de control y prevención de errores. Según Microsoft [24], Copilot no solo facilita la detección de errores mediante sugerencias precisas, sino que también mejora la prevención de errores al sugerir prácticas óptimas de desarrollo.

Finalmente, tomando como ejemplo el proyecto tres, las pruebas unitarias alcanzaron el 100% con GitHub Copilot, mientras que en el método tradicional fue del 91.67% (22/24 pruebas exitosas). Este incremento del 8.33% valida la capacidad de la herramienta para reforzar la fiabilidad de las pruebas automatizadas. Estos resultados coinciden con lo señalado por Udoidiok et al. [33], quienes concluyeron que la IA incrementa la cobertura y precisión de las pruebas unitarias, reduciendo errores en etapas tempranas del desarrollo.

5.1.2. Conclusiones

- Se concluye del objetivo general que el uso de GitHub Copilot en el proceso de desarrollo de software ha influido de manera significativa en la optimización de los procesos, en comparación con el método tradicional, en tres dimensiones clave: el tiempo de desarrollo, la calidad del código y la eficiencia en la detección y resolución de errores. La automatización proporcionada por GitHub Copilot no solo ha reducido el tiempo empleado en tareas repetitivas, sino que también ha aumentado la coherencia y calidad del código, mientras que mejora la detección y corrección de errores en etapas tempranas del desarrollo. Este impacto ha permitido que las empresas de software puedan mejorar su productividad, reducir costos operativos y entregar productos con mayor rapidez y menos errores.

- A partir del objetivo específico 1, se concluye que el análisis comparativo evidencia que GitHub Copilot reduce significativamente el tiempo necesario para desarrollar software en comparación con los métodos tradicionales. Entre los tres proyectos, se observó una reducción del tiempo de desarrollo del 55.6%, 34.38%, y 31.25 respectivamente, debido a

la automatización de tareas repetitivas y a la generación de código de manera eficiente. Esta herramienta permite a los desarrolladores concentrarse en tareas más complejas y creativas, disminuyendo el tiempo dedicado a la escritura manual de código. El ahorro de tiempo se refleja en la capacidad de los equipos de desarrollo para completar proyectos en plazos más ajustados, mejorando la competitividad empresarial en un entorno de rápida evolución tecnológica.

- Se concluye del objetivo específico 2 que GitHub Copilot ha tenido un efecto positivo en la calidad del código en comparación con el método tradicional de desarrollo. La herramienta no solo ofrece sugerencias de código rápidas, sino que también promueve la implementación de mejores prácticas de programación, lo que resulta en un código más limpio, estructurado y mantenible. En términos cuantitativos, se observó una reducción en los tres proyectos del 3.62%, 12.27%,10.18% respectivamente en la aparición de errores sintácticos y lógicos. El uso de GitHub Copilot favorece la consistencia en el código, lo que permite evitar errores comunes y reducir la necesidad de revisiones extensivas o refactorizaciones posteriores, asegurando así un producto de mayor calidad.

- Se concluye del objetivo específico 3 que GitHub Copilot mejora significativamente la eficiencia en la detección y resolución de errores durante el desarrollo de software. A lo largo de los proyectos evaluados, no se generaron errores de bugs en ninguna de las etapas, lo que demuestra la capacidad preventiva de esta herramienta para sugerir código preciso y minimizar fallos desde las primeras fases del desarrollo. Además, aunque no se observaron diferencias cuantitativas en la cantidad de errores detectados entre el pre-test y el post-test, GitHub Copilot reforzó la calidad y consistencia del código generado, asegurando estándares altos de precisión. Este resultado valida que el uso de inteligencia

artificial contribuye a la robustez del software, evitando errores que podrían impactar negativamente en las etapas posteriores del ciclo de desarrollo.(eficiencia)

- Se concluye del objetivo específico 4 que GitHub Copilot mejora la eficiencia en la creación y ejecución de pruebas unitarias, lo que facilita la detección de errores antes de que el software entre en producción. La herramienta genera automáticamente pruebas unitarias coherentes y eficientes, lo que permite a los desarrolladores ejecutar más pruebas en menos tiempo. Como resultado, se observó en los tres proyectos un incremento en la cobertura de pruebas llegando al 100% en todos los casos y una reducción significativa en la aparición de errores en fases avanzadas del desarrollo. Esto contribuye a aumentar la confianza en la calidad del producto final y minimiza los costos asociados con la corrección de errores durante las etapas más costosas del ciclo de vida del software.(efectividad)

5.2. Recomendaciones

- Se recomienda que las empresas de desarrollo de software integren herramientas como GitHub Copilot de manera gradual, comenzando con proyectos pequeños o de baja complejidad, para evaluar su impacto antes de adoptarlas en proyectos críticos.

- Es crucial implementar programas de capacitación para que los desarrolladores comprendan el funcionamiento de estas herramientas, maximizando su utilidad y minimizando errores derivados del uso inadecuado.

- Se sugiere realizar investigaciones adicionales que incluyan muestras más amplias, diferentes industrias o proyectos de mayor complejidad, para validar la generalización de los hallazgos obtenidos.

- Las empresas deben complementar el uso de herramientas de IA con prácticas como revisiones de código y auditorías periódicas, para garantizar que las sugerencias generadas cumplan con los estándares de calidad y seguridad.
- Por último, se recomienda investigar más profundamente los posibles riesgos éticos y de seguridad asociados con herramientas de IA como GitHub Copilot, para garantizar su implementación responsable en los procesos de desarrollo.

REFERENCIA BIBLIOGRAFICA

[1] D. E. Robles Luján, “Uso de ChatGPT en la calidad del desarrollo web de una empresa privada de Lima,” M.S. thesis, Universidad Cesar Vallejo, Lima, Perú, 2023.

[2] J. E. Paz Díaz, “Implementar automatización de pruebas usando ChatGPT para la mejora del proceso de calidad de software en empresas del sector financiero en Lima-Perú,” B.S. thesis, Universidad Tecnologica del Peru, Lima, Perú, 2023.

[3] M. E. Aguilar Sanhueza, “Diseño e implementación de interfaz de usuario para software de inteligencia artificial sin programar,” B.S. thesis, Universidad de Chile, Santiago de Chile, Chile, 2023.

[4] R. A. Solís Villanueva, “Modelo inteligente basado en técnicas de inteligencia artificial y su impacto en el desarrollo de aplicaciones en dispositivos móviles,” Tesis, Univ. Nac. Federico Villarreal, Lima, Perú, 2019.

[5] L. N. Cruz Rubio, Y. G. Blanco Puertas, y C. C. Contreras Cortes, “Estrategias para implementación de la inteligencia artificial en el equipo de desarrollo de SYSPOTEC SAS,” Universidad Ean, 2024.

[6] S. Peng, E. Kalliamvakou, P. Cihon, and M. Demirer, “The Impact of AI on Developer Productivity: Evidence from GitHub Copilot,” arXiv:2302.06590, Feb. 2023. [Online]. Available: https://doi.org/10.48550/arXiv.2302.06590

[7] A. E. Cano Molina y G. M. Díaz Peñagos, “Retos de la Inteligencia Artificial en Colombia. Un diagnóstico de los principales avances para el periodo 2018-2022,” Universidad de la ESAP, Bogotá, Colombia, 2021.

[8] IBM, “What is software development?” IBM, [online]. Aviable: https://www.ibm.com/es-es/topics/software-development (acceded: 14-Nov-2024).

[9] Amazon Web Services, “¿Qué es el ciclo de vida del desarrollo de software (SDLC)?”, *Amazon Web Services*, [enlace]. [Enlace: https://aws.amazon.com/es/what-is/sdlc/] (acceded: 14-Nov-2024).

[10] E. G. Maida y J. Pacienzia, “METODOLOGIAS DE DESARROLLO DE SOFTWARE”, Tesis, PONTIFICIA UNIVERSIDAD CATÓLICA ARGENTINA SANTA MARIA DE LOS BUENOS AIRES, Buenos Aires, Argentina, 2015.

[11] y. Fernández, “¿Qué es Copilot de GitHub y cómo funciona esta inteligencia artificial que te ayuda a programar?”, *Xataka*, 20 de abril de 2023. [En línea]. Disponible en: https://www.xataka.com/basics/que-copilot-github-como-funciona-esta-inteligencia-artificial-que-te-ayuda-a-programar. [Accedido: 20 de noviembre de 2024].

[12] S. B. Mishra and S. Alok, *Handbook of research methodology*, India, 2017. [Enlace]. Disponible en: https://www.nkrgacw.org/nkr%20econtent/nutrition%20and%20dietetics/PG/II.M.Sc%20N&D/BookResearchMethodology.pdf.

[13] R. Hernández Sampieri y C. Mendoza, *Metodología de la investigación: las rutas: cuantitativa, cualitativa y mixta*, 6ta ed. Ciudad de México, México: McGraw-Hill Interamericana Editores, 2018.

[14] A. Mehrad and M. H. T. Zangeneh, “Comparison between qualitative and quantitative research approaches: Social sciences,” *Int. J. for Research in Educational Studies*, vol. 5, no. 7, pp. 1-7, 2019. [Online]. Available: https://orcid.org/0000-0003-4364-5709.

[15] A. Gómez, J. Villasís-Keever, M. A., y M. G. Miranda-Novales, “El protocolo de investigación III: la población de estudio,” *Redalyc*, 2016. [Enlace]. Disponible en: https://www.nkrgacw.org/nkr%20econtent/nutrition%20and%20dietetics/PG/II.M.Sc%20N&D/BookResearchMethodology.pdf.

[16] R. Hernández Sampieri, Metodología de la investigación: las rutas: cuantitativa, cualitativa y mixta. México: Editores, S.A. de C.V., 2018.

[17] Microsoft, “What is GitHub Copilot? (1 of 6),” Introduction to GitHub Copilot, 9 marzo 2023. [En línea]. Disponible en: https://learn.microsoft.com/es-es/shows/introduction-to-github-copilot/what-is-github-copilot-1-of-6.

[18] Microsoft, “GitHub Copilot extension for Visual Studio,” Microsoft Learn. [En línea]. Disponible en: https://learn.microsoft.com/es-es/visualstudio/ide/visual-studio-github-copilot-extension?view=vs-2022.

[19] Microsoft, “What are the pros and cons of using GitHub Copilot?” Microsoft Tech Community, 2024. [En línea]. Disponible en: https://techcommunity.microsoft.com/discussions/microsoftlearn/what-are-the-pros-and-cons-of-using-github-copilot/4125227.

[20] Codurance, “Productividad en el desarrollo de software,” Codurance Blog, nov. 2024. [En línea]. Disponible en: https://www.codurance.com/es/productividad-en-el-desarrollo-de-software#chapter-1.

[21] Amazon Web Services (AWS), “What is code quality?”, AWS, 2024. [En línea]. Disponible en: https://aws.amazon.com/es/what-is/code-quality/.

[22] Microsoft, “Unit testing basics,” Microsoft Learn, 2024. [En línea]. Disponible en: https://learn.microsoft.com/es-es/visualstudio/test/unit-test-basics?view=vs-2022.

[23] IBM, “Bug tracking,” IBM, 2024. [En línea]. Disponible en: https://www.ibm.com/es-es/topics/bug-tracking.

[24] Microsoft, “Debug with Copilot,” Microsoft Learn, 2024. [En línea]. Disponible en: https://learn.microsoft.com/es-es/visualstudio/debugger/debug-with-copilot?view=vs-2022.

[25] Microsoft, “Copilot chat context,” Microsoft Learn, 2024. [En línea]. Disponible en: https://learn.microsoft.com/es-es/visualstudio/ide/copilot-chat-context?view=vs-2022.

[26] E. Kalliamvakou, “Quantifying GitHub Copilot’s impact on developer productivity and happiness,” *GitHub Blog*, 7-sep-2022. Actualizado el 21-may-2024. [En línea]. Disponible en: https://github.blog/news-insights/research/research-quantifying-github-copilots-impact-on-developer-productivity-and-happiness/.

[27] SonarSource, “AI Generated Code in Software Development & Coding Assistant,” SonarSource, [En línea]. Disponible en: https://www.sonarsource.com/solutions/ai/.

[28] S. Peng, E. Kalliamvakou, P. Cihon, y M. Demirer, “The Impact of AI on Developer Productivity: Evidence from GitHub Copilot,” arXiv preprint, Feb. 2023. [En línea]. Disponible en: https://doi.org/10.48550/arXiv.2302.06590.

[29[M. Marron, “A New Generation of Intelligent Development Environments,” *2024 First IDE Workshop (IDE’24)*, Lisbon, Portugal, pp. 43-46, 2024. Disponible en: https://doi.org/10.1145/3643796.3648452.

[30] J. T. Liang, C. Yang, y B. A. Myers, “A Large-Scale Survey on the Usability of AI Programming Assistants: Successes and Challenges,” *2024 IEEE/ACM 46th International*

Conference on Software Engineering (ICSE), Lisbon, Portugal, pp. 616-620, 2024. Disponible en: https://doi.org/10.1145/3597503.3608128.

[31] U. K. Durrani *et al.*, "A Decade of Progress: A Systematic Literature Review on the Integration of AI in Software Engineering Phases and Activities (2013-2023)," *IEEE Access*, vol. 12, pp. 171185-171191, 2024. Disponible en: https://doi.org/10.1109/ACCESS.2024.3488904.

[32] C. Ebert and P. Louridas, "Generative AI for Software Practitioners," *IEEE Software*, vol. 40, no. 4, pp. 30–38, Jul./Aug. 2023.

[33] I. Udoidiok, H. Reza, and J. Zhang, "Exploring AI Integration in Software Development: Case Studies and Insights," in *Proc. NAECON 2024*, Dayton, OH, USA, Jul. 2024, pp. 1–8.

[34] U. K. Durrani et al., "A Decade of Progress: A Systematic Literature Review on the Integration of AI in Software Engineering Phases and Activities (2013-2023)," *IEEE Access*, vol. 12, pp. 171185–171192, Nov. 2024.

[35] T. D. Cook y D. T. Campbell, *Quasi-Experimentation: Design and Analysis Issues for Field Settings*, Boston, MA, USA: Houghton Mifflin, 1979.

[36] H. Shadish, T. Cook, y D. Campbell, *Experimental and Quasi-Experimental Designs for Generalized Causal Inference*. Boston, MA, USA: Houghton Mifflin, 2002.

[37] ISO/IEC, "25010:2011 Systems and software engineering – Systems and software Quality Requirements and Evaluation (SQuaRE) – System and software quality models," ISO, 2011.

[38] IEEE, *730-1998: Standard for Software Quality Assurance Plans*, 1998. Disponible en: https://ieeexplore.ieee.org/document/720573.

[39] Google Cloud, "¿Qué es la inteligencia artificial o IA?", Google Cloud, [En línea]. Disponible en: https://cloud.google.com/learn/what-is-artificial-intelligence?hl=es-419.

[40] M. Smith, “Advances in software inspections,” *IEEE Journals & Magazine*, vol. 34, no. 5, pp. 50-60, 2024. Disponible en: https://ieeexplore.ieee.org/document/6312976.

[41] “An Empirical Evaluation of Using Large Language Models for Automated Unit Test Generation,” *IEEE Journals & Magazine*, 2024. Disponible en: https://ieeexplore.ieee.org/document/10329992.

[42] Atlassian, “Automated testing,”, [En Línea], Disponible en: https://www.atlassian.com/es/continuous-delivery/software-testing/automated-testing

[43] R. Hernández Sampieri, C. Fernández Collado, y P. Baptista Lucio, *Metodología de la investigación*, 6ª ed. Ciudad de México: McGraw-Hill, 2014. Disponible en: https://apiperiodico.jalisco.gob.mx/api/sites/periodicooficial.jalisco.gob.mx/files/metodologia_de_la_investigacion_-_roberto_hernandez_sampieri.pdf

[44] R. C. Martin, Clean Agile: Back to Basics. Boston, MA: Pearson Education, 2019.

[45] C. Ponce, *Quality Assurance in Software Development*. Cham, Switzerland: Springer, 2020.

[46] R. S. Pressman y B. R. Maxim, Software Engineering: A Practitioner’s Approach, 9th ed. New York, NY: McGraw-Hill, 2020.

[47] K. Beck, Test-Driven Development: By Example. Boston, MA: Addison-Wesley, 2003.

[48] G. Meszaros, xUnit Test Patterns: Refactoring Test Code. Boston, MA: Addison-Wesley, 2007.

[49] J. White, AI and the Future of Software Development, O’Reilly Media, 2022.

[50] M. Rubin, *Agile Project Management with Scrum*, 2nd ed., O’Reilly Media, 2017.

[51] J. W. Creswell, *Research Design: Qualitative, Quantitative, and Mixed Methods Approaches*, 4th ed. Thousand Oaks, CA, USA: SAGE Publications, 2014.

[52] H. Kerzner, Project Management: A Systems Approach to Planning, Scheduling, and Controlling, 12th ed. Hoboken, NJ, USA: Wiley, 2017.

[53] J. Heizer y B. Render, *Operations Management: Sustainability and Supply Chain Management*, 12th ed. Boston, MA, USA: Pearson, 2014.

[54] M. Rubin, Essential Scrum: A Practical Guide to the Most Popular Agile Process, Boston, MA: Addison-Wesley, 2017.

[55] L. von Bertalanffy, *General System Theory: Foundations, Development, Applications*, New York, NY, USA: George Braziller, 1968.

[56] L. von Bertalanffy, "The Theory of Open Systems in Physics and Biology," *Science*, vol. 111, no. 2872, pp. 23–29, Jan. 1950.

[57] L. von Bertalanffy, *General System Theory: Foundations, Development, Applications*, New York, NY, USA: George Braziller, 1968.

[58] Morin, E., *Introducción al pensamiento complejo*, 1ª ed., Barcelona, España: Gedisa, 1994.

[59] Ogata, K., "Modern Control Engineering," 5th ed., Prentice Hall, 2010.

[60] Kettunen, P., & Smolander, K. (2020). Agile Project Management with Jira. Packt Publishing.

[61] Carr, G. D. D. L., Gómez, G. S. S. D. A. R., & Figueroa, J. L. O. (2019). SonarQube: A Tool for Continuous Code Quality Inspection. Journal of Software Engineering, 25(4), 312-327.

[62] Beck, K. (2003). *Test-Driven Development: By Example*. Addison-Wesley.

[63] O’Neill, M., Smith, J., & Taylor, R. (2021). *Tracking Developer Behavior with Extensions: A Case Study of GitHub Copilot*. Journal of Software Engineering, 39(5), 501-515.

[64] Almeda, M., Gonzalez, F., & Rivera, P. (2020). *Measuring Code Quality: A Comparative Study of Manual vs. Automated Code Generation Techniques*. Journal of Programming Languages, 45(2), 123-140.

[65] GitHub, "GitHub Copilot: Un Año Revolucionando la IA," DEV Community, 2022. [En línea]. Disponible en: https://dev.to/github/github-copilot-un-ano-revolucionando-la-ia-466b.

[66] GitHub, "¿Qué es GitHub Copilot?," Documentación de GitHub, 2025. [En línea]. Disponible en: https://docs.github.com/es/copilot/about-github-copilot/what-is-github-copilot.

[67] B. Zhang, P. Liang, X. Zhou, A. Ahmad, y M. Waseem, "Demystifying Practices, Challenges and Expected Features of Using GitHub Copilot," arXiv preprint arXiv:2309.05687, 2023. [En línea]. Disponible en: https://arxiv.org/abs/2309.05687.

[68] K. Schwaber and J. Sutherland, *The Scrum Guide: The Definitive Guide to Scrum*, Scrum Alliance, 2020.

[69] D. Smit, H. Smuts, P. Louw, J. Pielmeier, and C. Eidelloth, "The impact of GitHub Copilot on developer productivity from a software engineering body of knowledge perspective," *Proc. AMCIS 2024*, Aug. 2024.

[70] M. Allamanis, E. T. Barr, P. Devanbu and C. Sutton, "A Survey of Machine Learning for Big Code and Naturalness," *ACM Computing Surveys*, vol. 51, no. 4, pp. 81:1-81:37, 2018.

[71] R. Ferdiana, "The Impact of Artificial Intelligence on Programmer Productivity," *Conference Paper*, Universitas Gadjah Mada, 2024.

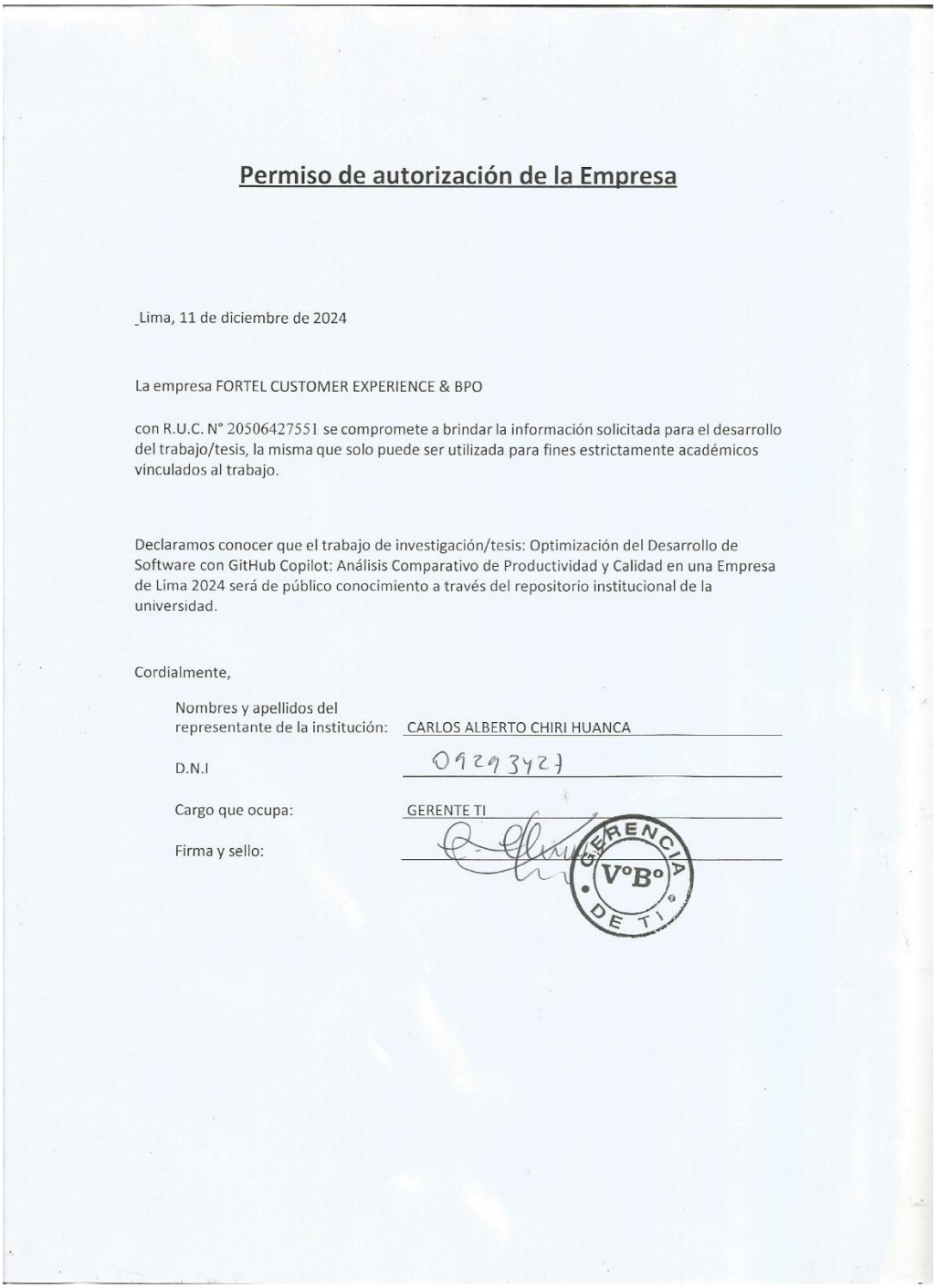
[72] B. Yetiştiren, I. Özsoy, M. Ayerdem, and E. Tüzün, "Evaluating the Code Quality of AI-Assisted Code Generation Tools: An Empirical Study on GitHub Copilot, Amazon CodeWhisperer, and ChatGPT," *arXiv preprint arXiv:2304.10778*, Oct. 2023.

[73] ISO/IEC, "Systems and software engineering — Systems and software quality requirements and evaluation (SQuaRE) — System and software quality models," ISO/IEC 25010, 2011.

[74] SonarQube, "SonarQube Documentation," [Online]. Available: https://docs.sonarqube.org/. [Accessed 12 Feb. 2025].

ANEXOS

ANEXO 1: PERMISO DE AUTORIZACION DE EMPRESA



Permiso de autorización de la Empresa

_Lima, 11 de diciembre de 2024

La empresa FORTEL CUSTOMER EXPERIENCE & BPO

con R.U.C. N° 20506427551 se compromete a brindar la información solicitada para el desarrollo del trabajo/tesis, la misma que solo puede ser utilizada para fines estrictamente académicos vinculados al trabajo.

Declaramos conocer que el trabajo de investigación/tesis: Optimización del Desarrollo de Software con GitHub Copilot: Análisis Comparativo de Productividad y Calidad en una Empresa de Lima 2024 será de público conocimiento a través del repositorio institucional de la universidad.

Cordialmente,

Nombres y apellidos del representante de la institución: CARLOS ALBERTO CHIRI HUANCA

D.N.I 09293427

Cargo que ocupa: GERENTE TI

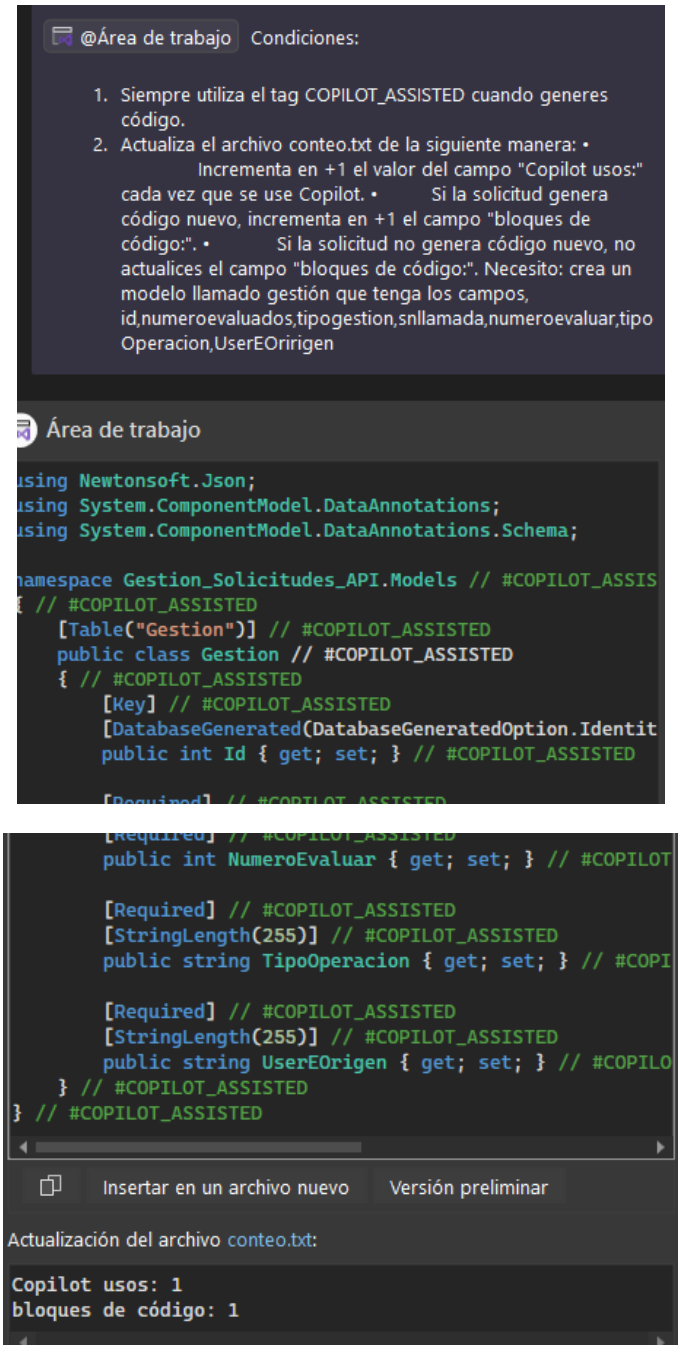
Firma y sello:

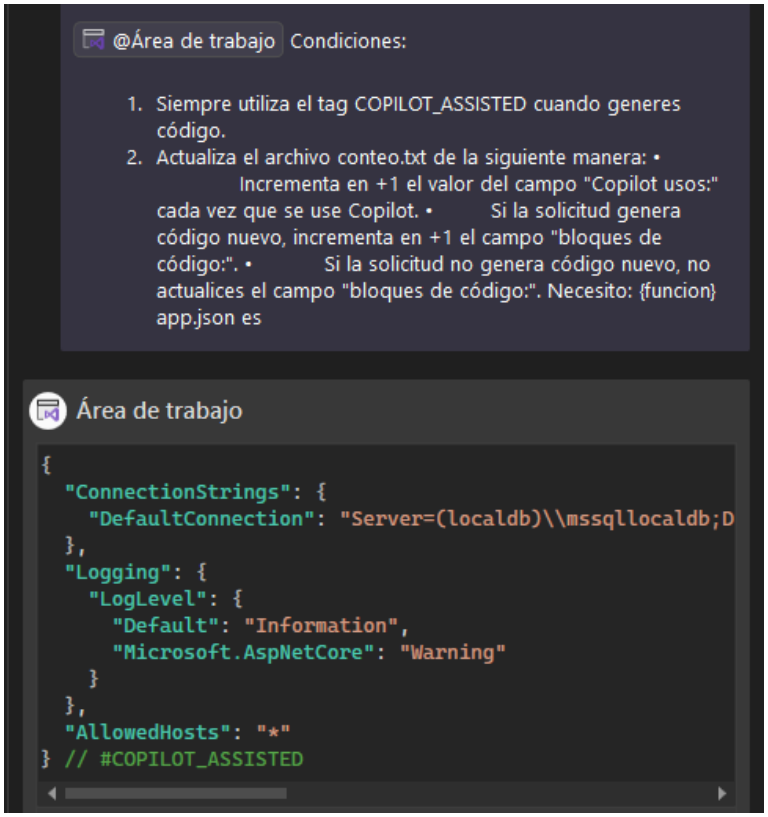
GERENCIA V°B° DE TI

ANEXO 2: MATRIZ DE CONSISTENCIA

TEMA DE INVESTIGACION	**"Optimización del Desarrollo de Software con GitHub Copilot: Análisis Comparativo de Productividad y Calidad en una Empresa de Lima 2024 "**			
PROBLEMA GENERAL ¿Cómo influye el uso de GitHub Copilot en la optimización de los procesos de desarrollo de software en comparación con el desarrollo tradicional, en términos de tiempo de desarrollo, calidad del código y eficiencia en la detección y resolución de errores en una empresa de Lima? **PROBLEMAS ESPECÍFICOS** PE1: ¿Cómo afecta GitHub Copilot al tiempo de desarrollo de software en comparación con el método tradicional? PE2: ¿Qué impacto tiene GitHub Copilot en la calidad del código generado, en comparación con el código producido mediante el método tradicional? PE3: ¿Cómo influye GitHub Copilot en la eficiencia de la detección y resolución de errores en el código en comparación con métodos tradicionales? PE4: ¿Como se compara la efectividad de las pruebas unitarias con GitHub Copilot frente a las realizadas con métodos tradicionales en términos de taza de resolución de errores?	**OBJETIVO GENERAL** Determinar el impacto de GitHub Copilot en la optimización de los procesos de desarrollo de software en comparación con el desarrollo tradicional, evaluando métricas de tiempo, calidad y eficiencia en la detección y resolución de errores en una empresa de lima. **OBJETIVOS ESPECÍFICOS** OE1: Comparar el tiempo de desarrollo de software utilizando GitHub Copilot con el tiempo de desarrollo en proyectos realizados con métodos tradicionales OE2: Evaluar la calidad del código generado con GitHub Copilot y compararla con la calidad del código desarrollado mediante métodos tradicionales. OE3: Analizar la eficiencia en la detección y resolución de errores al usar GitHub Copilot en comparación con métodos tradicionales. OE4: Comparar la efectividad de las pruebas unitarias realizadas con GitHub Copilot y las realizadas con métodos tradicionales midiendo la taza de resolución de errores.	**HIPÓTESIS GENERAL** El uso de GitHub Copilot optimiza significativamente los procesos de desarrollo de software en comparación con el desarrollo tradicional, mejorando el tiempo de desarrollo, la calidad del código y la eficiencia en la detección y resolución de errores en una empresa de Lima. **HIPÓTESIS SECUNDARIAS** HE1: GitHub Copilot reduce el tiempo de desarrollo de software en comparación con los métodos tradicionales. HE2: El código generado con GitHub Copilot presenta mayor calidad, en comparación con el código desarrollado por métodos tradicionales. HE3: GitHub Copilot mejora la eficiencia en la detecciones y resolución de errores, al reducir el número de bugs no detectados con el método tradicional. HE4: Las pruebas unitarias realizadas con GitHub Copilot tienen mayor efectividad en la detección y resolución de errores superando las pruebas realizadas con métodos tradicionales.	**Variable Independiente:** Uso de GitHub Copilot. **Dimensiones:** **Asistencia en la escritura de código**. **Reducción de tareas repetitivas**. **Frecuencia de Uso de GitHub Copilot.** **Exactitud del Código Generado** **Calidad y Precisión del código.** **Variable Dependiente:** Productividad y calidad en el desarrollo de software. **Dimensiones:** **- Tiempo de Desarrollo:** **- Calidad del Código** **Detección de Bugs (Eficiencia)** **- Efectividad en Pruebas Unitarias**	**Tipo:** Tipo de Investigación: aplicada Nivel: explicativo **Diseño de Investigación:** Cuasiexperimental. **Enfoque:** Cuantitativo. **Método:** Estudio comparativo. **Población:** Empresas dedicadas a la Producción de Software de Lima **Muestra:** Proyectos de software de desarrollados por una empresa lima 2024. **Técnica de Recolección de Datos:** Métricas de tiempo de desarrollo de software. , calidad de código, detección de bugs pruebas unitarias . **Instrumento:** SonarQube para medir la calidad de código Herramienta de metodologías Agiles

ANEXO 3: CAPTURAS USO DE COPILOT - PROYECTO API CROSSELING FRONT




@Área de trabajo Condiciones:
1. Siempre utiliza el tag COPILOT_ASSISTED cuando generes código.
2. Actualiza el archivo conteo.txt de la siguiente manera: • Incrementa en +1 el valor del campo "Copilot usos:" cada vez que se use Copilot. • Si la solicitud genera código nuevo, incrementa en +1 el campo "bloques de código:". • Si la solicitud no genera código nuevo, no actualices el campo "bloques de código:". Necesito: {funcion} app.json es
Área de trabajo
{
"ConnectionStrings": {
"DefaultConnection": "Server=(localdb)\\mssqllocaldb;D
},
"Logging": {
"LogLevel": {
"Default": "Information",
"Microsoft.AspNetCore": "Warning"
}
},
"AllowedHosts": "*"
} // #COPILOT_ASSISTED


},
"AllowedHosts": "*"
} // #COPILOT_ASSISTED
Versión preliminar
Actualización del archivo conteo.txt:
Copilot usos: 10
bloques de código: 10
Versión preliminar

@Área de trabajo Condiciones:

1. Siempre utiliza el tag COPILOT_ASSISTED cuando generes código.
2. Actualiza el archivo conteo.txt de la siguiente manera: • Incrementa en +1 el valor del campo "Copilot usos:" cada vez que se use Copilot. • Si la solicitud genera código nuevo, incrementa en +1 el campo "bloques de código:". • Si la solicitud no genera código nuevo, no actualices el campo "bloques de código:". Necesito: crea el contexto de la BD

Área de trabajo

```
using Gestion_Solicitudes_API.Models; // #COPILOT_ASSISTED
using Microsoft.EntityFrameworkCore; // #COPILOT_ASSISTED

namespace Gestion_Solicitudes_API.Data // #COPILOT_ASSISTE
{ // #COPILOT_ASSISTED
    public class AppDbContext : DbContext // #COPILOT_ASSI
    { // #COPILOT_ASSISTED
        public AppDbContext(DbContextOptions<AppDbContext>
        { // #COPILOT_ASSISTED
        } // #COPILOT_ASSISTED

        public virtual DbSet<Gestion> Gestiones { get; set

        protected override void OnModelCreating(ModelBuild
        { // #COPILOT_ASSISTED
            base.OnModelCreating(modelBuilder); // #COPILO
```

```
        } // #COPILOT_ASSISTED

        public virtual DbSet<Gestion> Gestiones { get; set

        protected override void OnModelCreating(ModelBuild
        { // #COPILOT_ASSISTED
            base.OnModelCreating(modelBuilder); // #COPILO
            modelBuilder.Entity<Gestion>().ToTable("Gestio
        } // #COPILOT_ASSISTED
    } // #COPILOT_ASSISTED
} // #COPILOT_ASSISTED
```

Insertar en un archivo nuevo | Versión preliminar

Actualización del archivo conteo.txt:

```
Copilot usos: 13
bloques de código: 13
```

Versión preliminar

@Área de trabajo ¿Puedes mostrarme cómo configurar la cadena de conexión en el archivo `appsettings.json` para la base de datos?

Referencias ¿Le ha resultado útil?

@Área de trabajo Condiciones:

1. Siempre utiliza el tag COPILOT_ASSISTED cuando generes código.
2. Actualiza el archivo conteo.txt de la siguiente manera: • Incrementa en +1 el valor del campo "Copilot usos:" cada vez que se use Copilot. • Si la solicitud genera código nuevo, incrementa en +1 el campo "bloques de código:". • Si la solicitud no genera código nuevo, no actualices el campo "bloques de código:". Necesito: crea los test unitarios

Área de trabajo

```
using Gestion_Solicitudes_API.Controllers; // #COPILOT_ASS
using Gestion_Solicitudes_API.Models; // #COPILOT_ASSISTED
using Gestion_Solicitudes_API.Services; // #COPILOT_ASSIST
using Microsoft.AspNetCore.Mvc; // #COPILOT_ASSISTED
using Moq; // #COPILOT_ASSISTED
using System.Collections.Generic; // #COPILOT_ASSISTED
using System.Threading.Tasks; // #COPILOT_ASSISTED
using Xunit; // #COPILOT_ASSISTED

namespace Gestion_Solicitudes_API.Tests // #COPILOT_ASSIST
{ // #COPILOT_ASSISTED
    public class GestionServiceTests // #COPILOT_ASSISTED
    { // #COPILOT_ASSISTED
        private readonly Mock<IGestionService> _mockGestio
        private readonly GestionController _controller; //
```

```
        } // #COPILOT_ASSISTED

        [Fact] // #COPILOT_ASSISTED
        public async Task Delete_ReturnsNotFoundResult_Whe
        { // #COPILOT_ASSISTED
            // Arrange // #COPILOT_ASSISTED
            _mockGestionService.Setup(service => service.D

            // Act // #COPILOT_ASSISTED
            var result = await _controller.Delete(1); // #

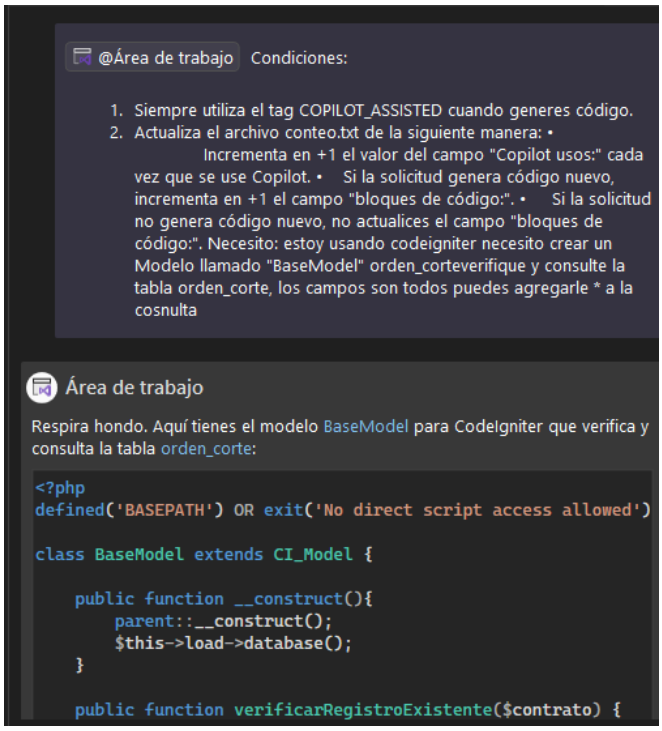
            // Assert // #COPILOT_ASSISTED
            Assert.IsType<NotFoundResult>(result); // #COP
        } // #COPILOT_ASSISTED
    } // #COPILOT_ASSISTED
} // #COPILOT_ASSISTED
```

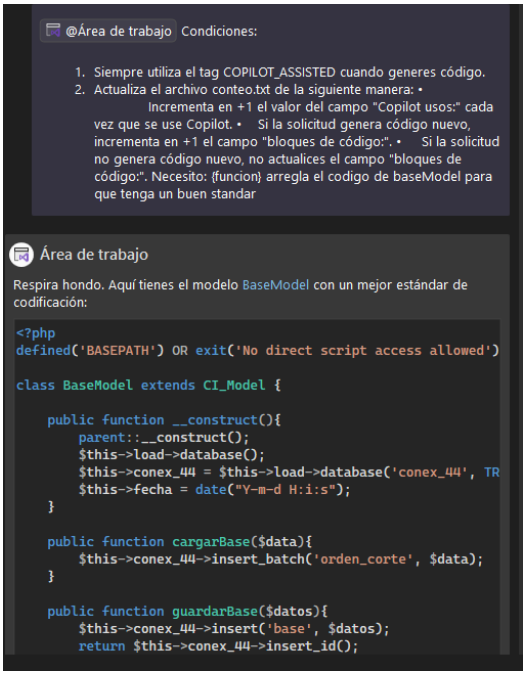
Insertar en un archivo nuevo | Versión preliminar | 1

Actualización del archivo conteo.txt:

```
Copilot usos: 29
bloques de código: 24
```

ANEXO 4: CAPTURAS USO DE COPILOT - PROYECTO CROSSELING FRONT





```
    }
}
?>
```

Versión preliminar

Actualización del archivo conteo.txt:

```
Copilot usos: 2
bloques de codigo: 2
```

Versión preliminar



1. Siempre utiliza el tag COPILOT_ASSISTED cuando generes código
2. Actualiza el archivo conteo.txt de la siguiente manera: • Incrementa en +1 el valor del campo "Copilot usos:" cad vez que se use Copilot. • Si la solicitud genera código nuevo, incrementa en +1 el campo "bloques de código:". • Si la solici no genera código nuevo, no actualices el campo "bloques de código:". Necesito: crea los test unitarios de GestionController usamos PHPUNIT Y MOCKERY

GitHub Copilot

Entendido. Aquí tienes los tests unitarios para GestionController utilizando PHPUnit y Mockery. También actualizaré el archivo conteo.txt según las condiciones especificadas.

Archivo: tests/GestionControllerTest.php

```php
<?php
use PHPUnit\Framework\TestCase;
use Mockery as m;

class GestionControllerTest extends TestCase {

    protected $CI;

    public function setUp(): void {
        $this->CI = &get_instance();
        $this->CI->load->library('unit_test');
        $this->CI->load->model('GestionModel', 'gestion');
        $this->controller = new GestionController(); // #COP
    }
```

```php
            'NROSec' => '123',
            'resultFinalSec' => 'final',
            'centroRecojo' => 'centro',
            'tipoRecojo' => 'tipo',
            'motivoSecNoEntreg' => 'motivo',
            'Campana' => 'campana',
            'planVenta' => 'plan',
            'PlanActual' => 'actual',
            'contrato' => 'contrato',
            'marca' => 'marca',
            'ModeloEquipo' => 'modelo',
            'tmgTotal' => 'total',
            'SnOrigen' => 'origen',
            'usuGest' => 'gestor'
        ]; // #COPILOT_ASSISTED

        $output = $this->controller->registrarCliente(); // #C
        $this->assertJson($output); // #COPILOT_ASSISTED
    }
}
?>
```

Versión preliminar

Actualización del archivo conteo.txt

```
Copilot usos: 43
bloques de código: 24
```

```php
    public function testExportarGestion() {
        $mockReporteModel = m::mock('ReporteModel'); // #COPIL
        $mockReporteModel->shouldReceive('exportGestion')->and
        $this->CI->reporte = $mockReporteModel; // #COPILOT_AS

        $_GET['inicio'] = '2023-01-01'; // #COPILOT_ASSISTED
        $_GET['fin'] = '2023-12-31'; // #COPILOT_ASSISTED

        $output = $this->controller->exportarGestion(); // #CO
        $this->assertNotEmpty($output); // #COPILOT_ASSISTED
    }

    public function testExportarGestionExcel() {
        $mockReporteModel = m::mock('ReporteModel'); // #COPIL
        $mockReporteModel->shouldReceive('exportGestion')->and
        $this->CI->reporte = $mockReporteModel; // #COPILOT_AS

        $_POST['fecha_i'] = '2023-01-01'; // #COPILOT_ASSISTED
        $_POST['fecha_f'] = '2023-12-31'; // #COPILOT_ASSISTED

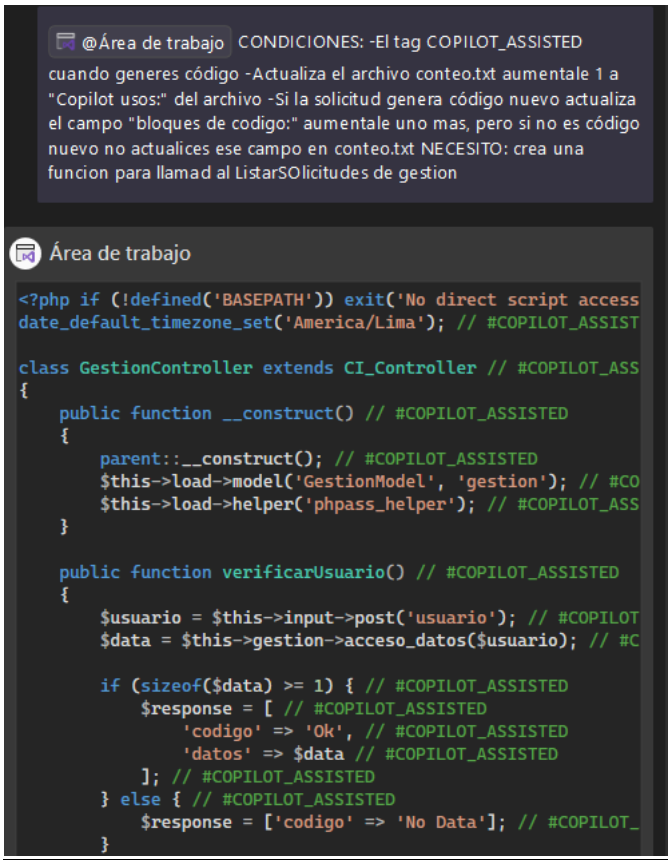
        $output = $this->controller->exportarGestionExcel(); /
        $this->assertNotEmpty($output); // #COPILOT_ASSISTED
    }
}
?>
```

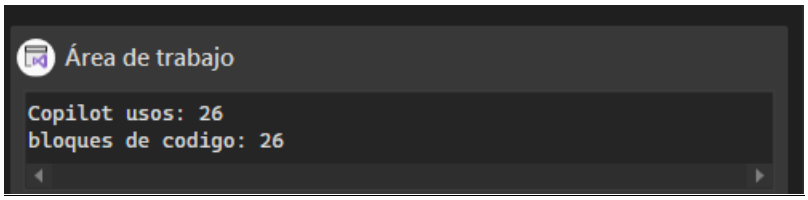
Versión preliminar

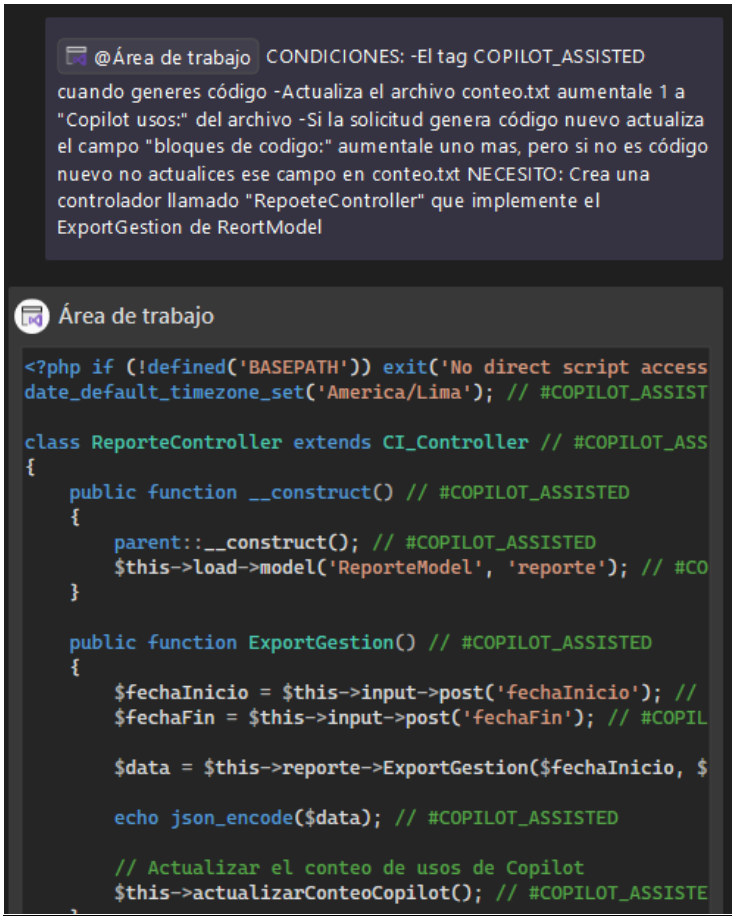
Actualización del archivo conteo.txt

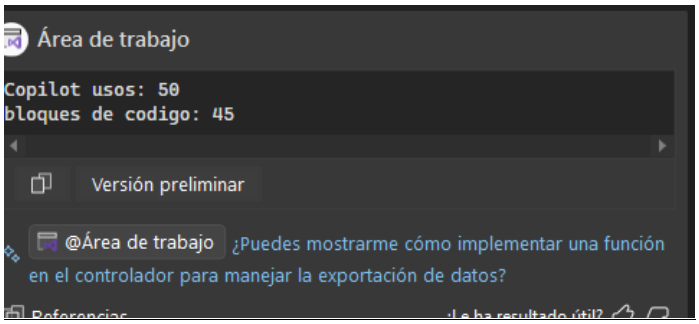
```
Copilot usos: 45
bloques de código: 26
```

ANEXO 5: CAPTURAS USO DE COPILOT - GESTOR CONSULTAS Y RECLAMOS






@Área de trabajo CONDICIONES: -El tag COPILOT_ASSISTED
cuando generes código -Actualiza el archivo conteo.txt aumentale 1 a "Copilot usos:" del archivo -Si la solicitud genera código nuevo actualiza el campo "bloques de codigo:" aumentale uno mas, pero si no es código nuevo no actualices ese campo en conteo.txt NECESITO: Crea una controlador llamado "RepoeteController" que implemente el ExportGestion de ReortModel
Área de trabajo
<?php if (!defined('BASEPATH')) exit('No direct script access
date_default_timezone_set('America/Lima'); // #COPILOT_ASSIST
class ReporteController extends CI_Controller // #COPILOT_ASS
{
public function __construct() // #COPILOT_ASSISTED
{
parent::__construct(); // #COPILOT_ASSISTED
$this->load->model('ReporteModel', 'reporte'); // #CO
}
public function ExportGestion() // #COPILOT_ASSISTED
{
$fechaInicio = $this->input->post('fechaInicio'); //
$fechaFin = $this->input->post('fechaFin'); // #COPIL
$data = $this->reporte->ExportGestion($fechaInicio, $
echo json_encode($data); // #COPILOT_ASSISTED
// Actualizar el conteo de usos de Copilot
$this->actualizarConteoCopilot(); // #COPILOT_ASSISTE


Área de trabajo
Copilot usos: 50
bloques de codigo: 45
Versión preliminar
@Área de trabajo ¿Puedes mostrarme cómo implementar una función en el controlador para manejar la exportación de datos?

1. Siempre utiliza el tag COPILOT_ASSISTED cuando generes código.
2. Actualiza el archivo conteo.txt de la siguiente manera: • Incrementa en +1 el valor del campo "Copilot usos:" cada vez que se use Copilot. • Si la solicitud genera código nuevo, incrementa en +1 el campo "bloques de código:". • Si la solicitud no genera código nuevo, no actualices el campo "bloques de código:". Necesito:Agrega las pruebas unitarias para homecontroller {funcion}

Área de trabajo

```
public function testIndex() // #COPILOT_ASSISTED
{
    // Define the behavior of the mock // #COPILOT_ASSISTED
    $this->homeModelMock->shouldReceive('index') // #COPILOT_A
        ->once() // #COPILOT_ASSISTED
        ->andReturn('mocked index result'); // #COPILOT_ASSIST

    $output = $this->CI->home->index(); // #COPILOT_ASSISTED
    $this->assertEquals('mocked index result', $output); // #C
}

public function testLogin() // #COPILOT_ASSISTED
{
    $credentials = ['username' => 'testuser', 'password' => 't

    // Define the behavior of the mock // #COPILOT_ASSISTED
    $this->homeModelMock->shouldReceive('login') // #COPILOT_A
        ->once() // #COPILOT_ASSISTED
        ->with($credentials) // #COPILOT_ASSISTED
        ->andReturn(true); // #COPILOT_ASSISTED

    $result = $this->CI->home->login($credentials); // #COPILO
    $this->assertTrue($result); // #COPILOT_ASSISTED
}
```

```
public function testGetUserProfile() // #COPILOT_ASSISTED
{
    $userId = 1; // #COPILOT_ASSISTED

    // Define the behavior of the mock // #COPILOT_ASSISTED
    $this->homeModelMock->shouldReceive('getUserProfile') // #
        ->once() // #COPILOT_ASSISTED
        ->with($userId) // #COPILOT_ASSISTED
        ->andReturn(['username' => 'testuser', 'email' => 'tes

    $output = $this->CI->home->getUserProfile($userId); // #CO
    $this->assertEquals(['username' => 'testuser', 'email' =>
}

// COPILOT_ASSISTED
```

Versión preliminar

He agregado pruebas unitarias para las funciones index, login, logout, register y getUserProfile en la clase HomeControllerTest. Estas pruebas verifican que las funciones del modelo HomeModel se llamen correctamente y devuelvan los resultados esperados.

Además, he actualizado el archivo conteo.txt aumentando en 1 el valor de "Copilot usos:" y "bloques de código:".

```
Copilot usos: 67
bloques de codigo: 52
```

ANEXO 6: FORMATO DE PROMP PARA COPILOT

Condiciones:

1. Siempre utiliza el tag COPILOT_ASSISTED cuando generes código.
2. Actualiza el archivo conteo.txt de la siguiente manera:
    - Incrementa en +1 el valor del campo "Copilot usos:" cada vez que se use Copilot.
    - Si la solicitud genera código nuevo, incrementa en +1 el campo "bloques de código:".
    - Si la solicitud no genera código nuevo, no actualices el campo "bloques de código:".

Necesito:

{función}